

UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

GENERACIÓN DE MALLAS DE POLIEDROS MEDIANTE CAVIDADES

TESIS PARA OPTAR AL GRADO DE
MAGÍSTER EN CIENCIAS, MENCIÓN COMPUTACIÓN

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN COMPUTACIÓN

VICENTE IGNACIO GONZÁLEZ BECERRA

PROFESORA GUÍA:
NANCY HITSCHFELD KAHLER

PROFESOR CO-GUÍA:
SERGIO SALINAS

MIEMBROS DE LA COMISIÓN:
PENDIENTE

SANTIAGO DE CHILE
2026

RESUMEN DE LA TESIS PARA OPTAR AL TÍTULO DE INGENIERO CIVIL EN COMPUTACIÓN Y MAGÍSTER EN CIENCIAS, MENCIÓN COMPUTACIÓN
POR: VICENTE IGNACIO GONZÁLEZ BECERRA
FECHA: 2026
PROF. GUIA: NANCY HITSCHFELD KAHLER

GENERACIÓN DE MALLAS DE POLIEDROS MEDIANTE CAVIDADES

Esta tesis estudia la generación indirecta de mallas poliédricas mediante la agrupación de tetraedros de una malla de entrada. Se propone una estrategia basada en cavidades definidas operacionalmente a partir de la circunsfera de un tetraedro semilla. El objetivo es investigar si esta forma de agrupación permite obtener celdas de calidad y un procedimiento eficiente, en comparación con otras variantes de la familia Polylla. La eficiencia, la robustez y la adecuación para métodos numéricos se plantean como hipótesis por contrastar.

Se revisan fundamentos geométricos, calidad, estructuras de datos y paralelización. La implementación secuencial ordena candidatos por radio creciente y recorre vecinos por cara; su aceptación compara circuncentros con la esfera de la semilla. Se documentan propietarios y fronteras, conservando la subdivisión tetraédrica junto con las agrupaciones.

El análisis diferencia el comportamiento activo de las capacidades aún incompletas. La reparación provisional no interviene en el operador principal, y la integración de Face y Edge en una interfaz común permanece pendiente. Se examinan además el cálculo de kernel y las razones geométricas disponibles, explicitando las condiciones de interpretación y las validaciones necesarias antes de usar sus exportaciones como evidencia experimental.

Se establece un diseño experimental que relaciona las preguntas de investigación con métricas de partición, reducción de celdas, kernel, tiempo y memoria. El protocolo contempla entradas locales inventariadas, familias controladas por generar y comparadores sujetos a integración.

Pendiente. Resultados del resumen: ejecutar el protocolo del capítulo 5 una vez validados entradas, métricas y comparadores; reemplazar este bloque por una frase con reducción, fracción de kernel, tiempo y memoria, indicando entradas y alcance de comparación. Si la hipótesis no se confirma, describir los límites observados sin atribuir mejoras inexistentes.

Se propone además paralelizar cálculos locales, resolver conflictos de propiedad y construir una salida aplanada. Su corrección y desempeño requieren desarrollo posterior.

Tabla de Contenido

1. Introducción **1**

1.1. Problema 1
1.2. Hipótesis 2
1.3. Preguntas de investigación 2
1.4. Objetivos 2
1.5. Estructura de la Tesis 3

2. Estado del arte **4**

2.1. Introducción 4
2.2. Métodos numéricos sobre mallas poliédricas 5
2.3. Calidad de mallas 10
2.4. Fundamentos geométricos: Delaunay, Voronoi y cavidades 14
2.5. Generación de mallas tetraédricas 19
2.6. Generación de mallas poliédricas 23
2.7. Estructuras de datos para mallas 30
2.8. Paralelización de algoritmos de mallas basados en cavidades 32
2.9. Síntesis y posicionamiento de la tesis 39

3. Trabajo anterior **43**

3.1. Polylla en dos dimensiones 43
3.2. GPolylla y la representación compacta 45
3.3. Extensión tridimensional 46
3.4. Criterios de unión y resultados publicados 47
3.5. Limitaciones y motivación del algoritmo de cavidades 49

4. Algoritmo de cavidades **51**

4.1. Generalización y alcance 51
4.2. Definiciones operacionales 52
4.3. Selección de semillas y construcción 53

4.4. Reparación y cobertura de pruebas 55
4.5. Representación de la malla y objetivo paralelo 56
4.6. Arquitectura y flujo de datos 57
4.7. Métricas implementadas y su interpretación 58
4.8. Visualización y trabajo de integración 60
4.9. Propuesta de versión paralela 60

5. Diseño experimental 62

5.1. Alcance y preguntas de evaluación 62
5.2. Métricas y reglas de interpretación 63
5.3. Conjunto de entradas y comparadores 63
5.4. Protocolo reproducible 64
5.5. Tablas previstas y criterio de contraste 65

6. Conclusión 67

6.1. Síntesis y alcance de la contribución 67
6.2. Respuestas a las preguntas de investigación 67
6.3. Trabajo futuro y balance de la hipótesis 68

Bibliografía 69

Índice de Tablas

Tabla 1 Hipótesis geométricas sobre las celdas en los métodos politopales revisados, y tipo de mallas usadas en su validación. 8
Tabla 2 Generadores de mallas tetraédricas revisados. 20
Tabla 3 Taxonomía de los métodos de generación de mallas poliédricas revisados. . . . 24
Tabla 4 Estrategias de resolución de conflictos en la generación paralela de mallas. La columna «Determinismo» indica lo que cada trabajo reporta sobre la reproducibilidad de su resultado. 36
Tabla 5 Posicionamiento de la propuesta de esta tesis respecto de los enfoques de generación poliédrica más cercanos. 40
Tabla 6 Valores totales reportados en la tabla 1 de [1] para los dos criterios de Face. 48
Tabla 7 Relación entre preguntas y verificaciones propuestas. 62
Tabla 8 Inventario declarado en los pares locales de archivos TetGen. 64
Tabla 9 Plantilla de reducción y calidad; sin resultados. 65
Tabla 10 Plantilla de recursos: tiempos en ms y memoria en MiB. 66
Tabla 11 Plantilla condicionada a implementar y validar paralelismo. 66

Índice de Ilustraciones

Figura 1 *Kernel* (azul) de tres polígonos, como análogo bidimensional del *kernel* de un poliedro (elaboración propia). (a) En un polígono convexo, el *kernel* coincide con el polígono. (b) Un polígono no convexo estrellado tiene *kernel* no vacío. (c) Un polígono que no es estrellado tiene *kernel* vacío. El *kernel* se obtiene intersectando los semiplanos interiores de todas las aristas. 13

Figura 2 Análogo bidimensional de los conceptos de esta sección (elaboración propia). (a) Triangulación de Delaunay (negro) y diagrama de Voronoi dual (rojo): cada vértice de Voronoi es el circuncentro de un triángulo, y la circunferencia circunscrita destacada no contiene vértices en su interior. (b) Cavidad de Bowyer–Watson de un punto p (rojo): los triángulos cuyas circunferencias circunscritas contienen a p forman una región estrellada respecto de p, que se re-triangula uniendo p con su frontera (líneas segmentadas). 15

Figura 3 Ejemplo de PLC con un polígono no convexo con un agujero y un polígono interior que separa dos subdominios. Extraído de [2] (Figura 1a). 17

Figura 4 Construcción de una región de arista terminal en Polylla: los triángulos cuyos Lepp terminan en la misma arista terminal (rojo) se unen en un único polígono. 28

Figura 5 Análogo bidimensional de la concurrencia entre cavidades (elaboración propia). Las cavidades de Bowyer–Watson de dos puntos (naranjo y azul) comparten dos triángulos (rojo), por lo que no pueden procesarse simultáneamente. La cavidad del tercer punto (verde) no comparte triángulos con las anteriores y puede procesarse en paralelo con cualquiera de ellas. 34

Figura 6 Análogo bidimensional de una región inducida por la circunsfera de un elemento semilla (elaboración propia). El triángulo semilla (azul oscuro) y su circunferencia circunscrita (segmentada) determinan qué vecinos se agregan: aquellos cuyo circuncentro (punto relleno) cae dentro de ella. Los circuncentros que quedan fuera se muestran vacíos. A diferencia de la cavidad del panel (b) de la Figura 2, la región no se define respecto de un punto que se inserta. . . . 41

Capítulo 1

Introducción

La generación de mallas es un área que ha estado en el foco de atención constantemente desde el asentamiento de la computación como disciplina. Tanto a nivel académico como industrial, discretizar un espacio, 2D o 3D, en primitivas de manera automática es clave para poder efectuar simulaciones avanzadas. En el contexto de poliedros, comúnmente se reconocen dos formas de generar estas mallas (Sección 2.6): de manera directa, que toma información espacial directa, como una nube de puntos o un dominio geométrico, y genera la estructura de datos; y de manera indirecta, que toma una malla de tetraedros previamente construida y le aplica una serie de transformaciones para generar una malla de poliedros manteniendo sus características en rasgos generales. El presente trabajo se centrará en esta última forma.

Actualmente, existe un algoritmo en dos dimensiones capaz de generar una malla de polígonos a partir de una malla de triángulos conocido como POLYLLA [3]. Tomando este como inspiración se desarrolló una versión tridimensional [1], donde posteriormente se realizó un estudio en profundidad de la viabilidad y calidad de dichas mallas (Sección 3.3).

1.1. Problema

Debido al creciente interés y consolidación de nuevos métodos numéricos, como el método de elementos virtuales [4] (*VEM* por sus siglas en inglés), se necesita disponer de un algoritmo que sea escalable con respecto a la cantidad de células (tetraedros) mientras contenga propiedades deseables para que dicho método funcione correctamente. No obstante, el estudio realizado sobre POLYLLA 3D dio resultados poco satisfactorios en este ámbito, donde se concluyó que uno de los algoritmos, POLYLLA 3D Face, era capaz de obtener buenos resultados, pero que la ausencia de *kernel* en ciertos poliedros podría afectar su usabilidad para VEM [5].

Pendiente. Fuente del diagnóstico de kernel: confirmar en el documento identificado por la clave bibliográfica `pol3d` la afirmación de que la ausencia de kernel podría afectar la usabilidad para VEM. Localizar la página, el dato exacto y la variante evaluada; P005 ([1]) no cuantifica kernels vacíos (ver Sección 3.5).

Con esto en mente se propone una nueva forma de crear poliedros, alejada de la forma tradicional de unir triángulos por sus aristas en POLYLLA 2D: usar cavidades. En este trabajo, las regiones candidatas se definen a partir de la circunsfera asociada a cada tetraedro semilla, es decir, la esfera que pasa por sus cuatro vértices. Esta debe distinguirse tanto de la esfera inscrita, tangente a las cuatro caras, como de la cavidad de Bowyer–Watson, que es un conjunto de tetraedros en conflicto respecto de un punto de inserción [2].

1.2. Hipótesis

Es posible diseñar e implementar un algoritmo que genere mallas de poliedros a partir de una malla de tetraedros utilizando las cavidades de dichos tetraedros, seleccionando unos como semilla para formar el poliedro. Tal algoritmo resultaría más eficiente y robusto que otros implementados en la familia Polylla, produciendo mallas de calidad según los criterios de los métodos numéricos.

1.3. Preguntas de investigación

Pregunta 1 ¿Es posible crear poliedros a partir de un algoritmo de generación de poliedros basado en cavidades? ¿Cómo se deberían escoger las semillas para formar los poliedros? ¿Qué pasos de refinamiento se deberían realizar sobre la malla creada?

Pregunta 2 ¿Qué estructura de datos puede proveer un uso eficiente de memoria y tiempo de ejecución permitiendo manejar la topología de la malla? ¿Es posible crear una estructura que permita unir todos los algoritmos dentro de la familia Polylla?

Pregunta 3 ¿El algoritmo final es una solución viable para problemas con mallas típicas? ¿Qué porcentaje de poliedros de la malla final poseen un *kernel*? ¿Se obtiene una reducción sustancial de primitivas? ¿Cuáles son límites que el algoritmo tiene en términos de tamaño de la malla y cantidad de vértices?

Pregunta 4 ¿Es posible paralelizar el algoritmo? ¿Existe una forma directa de hacerlo? ¿Cómo luciría una posible versión paralela?

1.4. Objetivos

El objetivo general de esta investigación es diseñar, implementar y analizar un nuevo algoritmo de creación de mallas de poliedros de forma indirecta, inspirado en el uso de cavidades de los tetraedros.

Objetivo 1 Diseñar e implementar un nuevo algoritmo basado en las cavidades de tetraedros semillas.

Objetivo 2 Crear o seleccionar una estructura de datos que sea consistente con un modelo de programación en paralelo.

Objetivo 3 Analizar el desempeño del algoritmo basándose en métricas que permitan evaluar el sobrecosto en memoria y la eficiencia en tiempo de ejecución de un algoritmo paralelo tomando en cuenta su versión secuencial.

Objetivo 4 Estudiar y comparar la calidad de los poliedros contenidos en las mallas generadas con respecto a las versiones actuales de Polylla 3D.

Objetivo 5 Investigar y diseñar una posible versión paralela del algoritmo propuesto.

1.5. Estructura de la Tesis

El capítulo 2 presenta los fundamentos y el estado del arte. El capítulo 3 desarrolla los antecedentes de Polylla, con énfasis en las regiones terminales y el alcance de la evidencia publicada. El capítulo 4 describe el algoritmo de cavidades, su implementación secuencial y una propuesta de paralelización. El capítulo 5 establece el diseño experimental; no contiene resultados de ejecución. El capítulo 6 sintetiza las contribuciones, responde las preguntas hasta donde lo permite el trabajo actual y delimita las tareas pendientes.

Capítulo 2

Estado del arte

2.1. Introducción

La generación de mallas consiste en descomponer un dominio geométrico en una colección de celdas que no se superponen y que, en conjunto, lo cubren. Durante décadas, las celdas preferidas para la simulación numérica en tres dimensiones han sido los tetraedros y los hexaedros, porque los métodos de elementos finitos clásicos dependen de elementos de forma fija. La aparición de métodos numéricos capaces de operar sobre poliedros generales, entre los que destaca el método de elementos virtuales (VEM, por sus siglas en inglés), ha renovado el interés por las mallas poliédricas: una celda poliédrica puede ocupar la región de varios tetraedros y, con ello, reducir el número de incógnitas de una simulación, siempre que conserve las propiedades geométricas que el método numérico necesita.

Esta tesis estudia una forma *indirecta* de construir mallas poliédricas: a partir de una tetraedralización ya disponible, se agrupan tetraedros vecinos en poliedros sin insertar ni mover vértices. El criterio de agrupación propuesto se basa en las *circunsferas* de los tetraedros, es decir, en las esferas que pasan por sus cuatro vértices: desde un tetraedro semilla, se incorporan los tetraedros vecinos cuyos circuncentros quedan dentro de la circunsfera de la semilla. El trabajo se inscribe en la familia de algoritmos Polylla, que construye polígonos y poliedros uniendo triángulos o tetraedros según criterios geométricos locales, y aborda cuatro preguntas de investigación: si es posible construir poliedros mediante este criterio de cavidades y cómo escoger y refinar las regiones; qué estructura de datos permite representar eficientemente la topología de estas mallas y unificar la familia de algoritmos; si las mallas resultantes son viables, en particular qué fracción de sus poliedros posee un *kernel* no vacío y cuánto se reduce el número de celdas; y si el algoritmo puede paralelizarse.

2.1.1. Alcance y metodología de la revisión

El objetivo de este documento es establecer el marco bibliográfico de esas cuatro preguntas. La revisión se apoya en un corpus de 232 trabajos catalogados durante la investigación, de los cuales 134 se revisaron a texto completo. La selección de lo que aquí se discute obedece a un criterio de pertinencia: se incluyen los trabajos que determinan qué se le

exige a una celda poliédrica, cómo se obtiene la tetraedralización de entrada y qué propiedades geométricas tiene, cómo se generan hoy las mallas poliédricas, cómo se representan las mallas en memoria y cómo se paralelizan los algoritmos que, como el de esta tesis, construyen regiones de tetraedros a partir de pruebas sobre circunsferas. Quedan fuera, salvo mención puntual, la generación de mallas de superficie, las mallas curvas de alto orden y la adaptación anisotrópica, que no intervienen en el problema estudiado.

Cada afirmación atribuida a un trabajo se contrastó con su texto. Cuando el documento propone una interpretación propia, por ejemplo una relación entre conceptos de trabajos distintos, se señala explícitamente como tal. Del mismo modo, la afirmación de que el corpus no contiene un método equivalente al propuesto debe entenderse limitada a ese corpus y no como una prueba de novedad absoluta.

2.1.2. Organización del documento

La Sección 2.2 revisa los métodos numéricos sobre mallas poliédricas y las hipótesis geométricas que imponen a las celdas. La Sección 2.3 trata las medidas de calidad, primero para tetraedros y luego para poliedros generales, con énfasis en el *kernel* y en la relación entre calidad geométrica y precisión numérica. La Sección 2.4 presenta los fundamentos geométricos que sustentan el algoritmo: la tetraedralización de Delaunay, su dualidad con el diagrama de Voronoi, las cavidades de Bowyer–Watson y la robustez numérica de los predicados geométricos. La Sección 2.5 revisa los métodos y programas que producen la malla tetraédrica de entrada y las técnicas para mejorarla. La Sección 2.6 compara los enfoques de generación de mallas poliédricas, desde los diagramas de Voronoi hasta la aglomeración de celdas y la familia Polylla. La Sección 2.7 analiza las estructuras de datos para mallas, y la Sección 2.8, la paralelización de algoritmos basados en cavidades en CPU y GPU. Finalmente, la Sección 2.9 integra los hallazgos, compara los enfoques y delimita el espacio que ocupa la propuesta de esta tesis.

En todo el documento se distingue la *circunsfera* de un tetraedro, que pasa por sus cuatro vértices, de su esfera inscrita, tangente a sus cuatro caras. También se distingue la *cavidad* en el sentido de la literatura de Delaunay, definida respecto de un punto que se inserta, de la región que agrupa el algoritmo de esta tesis a partir de un tetraedro semilla; la diferencia se discute en la Sección 2.4.

2.2. Métodos numéricos sobre mallas poliédricas

La razón de ser de una malla poliédrica es un método numérico capaz de usarla. Esta sección revisa los métodos que admiten celdas poliédricas generales, con énfasis en el método de elementos virtuales, y examina con detalle las hipótesis geométricas que imponen a las celdas, porque son ellas las que definen qué significa que una malla generada por unión de tetraedros sea útil.

2.2.1. Motivación: más allá de los elementos de forma fija

Los métodos de elementos finitos clásicos dependen de funciones de base definidas explícitamente sobre elementos de forma fija. Beirão da Veiga et al. [6] señalan que los elementos finitos poligonales, que construyen funciones de base explícitas sobre polígonos, sufren serias dificultades en el caso tridimensional y con elementos no convexos, y que el método de elementos virtuales está libre de esos problemas. Las mallas poliédricas permiten, además, estrategias de refinamiento adaptativo y de unión de mallas que serían imposibles con tetraedros [7], y la flexibilidad de forma puede reducir el costo de ciertos problemas: un elemento finito conforme de clase C^1 sobre tetraedros requiere 220 grados de libertad por elemento, mientras que un método virtual de clase C^1 sobre poliedros usa cuatro por vértice [8]. Desde el punto de vista de la generación de mallas, la motivación es que una celda poliédrica puede cubrir la región de varios tetraedros y, con ello, reducir el número de elementos.

2.2.2. El método de elementos virtuales

El VEM fue introducido como una generalización del método de elementos finitos a mallas poligonales y poliédricas [4]. Su rasgo distintivo es que las funciones de base no se evalúan explícitamente dentro de cada elemento: se definen como soluciones de problemas diferenciales locales y se denominan *virtuales* porque nunca se calculan. El método trabaja con proyecciones polinomiales de estas funciones, computables a partir de los grados de libertad, y añade un término de estabilización que garantiza la buena definición del esquema [4, 9]. La guía de implementación de Beirão da Veiga et al. [10] admite elementos de forma muy general, convexos o no, y destaca que para problemas de difusión o reacción-difusión en dos y tres dimensiones el método no requiere fórmulas de cuadratura ni expresiones aproximadas de las funciones de base; en tres dimensiones, el método se construye usando en cada cara un espacio virtual bidimensional. Los autores resumen su actitud hacia la geometría con una frase elocuente: casi todo poliedro será aceptable, salvo los que provengan de la mente de un matemático dispuesto a construir contraejemplos [10].

El método se ha extendido en múltiples direcciones relevantes para mallas tridimensionales. Gain et al. [11] lo formulan para elasticidad lineal sobre poliedros arbitrarios y señalan que su construcción no requiere convexidad. Beirão da Veiga et al. [12] presentan la primera validación numérica sistemática del VEM tridimensional de orden alto, con grados de uno a cinco, y observan que hasta entonces los experimentos tridimensionales se limitaban al orden más bajo. Otros trabajos lo extienden a problemas no lineales e inelásticos, evaluando la ley constitutiva una vez por elemento [6]; a problemas de cuarto orden con un espacio de clase C^1 [8]; a ecuaciones acopladas entre el volumen y su superficie en dominios curvos [7]; a formulaciones de Stokes robustas respecto de la presión [13]; a problemas de autovalores [14]; y a estimaciones de error a posteriori que permiten refinamiento adaptativo con nodos colgantes y caras coplanares consecutivas sin posprocesar la malla [15].

2.2.3. Otros métodos politopales

El VEM no es el único método capaz de operar sobre poliedros generales. Los métodos esqueléticos discontinuos, entre ellos los métodos *Hybrid High-Order* (HHO), usan incógnitas en celdas y caras, admiten órdenes arbitrarios sobre mallas politopales y permiten eliminar las incógnitas interiores mediante condensación estática [16]. Boffi y Di Pietro [17] muestran que HHO, los volúmenes finitos híbridos, las diferencias finitas miméticas, el VEM mixto y los elementos de Raviart–Thomas y Crouzeix–Raviart pueden expresarse en una formulación unificada. El complejo discreto de De Rham extiende estas ideas a espacios compatibles de orden arbitrario [18], el método de Galerkin débil define un gradiente débil discreto sobre mallas politopales con conservación local de masa [19], y los métodos de Galerkin discontinuo con penalización interior operan sobre polígonos y poliedros con caras degeneradas [20]. La aglomeración de elementos es, además, la base de métodos multinivel para estos esquemas: las mallas gruesas se obtienen uniendo elementos de la malla fina [21–23].

Desde el punto de vista de la implementación, Cicuttin et al. [24] describen una biblioteca genérica para HHO sobre politopos no necesariamente convexos con caras planas, en la que las integrales sobre politopos estrellados respecto de su baricentro se calculan dividiéndolos en símplices; observan que la forma de los elementos influye en el tiempo de cálculo justamente por esa subdivisión, y que no pudieron ejecutar el *solver* sobre las mallas poliédricas por restricciones de memoria. En la misma línea, Pan y Persson [23] integran sobre elementos aglomerados sumando las contribuciones de los subelementos originales, sin costo adicional de cuadratura.

2.2.4. Hipótesis geométricas sobre las celdas

La flexibilidad de estos métodos respecto de la forma de las celdas no es irrestricta. La Tabla 1 resume las hipótesis que imponen los trabajos revisados. En el trabajo fundacional del VEM, el análisis de error supone que cada elemento es un polígono simple (A0.1), que existe $\gamma > 0$ tal que cada elemento K es estrellado respecto de una bola de radio mayor o igual a γh_K, donde h_K es su diámetro (A0.2), y que la distancia entre dos vértices cualesquiera de K es al menos γh_K (A0.3) [4]. Los propios autores advierten que la condición de estrellamiento puede debilitarse a la de ser unión de un número finito y uniformemente acotado de dominios estrellados, cada uno de los cuales satisface A0.2 [4]. En tres dimensiones, las hipótesis estándar exigen que cada poliedro sea estrellado respecto de una bola de radio comparable a su diámetro, que cada cara sea estrellada respecto de un disco de radio comparable al de la cara, y que las aristas y caras no sean pequeñas respecto del poliedro [8, 10, 12]. Estas condiciones implican que el número de aristas y caras de cada elemento está uniformemente acotado [9, 14]. Varios trabajos suponen explícitamente caras planas [14, 18, 24].

Trabajo	**Hipótesis principales sobre las celdas**	**Evidencia numérica**
VEM fundacional [4]	Polígono simple; estrellado respecto de una bola de radio γh_K; vértices separados al menos γh_K	Sin experimentos
VEM 3D, guía y orden alto [10, 12]	Celdas y caras estrelladas respecto de bolas y discos; aristas no pequeñas	Voronoi aleatorio, CVT y estructurado (3D)
Brenner y Sung [25]	Estrellado respecto de una bola; número acotado de caras; caras regulares en 2D; admite caras pequeñas	Sin experimentos nuevos
Huang y Yu [26]	Estrellado; caras con triangulaciones virtuales regulares y cuasiuniformes	Tetraedros y CVT (orden 1)
Cao y Chen [27, 28]	Número acotado de aristas o caras; condiciones locales de altura; subconjunto convexo y envolventes convexas con solapamiento acotado	Sin experimentos
Cangiani et al. [15]	Estrellado respecto de una bola; extensible a uniones de subelementos que la cumplen	2D, incluye celdas no convexas
Sorgente et al. [9]	Hipótesis G1–G3 como condiciones suficientes	Tetraedros, hexaedros, Voronoi y pares de tetraedros (3D)
HHO y DDR [17, 18]	Submalla simplicial de buena forma, con símplices de tamaño comparable a la celda; caras planas	Tetraedros, cubos y Voronoi (3D)
DG con aglomeración [20, 21]	Cubrimiento por símplices; sin cota sobre el número ni el tamaño de las caras	2D, incluye celdas aglomeradas

Tabla 1: Hipótesis geométricas sobre las celdas en los métodos politopales revisados, y tipo de mallas usadas en su validación.

Los métodos esqueléticos y el complejo de De Rham formulan la regularidad de otro modo: exigen que la malla admita una submalla simplicial de buena forma en la que cada celda se divide en símplices de tamaño uniformemente comparable al de la celda [17, 18]. Esta formulación es particularmente pertinente para esta tesis, porque una celda obtenida por unión de tetraedros trae consigo, por construcción, una subdivisión simplicial. En el mismo sentido, Beirão da Veiga et al. observan que, bajo sus hipótesis, cada poliedro es la unión de tetraedros de buena forma que comparten un vértice central [8].

2.2.5. Relajación de las hipótesis y evidencia de robustez

Una línea de trabajo teórico ha buscado debilitar estas condiciones. Brenner y Sung [25] demuestran estimaciones de error para mallas con aristas o caras pequeñas: en tres dimensiones, si cada poliedro es estrellado respecto de una bola de diámetro comparable al suyo, su número de caras está acotado y las caras satisfacen las condiciones bidimensionales, las cotas son óptimas salvo un factor logarítmico que depende de la razón entre la arista más larga y la más corta. Huang y Yu [26] obtienen desigualdades inversas, equivalencia de normas y estimaciones de interpolación para poliedros estrellados cuyas caras admiten triangulaciones virtuales regulares y cuasiuniformes. Cao y Chen [27, 28] reemplazan el estrellamiento uniforme por condiciones locales: en su análisis bidimensional, el estrellamiento se requiere para la aproximación y la ausencia de aristas cortas para la estabilidad, y la convexidad de la envolvente convexa del elemento permite prescindir de su regularidad de forma; en tres dimensiones admiten caras pequeñas o anisotrópicas siempre que el

elemento sea isotrópico en un sentido que se expresa mediante un subconjunto convexo de tamaño comparable. Cangiani et al. [15] extienden su análisis a elementos que son unión finita y simplemente conexa de subelementos que satisfacen las hipótesis, y Antonietti et al. [22] suponen que cada elemento aglomerado es unión de un número acotado de dominios estrellados. Para los métodos de Galerkin discontinuo, las hipótesis de Antonietti et al. [21] no restringen el número ni la medida de las caras, condición necesaria porque, al aglomerar, el número de caras crece y su medida puede degenerar.

La evidencia numérica es, en general, más favorable que la teoría. Beirão da Veiga et al. [12] obtienen las tasas de convergencia teóricas para todos los órdenes en mallas de Voronoi aleatorias con poliedros estirados, aunque con errores mayores que en mallas estructuradas. Dassi y Mascotto [29] no imponen hipótesis geométricas y prueban el método con elementos que colapsan; identifican dos causas de curvas de convergencia subóptimas o erróneas: el mal condicionamiento de la matriz de rigidez, que hace crecer el error en la prueba de parche y depende de la base elegida para los momentos, y elecciones poco adecuadas de la estabilización. Beirão da Veiga et al. [13] observan que el deterioro de la regularidad de una familia de mallas de Voronoi no parece afectar sus esquemas, y usan bases ortonormalizadas para mejorar el condicionamiento en presencia de caras o elementos alargados. Sorgente et al. [9] construyen familias de mallas tetraédricas, hexaédricas, de Voronoi y poliédricas, estas últimas agregando pares de tetraedros vecinos, lo que produce celdas no convexas pero estrelladas; todas las familias cumplen G1 y varias violan G2 o G3. Observan que el VEM sigue convergiendo en mallas de Voronoi aleatorias que violan G2 y G3, pero se deteriora en mallas fuertemente anisotrópicas que violan G2 de manera severa, y concluyen que las restricciones actuales son probablemente más severas de lo necesario. El estrellamiento mismo también parece admitir excepciones: en los conjuntos bidimensionales optimizados por aglomeración de Sorgente et al. [30], una cantidad significativa de elementos no es convexa o ni siquiera estrellada, y aun así se preservan las tasas de convergencia. Debe notarse, sin embargo, que buena parte de la evidencia que relaja las hipótesis es bidimensional o de orden bajo [15, 22, 27].

2.2.6. Implicancias para esta tesis

La revisión muestra que muchos de los análisis citados suponen que cada celda es estrellada respecto de una bola de tamaño comparable a su diámetro, mientras que otros admiten celdas que son unión de un número acotado de subdominios estrellados [4, 15, 22]. Si un poliedro es estrellado respecto de una bola, todos los puntos de esa bola pertenecen a su *kernel*, el conjunto de puntos desde los que todo el poliedro es visible, de modo que el *kernel* es no vacío y tiene volumen positivo. Para el primer grupo de análisis, la no vacuidad del *kernel* es, por tanto, una condición necesaria, aunque no suficiente; una medida más fiel exigiría comparar el tamaño de la mayor bola contenida en el *kernel* con el diámetro de la celda. Para el segundo grupo, una celda puede tener *kernel* vacío y aun así satisfacer las hipótesis si se descompone en pocos subdominios estrellados, de modo que el *kernel* es una medida conservadora. Esta observación motiva el uso del *kernel* como métrica central en este trabajo, complementado con medidas de forma.

La revisión sugiere además tres consideraciones específicas para celdas formadas por unión de tetraedros. Primero, estas celdas traen una subdivisión simplicial, lo que encaja con las formulaciones que exigen una submalla simplicial de buena forma o que admiten uniones de subelementos, siempre que los tetraedros sean de buena forma y de tamaño comparable a la celda [15, 17]; esa misma subdivisión permite integrar sin costo adicional [23]. Segundo, la frontera entre dos celdas es una unión de triángulos que no necesariamente forman una cara plana, y tratar cada triángulo como una cara multiplica el número de caras pequeñas; algunos análisis admiten caras coplanares consecutivas [15] o un número no acotado de caras pequeñas, representando las interfaces mediante triángulos coplanares [21], lo que sugiere tratar cada triángulo, o cada grupo de triángulos coplanares, como una cara plana; la evidencia tridimensional al respecto es escasa. Tercero, la medida basada en la envolvente convexa tiene un respaldo teórico en los análisis de Cao y Chen, que usan subconjuntos convexos y envolventes convexas para reemplazar el estrellamiento uniforme [27, 28].

2.3. Calidad de mallas

La calidad de una malla interviene dos veces en esta tesis. Primero, la calidad de la tetraedralización de entrada condiciona la posición de las circunsferas y, por lo tanto, las regiones que forma el algoritmo. Segundo, la calidad de los poliedros resultantes determina si la malla es útil para un método numérico. Esta sección revisa ambas dimensiones: las medidas de calidad para tetraedros, el problema particular de los *slivers*, la relación entre calidad geométrica y error numérico, y las medidas disponibles para poliedros generales.

2.3.1. Medidas de calidad para tetraedros

Knupp [31] define una medida de calidad de elementos como una función escalar de las posiciones de los nodos que mide alguna propiedad geométrica del elemento, y construye una teoría algebraica de estas medidas a partir de la matriz jacobiana A, cuyas columnas son las aristas que salen de un nodo. Comparando A con la matriz W de un elemento ideal de referencia, muestra que orientación, volumen, forma, razón de longitudes y sesgo forman una lista completa de las propiedades contenidas en la matriz jacobiana; otras propiedades, como la curvatura o las relaciones entre elementos adyacentes, no lo están [31]. Dentro de este marco, el *mean ratio* y la razón de radios son medidas de forma, mientras que el ángulo diedro mínimo y la razón de aristas no lo son, y el número de condición mide la distancia de un elemento al conjunto de elementos degenerados [31]. Un trabajo posterior del mismo autor da fórmulas explícitas de tamaño y forma para triángulos, tetraedros, cuadriláteros y hexaedros, y de sesgo solo para cuadriláteros y hexaedros, porque en triángulos y tetraedros no es posible cambiar el sesgo sin cambiar la forma; todas toman valores en $[0, 1]$, iguales a uno solo para la configuración ideal y a cero solo para elementos degenerados [32]. Knupp advierte, sin embargo, que estas medidas a priori no garantizan que la malla sea adecuada para una simulación concreta, y que los elementos mal formados pueden producir matrices mal condicionadas que retardan o impiden la convergencia de los *solvers* iterativos [32]. Para la optimización de mallas volumétricas, Knupp recomienda el número de condición $\kappa(A) = |A|\ |A^{-1}|$ y muestra que las funciones objetivo con barrera mantienen la validez de

una malla que parte siendo válida, mientras que las que no tienen barrera no lo hacen de forma consistente [33].

En la literatura de Delaunay predomina otra medida. La *razón radio-arista* es el cociente entre el radio R de la circunsfera y la arista más corta ℓ del tetraedro; vale al menos $\sqrt{6}/4 \approx 0,612$, valor que alcanza el tetraedro regular, y es grande para la mayoría de los tetraedros mal formados [2]. Su ventaja es que los algoritmos de refinamiento de Delaunay pueden acotarla. Su debilidad es que no detecta una clase de elementos degenerados: Alliez et al. [34] señalan que la razón radio-arista no es una medida justa porque no tiende a cero para los *slivers*, mientras que la *razón de radios*, cociente entre los radios de las esferas inscrita y circunscrita multiplicado por tres, sí detecta cualquier tipo de degeneración. Finalmente, los ángulos diedros tienen un efecto directo sobre el método numérico: cuando son demasiado grandes aumenta el error de discretización, y cuando son demasiado pequeños aumenta el número de condición de la matriz del elemento [35].

2.3.2. El problema de los *slivers*

Un *sliver* es un tetraedro cuyos cuatro vértices están cerca de un plano y cuya proyección ortogonal sobre ese plano es un cuadrilátero convexo sin aristas cortas [36]; dicho de otro modo, sus cuatro vértices están cerca de un círculo máximo de una esfera y aproximadamente equiespaciados sobre él [34]. A diferencia del caso bidimensional, incluso vértices bien espaciados pueden producir estos elementos degenerados [34], y son notoriamente frecuentes en tetraedralizaciones de Delaunay tridimensionales [36]. La literatura identifica varias razones por las que son difíciles de eliminar. El refinamiento de Delaunay acota la razón radio-arista, pero puede generar *slivers* [34]. Las energías basadas en teselaciones de Voronoi centroidales optimizan la compacidad de las celdas de Voronoi duales, pero no la de los símplices, de modo que la presencia de un *sliver* no se penaliza [34]. Los tetraedros de mala calidad tienden a agruparse, y en esos grupos el intercambio de caras falla porque la reconexión local no es legal y el suavizado falla porque mejorar un tetraedro empeora a su vecino [35]. Cheng et al. [36] demuestran que, para conjuntos de puntos periódicos cuya triangulación de Delaunay tiene acotada la razón radio-arista, existe una asignación de pesos a sus vértices tal que la triangulación ponderada resultante no contiene *slivers*, sin mover ni insertar puntos.

Para esta tesis, los *slivers* tienen una consecuencia adicional: como sus cuatro vértices son casi cosféricos y casi coplanares, el volumen que aparece en el denominador del cálculo de su circuncentro es casi nulo, y la posición del circuncentro es numéricamente inestable. La Sección 2.4 vuelve sobre este punto.

2.3.3. Calidad geométrica y error numérico

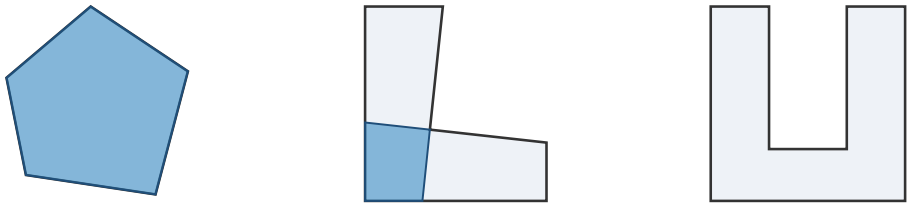
La relación entre calidad y precisión ha sido estudiada empíricamente a gran escala. Schneider et al. [37] usan una estimación a priori del error de elementos finitos que depende explícitamente de la forma del elemento, mediante el cociente σ entre su inradio y su arista más larga, en la que el error en norma L^2 escala como h^{k+1}/σ^2; concluyen que

los elementos mal formados producen errores grandes en la solución misma y no solo en sus derivadas. Advierten, además, que ningún método tridimensional conocido garantiza, para una superficie de entrada general, una cota sobre la razón entre arista máxima e inradio, y que, si debe preservarse la frontera, algunos elementos tendrán necesariamente mala forma [37]. Su propuesta es elevar localmente el grado polinomial de los elementos de mala calidad. En un estudio posterior que compara pipelines completos de elementos finitos sobre miles de modelos, los mismos autores encuentran que los tetraedros lineales rinden mal, mientras que los tetraedros cuadráticos igualan o superan a los hexaedros, y que el refinamiento en grado puede compensar los elementos mal formados que introducen los generadores [38]. Estos resultados confirman que la calidad geométrica importa, pero también que su efecto depende del método numérico.

2.3.4. Calidad de polígonos y poliedros

Para celdas generales, la noción de calidad se vuelve difusa. Knupp señala que la forma de un elemento no simplicial no puede definirse a partir de una única matriz jacobiana, y deja la extensión de sus medidas a prismas, pirámides, poliedros y elementos de alto orden como trabajo futuro, advirtiendo que probablemente requerirá ir más allá de la matriz jacobiana [31, 32]. En su revisión de indicadores, Sorgente et al. [39] precisan que la matriz jacobiana en un vértice de un poliedro solo está definida si este tiene tres aristas incidentes, y en general no lo está para politopos no convexos. Huang y Wang [40] proponen medidas de alineación y equidistribución para polígonos convexos y muestran que un polígono de buena calidad puede contener triángulos de mala calidad, por lo que recomiendan evaluar las celdas poligonales y no sus triangulaciones; reconocen, sin embargo, que su extensión a tres dimensiones no es trivial, entre otras razones porque algunos poliedros no convexos, como el de Schönhardt, no admiten una subdivisión simplicial. Para delimitar el problema y excluir configuraciones poco probables, la mayoría de los generadores de mallas y de los esquemas numéricos restringen las celdas a formas convexas o estrelladas [39].

En ese contexto, el *kernel* se ha convertido en la herramienta central. El *kernel* de un poliedro es el conjunto de puntos desde los que todo el poliedro es visible; coincide con el poliedro si este es convexo, y es vacío si el poliedro no es estrellado [39, 41] (Figura 1). Los indicadores basados en él, como la razón entre el volumen del *kernel* y el de la celda, valen uno para celdas convexas, un valor en $(0, 1)$ para celdas cóncavas estrelladas y cero para celdas no estrelladas [30, 42]. Este cociente es el núcleo del indicador de calidad para el VEM de Sorgente et al. [9], que lo combina con la relación entre los tamaños de aristas y caras y con el número de caras para medir el grado de violación de las hipótesis G1–G3 descritas en la Sección 2.2. En experimentos con el problema de Poisson sobre familias de mallas tetraédricas, hexaédricas, de Voronoi y poliédricas, el indicador predice la tendencia del error en las normas L^2 y H^1, aunque captura con menos precisión las oscilaciones de la norma L^∞ [9]. En las versiones posteriores, la componente volumétrica multiplica la razón del *kernel* de la celda por la de cada una de sus caras, de modo que basta una cara no estrellada para anular el indicador [30].



(a) (b) (c)

Figura 1: *Kernel* (azul) de tres polígonos, como análogo bidimensional del *kernel* de un poliedro (elaboración propia). (a) En un polígono convexo, el *kernel* coincide con el polígono. (b) Un polígono no convexo estrellado tiene *kernel* no vacío. (c) Un polígono que no es estrellado tiene *kernel* vacío. El *kernel* se obtiene intersectando los semiplanos interiores de todas las aristas.

El cálculo del *kernel* es, por lo tanto, una operación básica para evaluar mallas poliédricas. En dos dimensiones se resuelve intersectando los semiplanos definidos por las aristas, y existe un algoritmo óptimo de costo lineal [41]. En tres dimensiones, la extensión directa de esa idea fue descartada tempranamente por razones de complejidad, y el enfoque establecido es algebraico: transforma la intersección de semiespacios en el cálculo de una envolvente convexa mediante dualidad, con costo $O(n \log n)$, y es el que puede implementarse con CGAL [41]. Sorgente et al. [41] retoman el enfoque geométrico: parten de la caja envolvente alineada con los ejes del poliedro y la recortan sucesivamente con el plano de cada cara, usando predicados geométricos exactos, precalculando las comparaciones de vértices y terminando apenas el *kernel* se vuelve vacío. Aunque el método es cuadrático en general, resulta más rápido que el algebraico cuando el poliedro es pequeño, cuando tiene muchas caras coplanares o cuando su *kernel* es vacío; los autores sitúan el límite práctico en torno a 10^3 vértices, recomiendan el método para analizar teselaciones volumétricas con elementos no convexos y reportan que calculó correctamente el *kernel* de todos los poliedros probados [41]. Las celdas producidas por la unión de unos pocos tetraedros son, precisamente, poliedros pequeños.

2.3.5. Implicancias para esta tesis

La revisión deja tres conclusiones. Primero, las medidas basadas en la matriz jacobiana, maduras para tetraedros, no se extienden a poliedros generales, por lo que la evaluación de celdas obtenidas por unión de tetraedros debe apoyarse en medidas definidas para politopos, como las basadas en el *kernel*, en la envolvente convexa o en relaciones entre longitudes. Segundo, la existencia del *kernel* y la razón entre su volumen y el de la celda son las medidas más directamente vinculadas con las hipótesis del VEM, y existe un algoritmo de cálculo adecuado para celdas pequeñas. Tercero, la calidad de la tetraedralización de entrada no es un dato menor: los *slivers* son frecuentes, difíciles de eliminar y problemáticos precisamente para un criterio basado en circuncentros.

2.4. Fundamentos geométricos: Delaunay, Voronoi y cavidades

El algoritmo estudiado en esta tesis opera sobre una tetraedralización y decide qué tetraedros unir comparando circuncentros con circunsferas. Su comportamiento depende, por lo tanto, de las propiedades de las circunsferas en las tetraedralizaciones que recibe, que normalmente son de Delaunay o de Delaunay con restricciones. Esta sección revisa esas propiedades: la definición y unicidad de la tetraedralización de Delaunay, su dualidad con el diagrama de Voronoi, la posición de los circuncentros, la construcción incremental mediante cavidades, las tetraedralizaciones con restricciones que producen los generadores prácticos y los problemas de robustez numérica asociados.

2.4.1. La tetraedralización de Delaunay

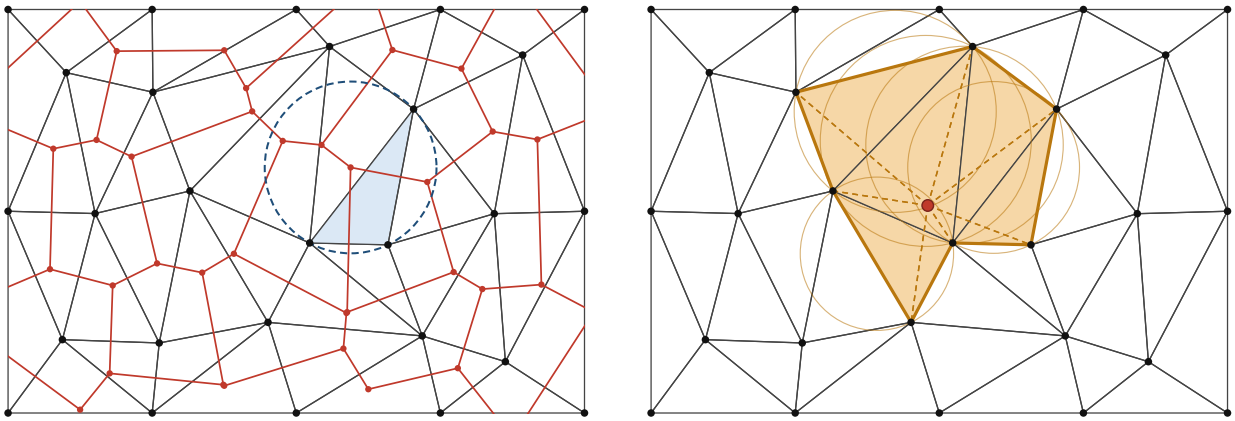
Sea V un conjunto finito de puntos en $\mathbb{R}^3$. Un símplice con vértices en V es de Delaunay si existe una esfera circunscrita a él que no encierra ningún vértice de V, y es *fuertemente* de Delaunay si, además, ningún vértice de V distinto de los suyos está sobre esa esfera [43]. Si V está en posición general, es decir, si no hay cinco puntos de V sobre una misma esfera, la tetraedralización de Delaunay de V es única [2, 44]. Su tamaño puede ser cuadrático en el número de puntos: una tetraedralización de Delaunay de n puntos puede tener $O(n^2)$ tetraedros [2].

Una caracterización clásica explica la estructura de estas tetraedralizaciones. Si cada punto se eleva a $\mathbb{R}^4$ añadiéndole como cuarta coordenada el cuadrado de su norma, un símplice es de Delaunay si y solo si su elevación está contenida en alguna cara de la parte inferior de la envolvente convexa de los puntos elevados [43]. Cuando cinco o más puntos elevados quedan sobre un mismo hiperplano no vertical que soporta la parte inferior de la envolvente, lo que ocurre cuando los puntos originales están sobre una esfera vacía común, algunas caras de esa envolvente no son símplices, y pueden rellenarse con cualquier triangulación compatible [43]. En otras palabras, la tetraedralización de Delaunay deja de ser única: todo subconjunto afínmente independiente de los vértices cosféricos define un símplice de Delaunay, algunos de esos símplices se superponen y distintas elecciones producen distintas tetraedralizaciones [43]. Spielman et al. [45] describen el mismo fenómeno al definir la triangulación de Delaunay como un complejo lineal por partes que contiene todos los símplices de Delaunay, y que es un complejo simplicial cuando los puntos están en posición general.

Esta observación es pertinente para esta tesis. Cuando cinco o más vértices comparten una esfera vacía, los tetraedros que los unen son solo una de varias subdivisiones posibles del poliedro convexo inscrito en esa esfera, de modo que unirlos elimina una elección arbitraria y recupera una celda canónica de la estructura de Delaunay. En la práctica, los generadores resuelven estas degeneraciones mediante perturbaciones simbólicas que simulan la posición general sin mover los vértices [44, 46], o desempatando según el orden en que los vértices están almacenados en memoria [47]. En ambos casos la elección entre las posibles subdivisiones es convencional, no geométrica.

2.4.2. Dualidad con el diagrama de Voronoi y posición de los circuncentros

La tetraedralización de Delaunay es dual del diagrama de Voronoi de los mismos puntos. Unir los centros de las circunsferas de los símplices de Delaunay produce el diagrama de Voronoi [48]: el circuncentro de cada tetraedro es un vértice de Voronoi, la celda de Voronoi de un vértice se obtiene conectando los circuncentros de los tetraedros que lo contienen [49], y la arista de Voronoi asociada a una cara es el segmento que une los circuncentros de los dos tetraedros que la comparten [50] (Figura 2, panel a). Esta dualidad permite leer las configuraciones casi degeneradas en términos de Voronoi: según Abdelkader et al. [51], aparecen aristas de Voronoi cortas cuando la distancia de un vértice de Voronoi a sus $d + 1$ semillas más cercanas es apenas menor que la distancia a la siguiente, es decir, cuando una semilla queda cerca de la esfera de Delaunay centrada en ese vértice. Así, dos tetraedros adyacentes con circuncentros muy próximos corresponden a una arista de Voronoi corta y a una configuración cercana a la cosfericidad.



(a) (b)

Figura 2: Análogo bidimensional de los conceptos de esta sección (elaboración propia). (a) Triangulación de Delaunay (negro) y diagrama de Voronoi dual (rojo): cada vértice de Voronoi es el circuncentro de un triángulo, y la circunferencia circunscrita destacada no contiene vértices en su interior. (b) Cavidad de Bowyer–Watson de un punto p (rojo): los triángulos cuyas circunferencias circunscritas contienen a p forman una región estrellada respecto de p, que se re-triangula uniendo p con su frontera (líneas segmentadas).

La posición de un circuncentro respecto de su tetraedro no está garantizada. Un símplice es *bien centrado* si contiene su circuncentro en su interior, y una triangulación de Delaunay no es necesariamente bien centrada; a la inversa, una malla completamente bien centrada de un dominio convexo es de Delaunay [52]. VanderZee et al. [52] caracterizan esta propiedad mediante bolas ecuatoriales, muestran que en tres dimensiones cada vértice interior debe tener al menos siete aristas incidentes para admitir una malla bien centrada, y reconocen que sus resultados tridimensionales son rudimentarios y que los obstáculos topológicos no se comprenden del todo. Engwirda [50] observa que exigir que todos los vértices de Voronoi duales estén dentro de sus triángulos es particularmente oneroso, porque requiere que todos los elementos sean agudos, y lo persigue mediante optimización. En la frontera del

dominio el problema se agrava: los tetraedros de Delaunay pueden ser muy planos y tener su circuncentro fuera del dominio [49], y varios métodos optan por descartar o desactivar esos elementos, como los que solo conservan los tetraedros cuyo circuncentro está dentro del dominio [53, 54]. Cheng y Poon [55] recuerdan condiciones suficientes, basadas en esferas diametrales y ecuatoriales vacías, para que el circuncentro de un tetraedro quede dentro del dominio.

Para esta tesis, estas observaciones tienen consecuencias directas. Un criterio que compara circuncentros con circunsferas opera sobre puntos que pueden estar fuera del tetraedro al que pertenecen e incluso fuera del dominio, especialmente cerca de la frontera. Por otra parte, para dos tetraedros de Delaunay adyacentes, que el circuncentro del vecino quede dentro de la circunsfera de una semilla equivale a que la arista de Voronoi dual a la cara compartida sea más corta que el radio de esa circunsfera; en el límite cosférico, los circuncentros coinciden y la arista de Voronoi tiene longitud nula. Esta lectura, que se desarrolla en la Sección 2.9, es una interpretación propia derivada de la dualidad.

2.4.3. Construcción incremental y cavidades de Bowyer–Watson

Los algoritmos incrementales de Bowyer [56] y Watson [57] construyen la triangulación de Delaunay insertando los puntos uno a uno. Al insertar un punto p, la *cavidad de Delaunay* es el conjunto de todos los tetraedros cuya circunsfera contiene a p; esos tetraedros se eliminan y el hueco poliédrico resultante se rellena con nuevos tetraedros que unen p con las caras de su frontera [58]. Si [2] precisa que esta cavidad es un poliedro simplicial estrellado respecto de p, lo que permite re-tetraedralizarla de esa manera, y que la tetraedralización resultante vuelve a ser de Delaunay (Figura 2, panel b). En el núcleo de inserción de HXT, la construcción de la cavidad representa cerca de un tercio del tiempo total de ejecución en las implementaciones disponibles [58].

El análisis de estos algoritmos se apoya en la aleatoriedad. Guibas et al. [59] definen el *alcance* de un triángulo como el número de sitios contenidos en el interior de su circunferencia circunscrita, de modo que los triángulos de alcance cero son exactamente los de Delaunay, y demuestran que la inserción en orden aleatorio, con un historial de todas las versiones de la triangulación para localizar los puntos, construye la triangulación de Delaunay plana en tiempo esperado $O(n \log n)$. En tres dimensiones acotan por $O(n^2)$ el número esperado de tetraedros que aparecen durante la construcción, pero dejan abierto el diseño de un algoritmo eficiente; registran, además, el resultado de Joe según el cual el procedimiento incremental de *flips* de tetraedros siempre produce la tetraedralización de Delaunay [59]. En cambio, partiendo de una triangulación arbitraria, el algoritmo de *flips* puede quedar atrapado antes de alcanzar la de Delaunay en tres o más dimensiones [48]. En la práctica, la eficiencia depende del orden de inserción: TetGen agrupa los puntos según un orden aleatorizado sesgado y, dentro de cada grupo, los ordena a lo largo de una curva de Hilbert, para preservar la localidad sin perder el carácter aleatorio [2]. Mavriplis [60], que combina el frente de avance con la conectividad de Delaunay en dos dimensiones, ofrece una idea útil para esta tesis: para encontrar rápidamente qué circunferencias contienen un

punto, representa cada una como un punto (x, y, r) de un espacio tridimensional y la busca con un octree, aunque reconoce que esa búsqueda sigue consumiendo una parte significativa del tiempo.

El término *cavidad* requiere precisión en esta tesis. En la literatura de Delaunay designa una región definida respecto de un punto que se desea insertar. El algoritmo estudiado en este trabajo, en cambio, agrupa tetraedros existentes a partir de la circunsfera de un tetraedro semilla, sin insertar puntos. Ambas nociones comparten la herramienta geométrica, la prueba de pertenencia a una circunsfera, pero no su significado; en particular, la región de la tesis no hereda automáticamente el estrellamiento respecto de un punto que caracteriza a la cavidad de Bowyer–Watson.

2.4.4. Tetraedralizaciones con restricciones

Los generadores prácticos no triangulan nubes de puntos, sino dominios descritos por un *complejo lineal por partes* (PLC): un conjunto de vértices, aristas, polígonos y poliedros que describe el dominio y sus fronteras internas [2] (Figura 3). Como la tetraedralización de Delaunay de los vértices no respeta, en general, las aristas y caras del PLC, se recurre a la *tetraedralización de Delaunay con restricciones* (CDT), en la que se exige que exista una circunsfera de cada símplice tal que ningún vértice dentro de ella sea visible desde el interior relativo del símplice, considerando que las caras del PLC bloquean la visibilidad [43, 61]. Una CDT contiene tetraedros que no son globalmente de Delaunay, aunque sí localmente, y conserva propiedades cercanas a las de una tetraedralización de Delaunay [44].

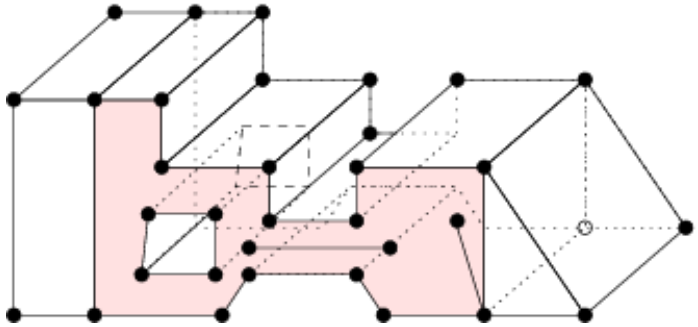


Figura 3: Ejemplo de PLC con un polígono no convexo con un agujero y un polígono interior que separa dos subdominios. Extraído de [2] (Figura 1a).

A diferencia del caso plano, donde una colección de segmentos que no se cruzan siempre admite una CDT, en tres dimensiones la CDT puede no existir [47], y decidir si un PLC no genérico la tiene es un problema NP-completo [43]. Shewchuk demuestra que todo PLC cuyas aristas son fuertemente de Delaunay tiene una CDT, condición suficiente pero no necesaria [43]. Por ello, los generadores agregan puntos de Steiner. TetGen construye primero la tetraedralización de Delaunay de los vértices, luego recupera las aristas del PLC insertando puntos de Steiner y finalmente recupera las caras mediante cavidades [44]; su autor reconoce que aún no se conoce una cota polinomial para el número de puntos de Steiner que agrega la recuperación de aristas [44]. Diazzi et al. [47] muestran que la

expansión de cavidades de ese procedimiento puede fallar con independencia de los errores numéricos, aunque en la práctica ocurrió en solo dos de 4.408 modelos, y reportan que TetGen falla en 378 de esos 4.408 modelos de Thingi10k, mientras que su método robusto los procesa todos.

Para esta tesis, la consecuencia es que la malla de entrada producida por un generador con restricciones puede contener puntos de Steiner y tetraedros que solo son localmente de Delaunay. Las propiedades de las circunsferas descritas en esta sección valen, por lo tanto, de forma local, y las caras del PLC son restricciones que una región formada por unión de tetraedros no debería atravesar si se desea preservar las fronteras internas del dominio.

2.4.5. Refinamiento de Delaunay y calidad

El refinamiento de Delaunay mejora una tetraedralización insertando los circuncentros de los tetraedros de mala calidad [44, 61]. La medida que controla es la razón radio-arista, cociente entre el radio de la circunsfera y la arista más corta del tetraedro [55]. Cheng y Poon [55] resumen sus límites: el algoritmo de Shewchuk puede no terminar cuando algún ángulo de entrada es menor que $\pi/2$, y, aunque acota la razón radio-arista, pueden quedar *slivers*, que son la única clase de tetraedros malos que persiste cuando esa razón está acotada. Su propio algoritmo garantiza una malla de Delaunay conforme con razón radio--arista acotada fuera de una zona de protección alrededor de los ángulos pequeños, pero reconocen como problema abierto construir mallas conformes con razón de aspecto acotada en presencia de ángulos pequeños [55]. Si [44] precisa que un *sliver* puede tener una razón radio-arista tan baja como $\sqrt{2}/2 \approx 0,707$ y que no está demostrado que su refinamiento los elimine; además, como el circuncentro de un tetraedro puede caer fuera del dominio, solo se considera para inserción si está dentro de él. Labelle y Shewchuk [62] señalan que las cotas demostrables sobre ángulos diedros de los métodos de Delaunay y de octree son menores que $0,1°$, y que la exudación de *slivers* suele eliminar la mayoría de los tetraedros malos de una malla de Delaunay, pero rara vez todos.

2.4.6. Robustez numérica

Las decisiones de un generador dependen de predicados geométricos como `orient3d`, que determina de qué lado de un plano está un punto, e `insphere`, que determina si un punto está dentro de la esfera definida por otros cuatro; ambos se calculan como el signo de un determinante, y su evaluación en punto flotante puede producir resultados incorrectos o hacer fallar el programa [2]. Shewchuk [63] propuso predicados adaptativos que evalúan primero una aproximación con una cota de error y solo aumentan la precisión cuando el signo es incierto, de modo que los casos regulares terminan rápidamente y los degenerados llegan al cálculo exacto; TetGen y Gmsh usan predicados de este tipo [2, 64].

Estas garantías tienen un límite importante para esta tesis. Diazzi et al. [47] advierten que los predicados exactos no bastan cuando algunos de los puntos involucrados son construidos por el algoritmo, porque sus coordenadas se redondean y los predicados dejan de garantizar la respuesta correcta. Los circuncentros son, precisamente, puntos construidos:

se obtienen dividiendo por un determinante proporcional al volumen del tetraedro, de modo que en los *slivers*, cuyo volumen es casi nulo, su posición es numéricamente inestable. Boissonnat et al. [65] muestran, en otro contexto, que las inconsistencias de sus mallas anisotrópicas provienen solo de configuraciones casi cosféricas y que insertar puntos en los centros de las esferas circunscritas puede llevar a una sucesión interminable de ellas. Un criterio de pertenencia basado en circuncentros es, por lo tanto, sensible justamente en las configuraciones casi cosféricas y casi degeneradas, y una tolerancia fija no resuelve el problema de fondo.

2.4.7. Implicancias para esta tesis

La revisión de esta sección fundamenta tres ideas que orientan el trabajo. Primero, la estructura de Delaunay contiene, en el caso degenerado, celdas poliédricas convexas cuya subdivisión en tetraedros es arbitraria, lo que da un sentido geométrico a unir tetraedros con circunsferas coincidentes o cercanas. Segundo, los circuncentros pueden quedar fuera de sus tetraedros y del dominio, y la malla de entrada de un generador con restricciones solo es localmente de Delaunay, de modo que cualquier interpretación global de las circunsferas debe tomarse con cautela. Tercero, las configuraciones que el algoritmo tendería a unir, las casi cosféricas, son también las más sensibles numéricamente.

2.5. Generación de mallas tetraédricas

Un método indirecto hereda las propiedades de su malla de entrada: su número de elementos, la distribución de sus vértices, la calidad de sus tetraedros y la forma en que respeta la frontera del dominio. Esta sección revisa cómo se generan las mallas tetraédricas, con énfasis en los programas que producen las entradas habituales de los métodos indirectos, y cómo se mejoran una vez construidas.

2.5.1. Familias de métodos

Los métodos de generación de tetraedros pueden agruparse en cuatro familias, que a menudo se combinan.

Retículas y octrees. Estos métodos parten de una subdivisión regular del espacio y la adaptan a la geometría. El *isosurface stuffing* de Labelle y Shewchuk [62] rellena un dominio definido implícitamente usando la triangulación de Delaunay de una retícula cúbica centrada en el cuerpo, cuyos tetraedros son idénticos, y deforma los vértices cercanos a la superficie; garantiza que todos los ángulos diedros quedan entre $10,78°$ y $164,74°$ y es numéricamente robusto porque no depende de predicados sensibles. Su precio es que no preserva aristas ni esquinas agudas y que, al graduar el tamaño en la superficie y en el interior, las garantías se debilitan [62].

Frente de avance. Estos métodos construyen la malla desde la frontera hacia el interior. NETGEN combina modelado por geometría sólida constructiva, seguimiento de aristas y generación de mallas de superficie y de volumen por frente de avance, y separa las reglas locales de generación del código que las aplica [66]. Su autor reconoce que, en tres

dimensiones, no siempre es posible cortar un elemento sin insertar puntos nuevos y que la elección de las reglas es heurística [66]. Los métodos híbridos combinan la colocación de puntos del frente de avance con la conectividad de Delaunay [46, 60].

Delaunay. Estos métodos construyen una tetraedralización de Delaunay, recuperan las restricciones del dominio y la refinan, como se describió en la Sección 2.4. Son la base de TetGen [2], de los generadores de CGAL [67], de HXT [58] y del núcleo propio de Gmsh [64].

Muestreo y optimización. Una cuarta familia distribuye primero los puntos y luego los conecta. El muestreo de Poisson maximal coloca puntos separados por una distancia mínima que cubren el dominio, y conecta las muestras mediante una triangulación de Delaunay o regular [53]; sus autores advierten que incluso vértices bien espaciados pueden crear *slivers* y descartan los tetraedros cuyo circuncentro queda fuera del dominio. Los métodos variacionales mueven los vértices para minimizar una energía, como se discute más adelante en esta sección [34, 68].

2.5.2. Generadores de referencia

La Tabla 2 resume los generadores más relevantes para esta tesis.

Generador	**Entrada**	**Enfoque**	**Observaciones**
TetGen [2, 44]	PLC o nube de puntos	Delaunay incremental, CDT y refinamiento de Delaunay	Solo fronteras lineales por partes; puede fallar ante entradas defectuosas
CGAL [67]	Frontera del dominio	Refinamiento de Delaunay restringido a la superficie	La salida no necesariamente respeta el complejo de entrada [61]
NETGEN [66]	Geometría sólida constructiva	Frente de avance con reglas	Reglas heurísticas en 3D
Gmsh [64]	Modelo CAD	Delaunay propio o núcleos de NETGEN y TetGen, y optimización	Integra preproceso y postproceso
TetWild y fTetWild [69, 70]	Sopa de triángulos posiblemente defectuosa	Partición del espacio o inserción incremental, y optimización dentro de una tolerancia	Robustez casi total; aproxima la superficie
HXT [58, 71]	Superficie triangulada	Delaunay paralelo y mejora con cavidades crecientes	Recuperación de frontera secuencial
Isosurface stuffing [62]	Superficie implícita	Retícula y deformación de vértices	Ángulos diedros garantizados; no preserva rasgos agudos

Tabla 2: Generadores de mallas tetraédricas revisados.

TetGen. TetGen es la referencia principal de este grupo y el generador de las mallas de entrada de esta tesis [2]. Implementa dos algoritmos incrementales de Delaunay, la inserción de Bowyer–Watson y el método de *flips* aleatorizado; construye la CDT recuperando primero las aristas y luego las caras del PLC; y aplica refinamiento de Delaunay con

restricciones y mejora local para alcanzar los criterios de calidad pedidos por el usuario [2, 44]. Su autor reconoce como limitaciones que solo maneja fronteras lineales por partes y que la calidad de sus mallas puede ser inferior a la de otros programas [2]. Su estructura de datos y sus predicados se discuten en la Sección 2.7 y en la Sección 2.4.

CGAL, NETGEN y Gmsh. Los generadores de CGAL aproximan la frontera del dominio mediante triangulaciones de Delaunay restringidas a la superficie, por lo que su salida no necesariamente respeta el complejo de entrada [61, 67]. Gmsh se organiza en cuatro módulos (geometría, malla, *solver* y postproceso), usa predicados geométricos robustos en las partes críticas de sus algoritmos e integra, además de su propio algoritmo de Delaunay basado en Bowyer–Watson, los núcleos tetraédricos de NETGEN y TetGen [64]. Sus autores usan una medida de calidad que, para tetraedros, combina el volumen, el área de las caras y la longitud de la arista más larga, toma valores en $[0, 1]$ y se anula en un *sliver*, y reconocen que los procedimientos de optimización de mallas tienen mucho de «magia negra» [64].

TetWild y fTetWild. Estas herramientas priorizan la robustez frente a entradas defectuosas. TetWild convierte una sopa de triángulos, posiblemente con autointersecciones y huecos, en una malla tetraédrica: construye una partición binaria del espacio con coordenadas racionales exactas y luego mejora la malla manteniéndola válida y dentro de una tolerancia ε respecto de la superficie de entrada; los autores reconocen que el método es más lento que la mayoría de sus competidores [69]. fTetWild sustituye esta etapa por una inserción incremental que garantiza una malla válida con coordenadas de punto flotante, a cambio de no garantizar que todas las caras de entrada queden insertadas, aunque en sus pruebas siempre lo estuvieron [70]. Sobre los 10.000 modelos de Thingi10k reportan tasas de éxito de 99,97 % para fTetWild, 99,89 % para TetWild, 79,00 % para CGAL y 49,50 % para TetGen; la mayoría de los fallos de CGAL y TetGen se clasifican como limitaciones del algoritmo en la tabla de los autores, y los tiempos promedio, de 49,8 s para fTetWild y 360,0 s para TetWild, se calculan sobre los modelos que cada método completa y no son directamente comparables [70]. En un contexto cercano, la CDT robusta de Diazzi et al. procesa los 4.408 modelos válidos de Thingi10k en los que TetGen falla en 378 [47].

HXT. HXT muestra que la construcción de la tetraedralización puede paralelizarse con gran eficiencia: su núcleo de Delaunay genera tres mil millones de tetraedros en 53 s sobre un procesador AMD EPYC [58]. Su etapa de mejora introduce un operador local, la *Growing SPR Cavity*, que hace crecer una cavidad y elige una mejor tetraedralización de ella, y que los autores describen como la madre de todos los *flips* [71]. La recuperación de la frontera se delega todavía en la implementación de TetGen [71]: es la única etapa del generador que no se paralelizó, y en uno de sus casos de prueba ocupa más de un tercio del tiempo total de mallado con el máximo número de hilos [58].

2.5.3. Mejora de mallas tetraédricas

Los generadores suelen complementar la construcción de la malla con una etapa de mejora que modifica la posición de los vértices, la conectividad o ambas. Esta etapa es relevante para esta tesis porque altera las circunsferas sobre las que opera el algoritmo y, en algunos casos, la propiedad de Delaunay de la malla.

Suavizado e intercambios locales. Freitag y Ollivier-Gooch [35] comparan intercambios de caras y de aristas con varias formas de suavizado sobre mallas aleatorias y de aplicaciones. Observan que el suavizado laplaciano puede producir mallas inválidas, con elementos invertidos, y que el óptimo alcanzado por los intercambios es casi con seguridad local. Su primera recomendación es no usar el criterio de la esfera vacía en la pasada final de intercambios, porque rinde mal respecto de los ángulos extremos; proponen comenzar con intercambios según ese criterio y continuar con intercambios que maximicen el seno mínimo de los ángulos diedros [35]. Este resultado tiene una consecuencia directa para un algoritmo basado en circunsferas: mejorar los ángulos diedros mediante intercambios puede alejar la malla de la propiedad de Delaunay. Dassi et al. [72] combinan un suavizado basado en ecuaciones diferenciales de movimiento de malla, que mantiene la malla no singular si lo es inicialmente, con una búsqueda perezosa y reversible de *flips*; reconocen que esta última es compleja y difícil de paralelizar.

Optimización basada en circuncentros. La triangulación de Delaunay óptima (ODT) minimiza una energía que alterna una triangulación de Delaunay global, que es la conectividad óptima para esa energía, con la reubicación de cada vértice interior en el promedio de los circuncentros de sus tetraedros incidentes ponderado por volumen, tratando por separado los vértices de frontera [34]. Tournois et al. [73] intercalan este paso con refinamiento de Delaunay por lotes, en el que el punto de Steiner de un tetraedro malo es su circuncentro, y observan que, cuando los vecinos de un vértice están sobre una esfera, el vértice se mueve al centro de esa esfera. En sus experimentos, tras cien iteraciones su variante reduce los tetraedros con ángulos diedros menores que 5° de 35, con refinamiento solo, y de 23, con Lloyd, a cero, aunque reconocen que no pueden garantizar la ausencia total de *slivers* y que las cadenas de *slivers* incidentes entre sí empeoran significativamente el resultado [73]. Du y Wang [68] optimizan globalmente una malla de Delaunay mediante iteraciones de Lloyd hasta obtener una teselación de Voronoi centroidal restringida, seguida de intercambios locales, y elevan la calidad mínima de la malla de un cubo de 0,012 a 0,237; Alliez et al. sostienen, en cambio, que la energía de Voronoi centroidal no penaliza los *slivers* [34]. Ninguno de estos métodos ofrece cotas teóricas sobre la calidad de los tetraedros resultantes [34].

Mejora en paralelo. Las implementaciones paralelas de la mejora comparten un patrón: procesar simultáneamente solo elementos o vértices independientes. D'Amato y Vénere [74] reconectan grupos de tetraedros cuyos vecindarios no se intersectan, de modo que pueden procesarse sin bloqueos, ordenándolos para tratar primero los peores elementos

y aceptando solo cambios que no empeoren la peor calidad; obtienen aceleraciones de 2,81 a 6,61 veces respecto de su versión secuencial, según la malla y la configuración, pero la asignación de grupos y la actualización de estructuras, que permanecen secuenciales, se convierten en el nuevo cuello de botella. Ströter et al. [75] colorean los vértices en conjuntos independientes para paralelizar en GPU, sin conflictos ni operaciones atómicas, una optimización armónica de la malla, y resuelven los *flips* en conflicto eligiendo el localmente más beneficioso; reportan aceleraciones de hasta dos órdenes de magnitud respecto de una CPU secuencial y observan que sus *flips* preservan bien el porcentaje de tetraedros localmente de Delaunay. Xi et al. [76] implementan en GPU un suavizado laplaciano que solo acepta un movimiento si mejora la calidad local, con una aceleración máxima de 22,6 veces sobre su versión secuencial, aunque reconocen que su variante de múltiples bloques no garantiza la ausencia de conflictos de datos.

2.5.4. Implicancias para esta tesis

La revisión muestra que la malla de entrada no es neutra. Su distribución de vértices determina qué configuraciones casi cosféricas aparecen; la estrategia de recuperación de la frontera determina si contiene puntos de Steiner y tetraedros solo localmente de Delaunay; y su etapa de mejora puede alejarla de la propiedad de Delaunay. Por ello, evaluar un método indirecto exige controlar el generador y los parámetros de la malla de entrada, como hicieron los trabajos de la familia Polylla al comparar distribuciones aleatorias, de Poisson, en retícula y mallas de calidad [1], o el estudio de Carrier Baudouin et al., que mostró que la tasa de recombinación de tetraedros en hexaedros depende fuertemente de la ubicación de los vértices [77].

2.6. Generación de mallas poliédricas

La generación de mallas poliédricas es un área menos madura que la generación de tetraedros. Los autores de VoroCrust lo expresan así: mientras la generación de mallas tetraédricas basada en refinamiento de Delaunay u optimización variacional está bien establecida, la investigación en mallas poliédricas lo está menos [51]. Esta sección organiza los enfoques existentes según una distinción habitual en el área: los métodos *directos* construyen la malla a partir de la geometría del dominio, mientras que los métodos *indirectos* parten de una malla inicial, típicamente de triángulos o tetraedros, y la transforman [3]. La ventaja de los indirectos es que pueden apoyarse en generadores simpliciales robustos y de libre acceso [3]. Dentro de los indirectos se distinguen, a su vez, tres estrategias: dualizar la malla de entrada, subdividir sus elementos y unir sus elementos. La Tabla 3 resume esta clasificación, que estructura el resto de la sección.

Estrategia	Mecanismo	Celdas resultantes	Trabajos
Directa: Voronoi recortado	Diagrama de Voronoi de semillas, recortado contra el dominio	Convexas en el interior; recortadas en la frontera	[78]
Directa: Voronoi conforme	Unión de bolas y pares de semillas a ambos lados de la frontera	Convexas, sin recorte	[51, 79]
Directa: fondo recortado	Octree recortado contra la superficie	Generales, estrelladas desde un centro	[80, 81]
Indirecta: dualización	Dual de una tetraedralización, con corrección de concavidades	Generales; caras posiblemente curvas	[49, 82]
Indirecta: subdivisión	Partición de tetraedros con los planos de la entrada	Convexas	[83]
Indirecta: recombinación	Patrones de tetraedros que forman hexaedros u otros elementos	Hexaedros, prismas, pirámides y tetraedros	[77, 84]
Indirecta: aglomeración	Partición del grafo de elementos u optimización de un criterio	Generales, posiblemente no estrelladas	[30, 42, 85, 86]
Indirecta: regiones locales	Unión de elementos según un criterio geométrico local	Generales, no necesariamente convexas; estrellamiento no evaluado	[1, 3]

Tabla 3: Taxonomía de los métodos de generación de mallas poliédricas revisados.

2.6.1. Métodos directos basados en diagramas de Voronoi

Las mallas de Voronoi–Delaunay son la realización geométrica por defecto de muchas formulaciones numéricas, y sus celdas tienen propiedades deseables que se siguen de su definición: convexidad, caras planas y una malla dual ortogonal dada por la tetraedralización de Delaunay de las semillas [51]. Su dificultad central es ajustarse a la frontera del dominio. La estrategia más común consiste en truncar las celdas que cruzan la frontera. Yan et al. [78] calculan de forma exacta el diagrama de Voronoi recortado en dominios no convexos, representados mediante una malla tetraédrica: deducen las celdas interiores directamente de la triangulación de Delaunay de las semillas, detectan las de frontera por propagación y recortan cada una contra los tetraedros que la intersectan. El método alimenta la construcción de teselaciones de Voronoi centroidales para generar mallas tetraédricas, y su costo es proporcional al número de pares celda–tetraedro incidentes [78]. Para superficies, el mismo grupo calcula el diagrama de Voronoi restringido intersectando cada celda con los triángulos de la superficie; como ambos son convexos, su intersección es un polígono convexo [87].

El recorte, sin embargo, sacrifica propiedades importantes. Según el análisis de VoroCrust, dualizar una malla tetraédrica y truncar sus celdas contra la superficie sacrifica la convexidad y la conexidad de las celdas [79]. En los experimentos de Abdelkader et al. [51], el diagrama restringido de un trabajo posterior de Yan et al. produjo numerosas facetas de

mala calidad en la superficie y una proporción de celdas no convexas que varió entre 3 % y 96 % según la curvatura de la entrada y las semillas elegidas. VoroCrust evita el recorte: cubre la superficie con una unión de bolas, coloca pares de semillas en las intersecciones de estas bolas para capturar y aislar la frontera, y luego siembra el interior [51]. Es el primer algoritmo demostrablemente correcto para mallas de Voronoi conformes en dominios no convexos y no *manifold*, con garantías sobre la calidad de los elementos de superficie y de volumen [51]. Su versión abstracta, para superficies cerradas y suaves, demuestra que la superficie reconstruida es isotópica a la original bajo condiciones de muestreo explícitas y acota la *gordura* de las celdas interiores y de frontera, aunque deja fuera los rasgos agudos [79]. Ambos trabajos reconocen una limitación pertinente para esta tesis: las aristas de Voronoi cortas en el interior, que pueden restringir algunos *solvers* y que surgen cuando un sitio queda cerca de la esfera de Delaunay centrada en un vértice de Voronoi [51, 79].

2.6.2. Métodos directos sobre un fondo recortado

Un segundo grupo de métodos directos parte de una subdivisión de fondo independiente de la geometría. Liu et al. [80] construyen un octree balanceado que encierra un modelo STL, lo refinan según la curvatura de la superficie y lo recortan contra ella de abajo hacia arriba, primero aristas, luego caras y luego celdas. La entrada puede contener triángulos degenerados, autointersecciones y facetas superpuestas. Las celdas resultantes se analizan con el método de elementos finitos de frontera escalada (SBFEM), que exige un *centro de escala* desde el cual sea visible toda la frontera del subdominio y que, por lo tanto, admite poliedros cóncavos estrellados [80]. El método recupera siempre los rasgos agudos paralelos a los ejes, pero no los oblicuos, que requieren fusionar celdas; los autores reconocen que esta fusión no garantiza que la frontera forme siempre una curva cerrada y que, en los casos difíciles, es razonable descartar la celda problemática [80]. Implementaciones posteriores del SBFEM en ABAQUS y en un *solver* explícito masivamente paralelo reiteran el requisito: la frontera de cada celda debe ser visible desde el centro de escala, es decir, la celda debe ser estrellada [81, 88]. Esta condición es, en esencia, la misma que la no vacuidad del *kernel* discutida en la Sección 2.2.

2.6.3. Mallas duales de tetraedralizaciones

La dualización produce celdas poliédricas a partir de una tetraedralización: cada vértice de la malla primal origina una celda formada por los circuncentros, o por otros puntos centrales, de los tetraedros que lo contienen. Garimella et al. [49] aplican esta idea a dominios no *manifold* y muestran que, si la malla de entrada es de Delaunay y bien centrada, el resultado es una malla de Voronoi con caras planas, mientras que en general las caras pueden ser curvas. El obstáculo es que, en la frontera, los tetraedros de Delaunay pueden ser muy planos y tener su circuncentro fuera del dominio; construir una malla de Voronoi válida exige que cada tetraedro de frontera contenga su circuncentro, condición muy difícil de lograr salvo en geometrías simples [49]. Los autores exigen, además, que cada celda supere una prueba conservadora de estrellamiento y optimizan la malla, pero en sus ejemplos persisten elementos inválidos en concavidades agudas [49].

Lee [82] estudia el origen de esas concavidades: las celdas cóncavas aparecen a lo largo de las aristas geométricas cóncavas del modelo. Compara tres remedios: insertar superficies no *manifold* a lo largo de esas aristas, que es prometedor pero pocos generadores lo soportan; descomponer las celdas en conos, que produce celdas típicamente un orden de magnitud más pequeñas que sus vecinas; y cortarlas a lo largo de las aristas cóncavas, que considera la alternativa más viable [82]. La validación es cualitativa, con un *solver* de fluidos incompresibles. Lee corrige las concavidades cortando las celdas, mientras que Garimella et al. optimizan la malla para que sus celdas superen una prueba de estrellamiento; en ambos casos, la no convexidad cerca de la frontera se trata como un defecto que corregir, mientras que los métodos numéricos poliédricos modernos admiten celdas no convexas estrelladas.

2.6.4. Subdivisión de tetraedros en celdas convexas

Diazzi y Attene [83] recorren el camino inverso a la unión de tetraedros. Calculan la tetraedralización de Delaunay de los vértices de una superficie triangulada, posiblemente defectuosa; asignan a cada tetraedro los triángulos de entrada que lo cruzan; lo subdividen con los planos de esos triángulos mediante una partición binaria del espacio; y clasifican las celdas en interiores y exteriores con un corte mínimo de grafo. Gracias a predicados indirectos, garantizan que la superficie de entrada quede representada exactamente como unión de facetas de la malla de salida, sin recurrir a aritmética racional, y que cada celda sea convexa, aunque no necesariamente estrictamente convexa [83]. Sobre Thingi10k procesan 9.999 de 10.000 modelos con un tiempo medio de 4,34 s y 225 MB por modelo. Reconocen, sin embargo, que la salida puede ser innecesariamente compleja y contener facetas de forma arbitrariamente mala [83]. El contraste con esta tesis es ilustrativo: aquí la convexidad se garantiza, pero la forma de las celdas la dictan los planos de la entrada y no un criterio de calidad.

2.6.5. Recombinación de tetraedros en elementos de forma fija

La unión de tetraedros tiene una larga tradición en la generación de mallas hexaédricas. H-Morph parte de una malla tetraédrica y avanza un frente de cuadriláteros desde la superficie: para cada hexaedro recupera sus seis caras en la malla tetraédrica, elimina los tetraedros que quedan en su interior y mantiene en todo momento una malla mixta válida; los tetraedros restantes se conectan con los hexaedros mediante pirámides [84]. Carrier Baudouin et al. [77] identifican, mediante tres patrones de ensamblaje, todas las combinaciones de tetraedros que forman hexaedros, las ordenan por calidad y crean primero las de mayor calidad, y luego aplican el mismo procedimiento a los prismas. Muestran que la tasa de recombinación depende fuertemente de la ubicación de los vértices: con vértices aleatorios, menos del 50 % de los tetraedros se combina en hexaedros, por lo que proponen colocarlos previamente en una estructura de tipo retícula mediante un frente de avance; aun así, no hay garantías sobre el porcentaje alcanzado [77]. El trabajo de Bukenberger et al. [89] señala que, en trabajos recientes, las mallas hex-dominantes se obtienen fusionando tetraedros en poliedros más complejos, que a menudo, pero no siempre, son hexaedros, y propone una clase restringida de mallas cuyas celdas no tienen más de seis caras ni caras de más de cuatro lados, aunque sus caras no son necesariamente planas.

H-Morph y la recombinación de Carrier Baudouin et al. comparten con la propuesta de esta tesis dos rasgos: la unión de tetraedros existentes y la selección codiciosa de las uniones según una prioridad. Difieren en que persiguen una topología de salida fija, mientras que en una malla poliédrica general la forma de la celda es libre.

2.6.6. Aglomeración de celdas para métodos poliédricos

En el contexto de los métodos numéricos poliédricos, la unión de elementos se denomina *aglomeración*. Como señalan Antonietti et al. [90], la aglomeración ha sido poco explorada porque no es posible desarrollar estrategias de este tipo dentro de los métodos de elementos finitos clásicos. Los trabajos recientes pueden agruparse según el alcance de la decisión de unión.

Aglomeración global por partición. Sorgente et al. [30] optimizan mallas poligonales y poliédricas para el VEM representando la malla mediante su grafo dual, ponderando nodos y arcos con un indicador de calidad específico para el VEM y particionando ese grafo con METIS; cada parte se reemplaza por la unión de sus elementos. Logran eliminar hasta un 80 % de los elementos preservando las tasas de convergencia, e incluso recuperan la convergencia en mallas patológicas; sus conjuntos de prueba tridimensionales incluyen mallas generadas con TetGen [30]. Un trabajo previo de los mismos autores, aplicado a las mallas poligonales de cada fractura de una red de fracturas discretas, minimiza mediante cortes de grafo una energía cuyo término de datos es uno menos la calidad de la unión de dos elementos adyacentes, y reduce hasta un 65 % los elementos y un 50 % los grados de libertad [42]. Ambos trabajos reconocen la misma limitación: el algoritmo evalúa la calidad de las uniones de a pares, de modo que no anticipa la calidad de la unión de tres o más elementos, y algunos elementos pequeños persisten [30, 42]. Antonietti et al. reformulan la aglomeración como un problema de partición de grafos resuelto con redes neuronales sobre grafos, primero para mallas poligonales [90] y luego para mallas tetraédricas de dominios heterogéneos [85]. En este último trabajo, una malla de 12.175 tetraedros con elementos severamente distorsionados, que no produce una solución numérica estable, se aglomera en 170 poliedros sobre los que el método de Galerkin discontinuo sí converge [85]. Ninguno de estos métodos garantiza convexidad ni estrellamiento: la calidad se mide *a posteriori* o se incorpora como peso.

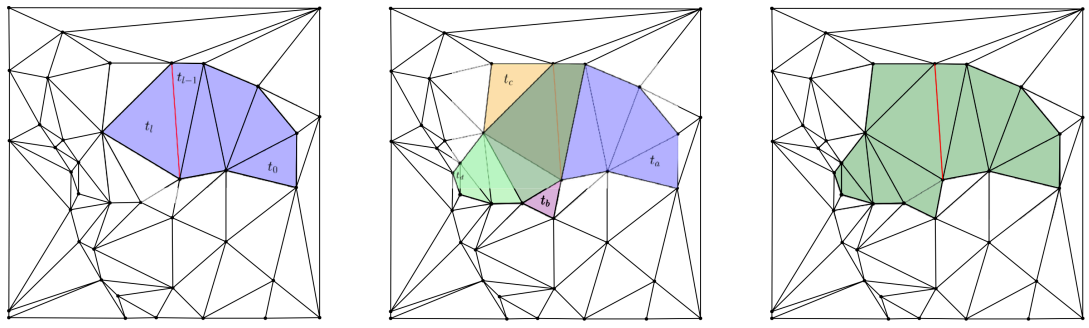
Aglomeración local alrededor de elementos defectuosos. Sukumar y Tupek [91] unen tetraedros y prismas mal formados con un vecino para formar elementos virtuales poliédricos, posiblemente no convexos, con el fin de aumentar el paso de tiempo crítico en elastodinámica explícita; en una viga cónica el paso de tiempo resulta hasta $4,5 \times 10^4$ veces mayor que el de los elementos finitos. Observan, sin embargo, que para tetraedros de tipo *spire* el resultado es deficiente cuando el volumen del poliedro aglomerado tiende a cero, recomiendan elegir el vecino de modo que ese volumen no se anule, y dejan como trabajo futuro un procedimiento automático y robusto de aglomeración [91]. Fu et al. [86] proponen precisamente un procedimiento automático de este tipo dentro del método de partículas de elementos finitos: los tetraedros cuyo paso de tiempo local es menor que una fracción

del promedio se unen de forma codiciosa con el vecino que maximiza el paso de tiempo del agregado, hasta superar el umbral, sin mover nodos. Como el remallado de Delaunay tridimensional genera elementos de volumen nulo o casi nulo, esta aglomeración permite manejar los *slivers* [86]. Su criterio controla el paso de tiempo, pero no la convexidad ni la forma de las celdas.

Un dato adicional muestra que las uniones de tetraedros ya forman parte del banco de pruebas del VEM: para estudiar su indicador de calidad, Sorgente et al. [9] construyen mallas poliédricas agregando pares de tetraedros vecinos, eligiendo los de mayor volumen y uniéndolos con el vecino que comparte la cara más amplia; las celdas resultantes son no convexas pero estrelladas.

2.6.7. La familia Polylla

Polylla es el antecedente directo de esta tesis: une elementos según regiones definidas por caminos de propagación sobre la propia triangulación. En dos dimensiones, el algoritmo construye polígonos a partir de *regiones de arista terminal* [3]. Estas se basan en el *camino de propagación de la arista más larga* (Lepp), introducido por Rivara [92] como la lista ordenada de triángulos vecinos sucesivos cuya arista más larga es mayor que la del triángulo anterior en el camino. Una arista es *terminal* si es la más larga de los dos triángulos que la comparten, y una región de arista terminal está formada por todos los triángulos cuyos Lepp terminan en la misma arista terminal [3] (Figura 4). El algoritmo clasifica las aristas según su longitud, recorre cada región para construir su polígono y repara los polígonos no simples, que contienen *aristas barrera* en su interior, dividiéndolos en polígonos simples [3]. No inserta puntos, tiene costo lineal en tiempo y memoria, y produce mallas con tres veces menos polígonos y la mitad de los puntos que las mallas de Voronoi con restricciones generadas a partir de la misma entrada, con precisión y costo del VEM similares en pruebas preliminares [3]. Como el algoritmo no divide ningún triángulo, el menor ángulo de la triangulación de entrada es una cota inferior del menor ángulo interior de la malla poligonal [3].



(a) (b) (c)

Figura 4: Construcción de una región de arista terminal en Polylla: los triángulos cuyos Lepp terminan en la misma arista terminal (rojo) se unen en un único polígono.

La familia se ha extendido en tres direcciones. En paralelismo, la versión para GPU implementa las tres fases como *kernels* sobre arreglos indexados [93]; sus resultados se discuten en la Sección 2.8. En memoria, la versión compacta reemplaza los arreglos por una estructura *half-edge* sucinta [94], discutida en la Sección 2.7. En dimensión, la versión preliminar de Polylla 3D presentó dos variantes, una basada en regiones de arista terminal y otra en una noción nueva de *regiones de cara terminal* [95].

La variante basada en caras, publicada posteriormente, necesita responder qué es la «cara más grande» de un tetraedro. Para ello introduce un *criterio de unión* que ordena las caras de cada tetraedro, como el área o el radio de la circunferencia inscrita en la cara, y define a partir de él la *cara terminal* (la mayor de los dos tetraedros que la comparten), el *camino de propagación de la cara mayor* y la *región de cara terminal*, formada por los tetraedros cuyos caminos terminan en la misma cara [1]. Las regiones de cara terminal no se superponen, de modo que forman una partición de la malla. El algoritmo conserva las tres fases: el etiquetado marca las caras frontera y escoge un tetraedro semilla por región; el recorrido construye cada poliedro con una búsqueda en profundidad desde su semilla; y la reparación divide los poliedros no simples, que contienen *caras barrera*, a partir de las *puntas de barrera* [1]. En pruebas con cuatro distribuciones de puntos en un cubo, con mallas de 500 a 50.000 vértices, el algoritmo reduce el número de celdas en cerca de un 70 %, une en promedio tres tetraedros por poliedro, alrededor de un quinto de los poliedros contiene caras barrera y en torno al 18,6 % de los elementos permanece como tetraedro [1].

Los autores destacan tres limitaciones que motivan directamente esta tesis. Los resultados son casi independientes del criterio de unión escogido, lo que indica que el concepto de región de cara terminal debe mejorarse con estrategias que consideren la calidad de los poliedros; los poliedros generados están muy lejos de ser convexos; y aún falta verificar su utilidad en simulaciones con el VEM [1]. El análisis detallado de la familia, incluidas sus variantes tridimensionales y los criterios de unión, corresponde al capítulo de trabajo previo de esta tesis.

2.6.8. Balance

La revisión de esta sección permite ubicar con precisión la propuesta de la tesis. Las celdas de Voronoi sin recorte son convexas, pero los métodos basados en ellas exigen construir y ubicar semillas, presentan aristas cortas, y el recorte contra dominios no convexos pierde la convexidad. Los métodos duales parten de tetraedros, pero deben corregir las celdas cerca de la frontera, y la subdivisión garantiza convexidad a costa de celdas cuya forma no se controla. Los métodos de recombinación y de aglomeración unen tetraedros, pero persiguen formas fijas o deciden mediante particiones globales, optimización o criterios ligados a la estabilidad temporal, como el paso de tiempo del agregado. Polylla une elementos con un criterio local, pero su versión tridimensional no controla la calidad de las celdas. Ninguno de los trabajos revisados usa como regla de unión la posición de los circuncentros respecto de la circunsfera de un tetraedro semilla; las circunsferas aparecen solo de forma auxiliar,

como en la construcción de duales [49] o en el filtrado de elementos por su radio circunscrito en el método de partículas [86].

2.7. Estructuras de datos para mallas

Un algoritmo que agrupa tetraedros recorre la malla por adyacencia, consulta incidencias entre caras y celdas, y produce una nueva malla cuyas celdas son listas de caras. La estructura de datos que soporta estas operaciones determina tanto el costo en memoria como el costo de cada consulta, y condiciona la posibilidad de paralelizar el algoritmo. Esta sección revisa los criterios generales de diseño y las soluciones adoptadas por los generadores y sistemas relevantes para esta tesis.

2.7.1. Representaciones completas y reducidas

La referencia clásica para elegir una estructura de datos de mallas volumétricas es el estudio de Garimella [96], que parte de la constatación de que el diseño de una base de datos de mallas debe equilibrar dos requisitos contradictorios: compacidad y eficiencia computacional. El trabajo analiza diez representaciones para mallas de tetraedros y hexaedros. Las representaciones *completas* almacenan explícitamente todas las entidades topológicas (vértices, aristas, caras y regiones) y un subconjunto de sus adyacencias; las *reducidas* omiten algunas entidades, como las aristas o las caras, y las crean bajo demanda cuando una consulta las necesita. Las reducidas ahorran memoria, pero encarecen ciertas consultas y obligan a resolver un problema que las completas no tienen: si una entidad temporal se crea dos veces, ambas instancias tienen direcciones distintas y su igualdad debe establecerse comparando sus vértices, por ejemplo ordenándolos según sus identificadores [96].

La representación reducida más simple, R1, almacena solo regiones y vértices con enlaces en ambos sentidos, y es un refinamiento de la clásica estructura elemento-nodo. R2 añade a cada región información sobre sus regiones adyacentes. R3 representa explícitamente vértices, caras y regiones, pero no aristas, y R4 agrega a R3 información de vecindad entre vértices [96]. Para comparar estas alternativas, el autor cuenta el número de operaciones de cada consulta de adyacencia y de cada operación de creación, y pondera esos costos con frecuencias de uso medidas en un generador de mallas real. Concluye que R2, R4 y F4 (una representación completa con dos de las tres adyacencias ascendentes) ofrecen un buen compromiso entre almacenamiento y costo: recomienda F4 si se necesitan caras y aristas explícitas, R4 si solo se necesitan caras explícitas y R2 en otro caso, y advierte que la clasificación depende de la carga de trabajo usada para ponderarla [96]. Esta última advertencia es metodológicamente importante: la elección de una estructura no se resuelve en abstracto, sino a partir de las consultas que efectivamente realiza el algoritmo.

2.7.2. Estructuras de los generadores tetraédricos

Los generadores de tetraedros han convergido hacia estructuras mínimas, centradas en el tetraedro y en la adyacencia por cara. TetGen representa una tetraedralización con una estructura basada en tetraedros, que según su autor es fácil de mantener y requiere menos memoria que las estructuras basadas en aristas o en pares cara-arista [2]. Cada tetraedro

guarda cuatro referencias a sus vecinos y cuatro a sus vértices; para distinguir la cara o arista común con cada vecino añade un entero de 4 bits por vecino, y un entero de 16 bits para marcar estados de caras, aristas y del propio tetraedro, de modo que un tetraedro ocupa básicamente ocho punteros y 32 bits. Cada vértice guarda sus coordenadas y una referencia a un tetraedro que lo contiene [2]. La navegación se hace mediante *handles* formados por un tetraedro y una de sus doce aristas dirigidas, cuya codificación permite moverse dentro de una misma cara con aritmética modular. Además, TetGen mantiene tetraedros ficticios que unen las caras de frontera con un punto en el infinito, de modo que toda cara pertenezca a exactamente dos tetraedros, lo que simplifica las operaciones de modificación [2].

HXT lleva esta idea a su forma más austera. Su implementación, escrita en C, almacena la malla en arreglos planos de números de punto flotante y enteros: para cada tetraedro t y cada índice local j, la posición $4t + j$ guarda el vértice j y la posición en la que está almacenada la adyacencia inversa, de modo que dividir esta última por cuatro entrega el tetraedro vecino opuesto al vértice j [58]. La única conectividad que se almacena explícitamente es la adyacencia entre tetraedros, porque basta para recorrer la malla con consultas locales, y los autores sostienen que este estilo de programación, aparentemente anticuado, facilita la optimización y la paralelización porque obliga a usar algoritmos simples [58]. El costo de esta austeridad se concentra en la actualización de adyacencias después de cada inserción, que según sus mediciones es la etapa más cara de la inserción incremental y que optimizan con operaciones especializadas para la cavidad [58].

2.7.3. Estructuras en la familia Polylla

En dos dimensiones, Polylla emplea una estructura indexada con adyacencias: arreglos de vértices, de triángulos y de vecinos, en la que las posiciones $3i$, $3i + 1$ y $3i + 2$ del arreglo de vecinos guardan los vecinos del triángulo i, con costo de memoria lineal [3]. La versión para GPU adopta la misma representación porque las GPU y sus modelos de programación están diseñados para trabajar eficientemente con arreglos, y almacena la malla de salida en un arreglo plano de polígonos acompañado de un arreglo de posiciones que indica dónde comienza cada polígono [93]. La versión compacta, en cambio, implementa la estructura *half-edge* sobre pemb, una representación sucinta de grafos planares basada en vectores de bits, y reduce en un 99 % la memoria de la información topológica respecto de una *half--edge* no compacta; como contrapartida, las consultas son más lentas, y en la triangulación compacta las coordenadas de los vértices, que no se pueden comprimir fácilmente, pasan a ocupar cerca del 89 % de la memoria [94]. Los autores argumentan que la versión compacta puede ser más rápida cuando la no compacta no cabe en memoria principal, por efecto de la jerarquía de memoria, y proponen extender la estructura *half-edge* a tres dimensiones como trabajo futuro [94].

2.7.4. Estructuras para procesamiento masivamente paralelo

En GPU, el problema dominante no es solo el volumen de memoria, sino la localidad de los accesos. RXMesh divide mallas triangulares de superficie en parches que se representan de

forma compacta mediante matrices, ampliados con franjas de elementos vecinos (*ribbons*) que eliminan la necesidad de comunicación entre bloques de hilos, de modo que las consultas se resuelvan en la memoria compartida de cada bloque [97]. Su modelo de programación oculta esta complejidad al usuario e induce balance de carga incluso en mallas irregulares, pero sus autores limitan el trabajo a aplicaciones estáticas que no modifican la topología de la malla [97]. En el refinamiento paralelo de mallas no estructuradas se ha usado la representación *array-based half-facet* (AHF), que unifica las estructuras *half-vertex*, *half--edge* y *half-face* para mallas de una, dos y tres dimensiones, incluso no *manifold*. AHF almacena solo dos mapas: el de *half-facets* hermanas, que enlaza en una lista circular las caras orientadas de los elementos que comparten una misma faceta, y el que asocia a cada vértice una *half-facet* incidente; las entidades de dimensión intermedia no se almacenan, sino que se referencian implícitamente [98]. Ray et al. extienden esta representación para soportar jerarquías de refinamiento y recorridos eficientes [98].

2.7.5. Implicancias para esta tesis

Los trabajos revisados coinciden en tres principios. Primero, la relación topológica primaria para recorrer una malla de tetraedros es la adyacencia por cara, que TetGen y HXT almacenan explícitamente [2, 58]. Segundo, las representaciones basadas en índices dentro de arreglos contiguos son las que permiten implementaciones paralelas, tanto en CPU como en GPU [58, 93]. Tercero, las entidades que no participan en las consultas frecuentes pueden omitirse y reconstruirse bajo demanda, a cambio de resolver la identidad de las entidades temporales [96]. Ninguna de estas estructuras fue diseñada para mallas poliédricas obtenidas por unión de tetraedros, cuyo resultado natural es una partición de los tetraedros en regiones y una lista de caras de frontera por región. En esa clasificación, una estructura que almacena vértices, caras y tetraedros, pero no aristas, corresponde a la familia de representaciones reducidas del tipo R3 descrita por Garimella [96], y la decisión de derivar las aristas solo cuando una consulta lo requiera debe evaluarse con la carga de trabajo real del algoritmo, como recomienda ese mismo estudio.

2.8. Paralelización de algoritmos de mallas basados en cavidades

La cuarta pregunta de investigación de esta tesis es si el algoritmo de cavidades puede paralelizarse y cómo luciría una versión paralela. El algoritmo tiene dos etapas de naturaleza distinta: el cálculo de la circunsfera de cada tetraedro, que es independiente para cada elemento, y el crecimiento de regiones desde semillas que compiten por los mismos tetraedros. Esta segunda etapa es análoga a la construcción concurrente de cavidades de Bowyer–Watson, que la literatura de generación de mallas ha estudiado durante más de dos décadas. Esta sección introduce los conceptos de computación paralela necesarios, describe el problema de la concurrencia entre cavidades y organiza las estrategias propuestas, primero en CPU y memoria distribuida y luego en GPU.

2.8.1. Conceptos de computación paralela

Navarro et al. [99] ofrecen un marco conceptual para diseñar y evaluar algoritmos paralelos. El *trabajo* $T(n, 1)$ es el tiempo total necesario para ejecutar un algoritmo paralelo con un

procesador, y el *span* $T(n,\infty)$ es el tiempo de la ruta de cómputo más larga ejecutada por un hilo; de ellos se derivan las cotas $T(n,p) \geq T(n,1)/p$ y $T(n,p) \geq T(n,\infty)$. La *aceleración* se define como $S_p = T_s(n,1)/T(n,p)$, donde T_s es el tiempo del mejor algoritmo secuencial, y la *eficiencia* como $E_p = S_p/p$; la ley de Amdahl corresponde al escalamiento fuerte, con tamaño de problema fijo, y la de Gustafson al escalamiento débil, en el que el problema crece con el número de procesadores [99]. El modelo PRAM supone procesadores síncronos sobre una memoria compartida de acceso unitario y distingue variantes según se permitan lecturas y escrituras concurrentes; en el caso de escrituras concurrentes, la variante de *prioridad* establece que escribe el procesador de mayor prioridad [99]. El modelo BSP, en cambio, organiza el cómputo en superpasos de cómputo local, comunicación y sincronización por barrera.

Para el diseño, Navarro et al. recomiendan la metodología de Foster en cuatro etapas: *partición*, que busca la descomposición con el mayor número de subproblemas; *comunicación*, en la que las condiciones de carrera se resuelven con secciones críticas u operaciones atómicas; *aglomeración*, que elige la granularidad adecuada, de modo que un algoritmo para CPU multinúcleo debería producir aglomeraciones mayores que el mismo algoritmo diseñado para GPU; y *mapeo*, que asigna las aglomeraciones a procesadores buscando patrones balanceados y simples [99]. Los autores advierten que en GPU los hilos de un mismo *warp* solo se ejecutan completamente en paralelo si siguen el mismo camino de ejecución, que el acceso coalescente a memoria es determinante, y que estructuras como arreglos dinámicos, tablas de dispersión y grafos complejos aún no son amigables con el rendimiento [99]. Blelloch et al. [100] añaden una medida útil para comparar algoritmos paralelos con sus contrapartes secuenciales: un algoritmo es α-eficiente en trabajo respecto de otro si realiza a lo sumo $1/\alpha$ veces sus operaciones.

2.8.2. Concurrencia entre cavidades

Dos puntos pueden insertarse concurrentemente en una triangulación de Delaunay si sus cavidades no se intersectan; en caso contrario, hay un conflicto [58] (Figura 5). Re-triangular cavidades que se intersectan puede producir una triangulación inconsistente o que no sea de Delaunay [101]. Nave et al. [102] precisan la condición: no basta con que las cavidades sean disjuntas, sino que deben serlo sus *clausuras*, formadas por la cavidad y los tetraedros que comparten una cara con ella. Bajo esa condición, la triangulación resultante es la misma independientemente del orden de las inserciones, y el mecanismo de bloqueo que la asegura es, en esencia, un algoritmo dinámico de conjuntos independientes, no necesariamente máximos [102]. Los mismos autores reconocen que este esquema simple puede caer en *livelock* y que diseñar un algoritmo determinista que lo evite no es directo; proponen como trabajo futuro calcular y re-triangular iterativamente conjuntos independientes maximales de cavidades [102].

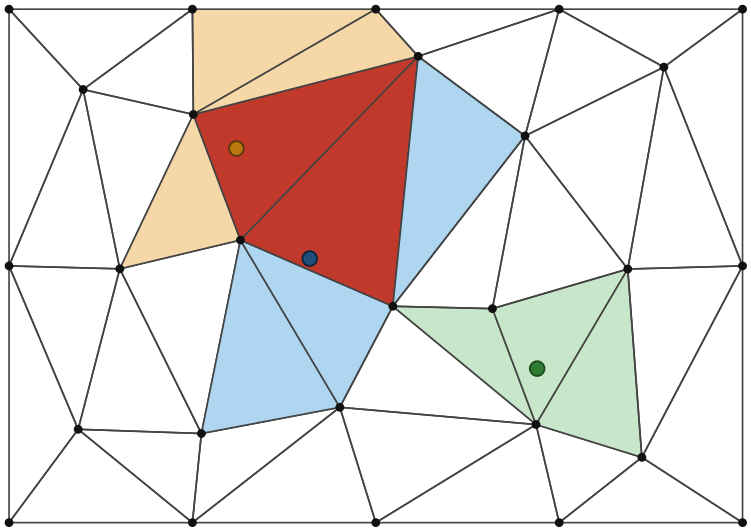


Figura 5: Análogo bidimensional de la concurrencia entre cavidades (elaboración propia). Las cavidades de Bowyer–Watson de dos puntos (naranjo y azul) comparten dos triángulos (rojo), por lo que no pueden procesarse simultáneamente. La cavidad del tercer punto (verde) no comparte triángulos con las anteriores y puede procesarse en paralelo con cualquiera de ellas.

El paralelo con esta tesis es directo: en el algoritmo de cavidades, cada tetraedro debe pertenecer a un único poliedro, de modo que dos regiones que crecen simultáneamente compiten por los tetraedros de su frontera común, igual que dos cavidades de Delaunay compiten por sus tetraedros compartidos. La diferencia es que aquí no hay re-triangulación: el conflicto se resuelve asignando cada tetraedro a una sola región.

2.8.3. Estrategias en CPU y memoria distribuida

La literatura ofrece cinco grandes estrategias para resolver o evitar los conflictos, resumidas en la Tabla 4.

Desacoplamiento geométrico a priori. Estas estrategias demuestran una condición local que vuelve independientes los subproblemas antes de resolverlos. Blelloch et al. [100] construyen en dos dimensiones un camino de aristas de Delaunay que separa dos subproblemas, proyectando los puntos sobre un paraboloide y calculando una envolvente convexa, lo que elimina la fase de unión; usan su algoritmo como particionador grueso y terminan cada subproblema con un algoritmo secuencial, con cerca de 50 % de eficiencia paralela. Linardakis y Chrisochoides [103] definen una *zona de desacoplamiento*, formada por los círculos diametrales abiertos de los segmentos separadores, y la prerrefinan para que los subdominios puedan refinarse con un programa secuencial sin modificarlo y sin comunicación; el precio es cierto sobre-refinamiento, y los autores reconocen que la extensión a tres dimensiones es difícil. Lo [104] lleva una idea similar a tres dimensiones con un criterio basado en circunsferas: agrupa los puntos en celdas y zonas que se triangulan simultáneamente, y añade capas de celdas en las fronteras entre zonas hasta que todas las

circunsferas de los tetraedros de frontera quedan acotadas. Los tetraedros duplicados se asignan a la zona de menor índice entre las de sus vértices, de modo que pueden eliminarse sin coordinación, y el resultado es exactamente la triangulación que produciría la inserción secuencial; con 12 procesadores y 20 millones de puntos alcanza una aceleración de 10,82, siempre que el número de zonas sea múltiplo del número de procesadores [104].

Descomposición del dominio y reutilización de un generador secuencial. Otra familia divide el dominio en muchos más subdominios que procesadores y malla cada uno con un generador secuencial. Ito et al. [105] fijan de antemano las interfaces entre subdominios para evitar comunicación adicional, pero el refinamiento de superficies y el pegado final, realizados por un único procesador, limitan la aceleración. Chen et al. [106] observan que la descomposición secuencial del dominio impide que todo el esquema sea escalable. Yu et al. [107] paralelizan el problema y no el algoritmo, con capas de partición generadas alrededor de los planos de corte que no quedan como restricciones en la malla final, y generan mallas de hasta 11 mil millones de tetraedros, aunque reconocen que el balance de carga es menos prometedor en dominios complejos. Chen et al. [108] reparan a posteriori los tetraedros que no son de Delaunay en las interfaces entre subdominios mediante una región de reconstrucción mínima, y muestran que la proporción del tiempo dedicada a esa reparación crece con el número de procesadores en los casos evaluados. En la misma línea, Loseille et al. [109] mallan en paralelo las partes del dominio manteniendo fijas sus interfaces y luego iteran el procedimiento sobre las interfaces, con un único operador basado en cavidades para todas las modificaciones.

Especulación con bloqueos. Kohout et al. [110] comparan estrategias de memoria compartida basadas en la inserción incremental aleatorizada para pocos procesadores; su método optimista bloquea los nodos necesarios y, si detecta una espera mutua, abandona la inserción, deshace los cambios y la reintenta. Observan que el número de bloqueos mutuos crece mucho más lento que el número de puntos y recomiendan minimizar el número y la duración de las secciones críticas [110]. Chrisochoides y Nave [101] aplican la especulación en memoria distribuida: suspenden las cavidades que requieren datos remotos y ejecutan otras inserciones locales mientras tanto, ocultando más del 90 % de la latencia a costa de un sobrecosto de comunicación de entre 2 % y 20 %.

Particiones dinámicas. HXT particiona los puntos según su coordenada en una curva de Moore y, cuando una inserción cae en conflicto con otra partición, la pospone y modifica las particiones en la siguiente ronda mediante un reescalamiento de la curva, evitando mecanismos de sincronización pesados [58].

Conjuntos independientes. Spielman et al. [45] analizan un refinamiento de Delaunay que en cada iteración inserta un conjunto de puntos independientes, y demuestran, bajo las hipótesis de su análisis, que el número de iteraciones es polilogarítmico en la razón entre el diámetro del dominio y la arista más pequeña. Ma et al. [111] convierten esta

idea en una garantía de reproducibilidad para el frente de avance: asignan a cada cara del frente un índice global invariante respecto de la descomposición del dominio y, en cada iteración, aceptan un avance solo si su índice es menor o igual que el de todos los avances vecinos con los que entra en conflicto. Con ello obtienen mallas independientes del número de procesadores, a diferencia de una búsqueda codiciosa cuyos resultados dependen del orden [111].

Trabajo	**Plataforma**	**Resolución de conflictos**	**Determinismo**
Kohout et al. [110]	Memoria compartida, 2–4 procesadores	Bloqueos optimistas con retroceso	No discutido
Chrisochoides y Nave [101]; Nave et al. [102]	Memoria distribuida	Bloqueo de clausuras de cavidades	Independiente del orden si las clausuras son disjuntas
Spielman et al. [45]	Análisis teórico	Conjuntos independientes por ronda	—
Lo [104]	Memoria compartida	Zonas con capas acotadas por circunsferas; dueño de menor índice	Igual a la inserción secuencial
Linardakis y Chrisochoides [103]	Memoria distribuida, 2D	Zona de desacoplamiento prerrefinada	No discutido; garantiza conformidad
Marot et al. [58]	Memoria compartida	Particiones por curva de Moore modificadas en cada ronda	Para un número fijo de hilos, como opción
Ma et al. [111]	Memoria distribuida	Conjunto independiente maximal con índices globales	Independiente del número de procesadores
Chen y Tan [61]	GPU	Marcado atómico por prioridad	No discutido
Coll y Guerrieri [112]	GPU, 2D	Una operación por triángulo por ronda; desempate por índice	No discutido
Zhou et al. [113]	GPU y CPU, 2D	Descarte de propuestas en conflicto con otras de menor índice	No: selección aleatoria

Tabla 4: Estrategias de resolución de conflictos en la generación paralela de mallas. La columna «Determinismo» indica lo que cada trabajo reporta sobre la reproducibilidad de su resultado.

Dos lecciones transversales emergen de estos trabajos. La primera es el balance de carga: cuando el trabajo se concentra espacialmente, como en la adaptación localizada de mallas, el desbalance es casi el peor posible, y conviene predecir la carga antes de ejecutar y repartir en consecuencia [114]. La segunda es la ley de Amdahl en la práctica: en casi todos los esquemas queda una fase secuencial o irregular, como la descomposición inicial [106], el pegado de subdominios [105] o la recuperación de la frontera en HXT, que en uno de sus casos ocupa más de un tercio del tiempo total [58], y esa fase termina limitando la aceleración.

2.8.4. Algoritmos en GPU

La GPU impone restricciones adicionales: miles de hilos deben recibir cargas de trabajo balanceadas y regulares, la sincronización entre hilos es costosa y la divergencia de ejecución degrada el rendimiento [61, 115]. El trabajo de Chen y Tan [61] sobre refinamiento de Delaunay con restricciones es la principal referencia de esta tesis en este aspecto. Para insertar puntos concurrentemente exigen que sean *independientes en el sentido de Delaunay*, es decir, que no formen una arista, para evitar que el algoritmo no termine; definen la *región de Delaunay* de un punto como la unión de los tetraedros cuyas circunsferas lo contienen y cuyo interior es visible desde él, y observan que dos puntos con regiones disjuntas, salvo vértices o aristas, son independientes. En dos dimensiones, los conflictos pueden resolverse deshaciendo *flips*; en tres dimensiones no se sabe si los *flips* pueden alcanzar todas las triangulaciones, de modo que esa estrategia podría quedar atrapada, y por ello calculan las regiones haciendo crecer cavidades antes de insertar los puntos [61].

El crecimiento se organiza para que el trabajo sea regular. Cada cavidad se representa con tuplas formadas por un tetraedro y una de sus caras, almacenadas en listas en memoria global; cada hilo procesa una tupla y prueba si el vecino a través de esa cara contiene al punto en su circunsfera, y las nuevas tuplas se agregan en posiciones calculadas con sumas prefijas y se compactan [61]. La exclusión mutua se resuelve con prioridades: cada tetraedro puede pertenecer a lo sumo a una cavidad; un elemento intenta marcar un tetraedro escribiendo atómicamente su prioridad, y si no lo logra, su cavidad deja de crecer y el punto se descarta en esa ronda. Entre elementos de igual dimensión, dar mayor prioridad a los de mayor medida mejoró la aceleración, porque dividir primero los elementos grandes genera más trabajo para la GPU [61]. Una fase posterior encoge las cavidades para respetar la visibilidad respecto de las caras restringidas, y requiere menos cómputo que el crecimiento. El resultado es hasta un orden de magnitud más rápido que TetGen, con cantidades similares de puntos de Steiner y calidades comparables; en modelos de Thingi10k, la aceleración llegó a 19 veces para la versión puramente en GPU y a 34 para la variante híbrida [61]. Esta variante traslada a la CPU la división de subsegmentos y subcaras, porque requiere muchas rondas con poco trabajo cada una y no aprovecha la capacidad de la GPU [61].

Otros trabajos completan el repertorio de patrones. Coll y Guerrieri [112] construyen en GPU la triangulación de Delaunay con restricciones en dos dimensiones mediante rondas de *kernels* en las que solo puede insertarse un punto por triángulo y voltearse una arista por triángulo, resuelven los empates eligiendo el índice menor y asignan los índices de los triángulos nuevos sin contadores atómicos, a partir del índice del triángulo y del punto insertado. SAFT paraleliza el frente de avance asignando una cara del frente a cada hilo, que propone un elemento nuevo; una propuesta se acepta si no intersecta a ninguna de índice menor, y los conflictos se resuelven descartando los elementos ilegales [113]. Su experiencia es ilustrativa: la selección y la actualización del frente quedaron en la CPU, y las transferencias y la parte secuencial pasaron a dominar el tiempo; además, la selección

aleatoria de caras hace que el resultado no sea reproducible, crítica que recogen Ma et al. [111]. Gao y Chen [116] presentan la primera triangulación de Delaunay tridimensional calculada completamente en GPU: cada punto se procesa de forma independiente gracias a una generalización del lema de Delaunay local, y triangulan diez millones de puntos uniformes en 0,91 s con un resultado idéntico al de CGAL, aunque su estructura de cubetas favorece las distribuciones uniformes. GPredicates reduce la divergencia separando en dos *kernels* la evaluación rápida de los predicados, que resuelve la mayoría de los casos, de la evaluación exacta, a la que llegó cerca de un 5 % de las consultas en datos sintéticos y un 8 % en datos reales [115]. En la familia Polylla, la versión para GPU reserva posiciones en el arreglo de salida mediante sumas atómicas; su fase más costosa fue el etiquetado y la reparación no mostró ventaja significativa sobre la versión secuencial, pese a que muy pocos polígonos la requieren [93].

Finalmente, dos patrones de bajo nivel son relevantes para construir las estructuras de salida. Mlakar et al. [117] expresan la subdivisión de superficies como operaciones de álgebra dispersa y construyen la conectividad en GPU con el patrón contar, sumar prefijos y llenar, usando operaciones atómicas cuya contención está acotada por la valencia de cada vértice; argumentan que las estructuras *half-edge* son difíciles de paralelizar porque requieren bloqueos y sufren accesos dispersos a memoria. Morrical et al. [118] ordenan los elementos de mallas volumétricas según la curva de Hilbert de sus centroides y agrupan elementos consecutivos, obteniendo en segundos una partición con buena localidad para cientos de millones de elementos.

2.8.5. Implicancias para esta tesis

La literatura distingue tres tipos de trabajo, que se corresponden con las etapas del algoritmo de cavidades. Los cálculos locales por elemento, como los predicados y las circunsferas, son trivialmente paralelos. La construcción de regiones que compiten por elementos requiere resolver conflictos, y las opciones probadas son los bloqueos con retroceso [102, 110], las particiones [58, 104], los conjuntos independientes por ronda [45, 111], el marcado atómico por prioridad [61] y la selección por índice en rondas [112]. Las fases irregulares o con poco trabajo concurrente, como la reparación [93], suelen limitar la aceleración global.

Dos observaciones, que son interpretaciones propias a partir de esta revisión, orientan el diseño de una versión paralela. Primero, el algoritmo secuencial procesa las semillas en orden de radio circunscrito, de modo que su resultado depende de ese orden; una versión paralela por rondas que resuelva los conflictos con una prioridad derivada de ese mismo orden, al estilo del marcado de Chen y Tan o del conjunto independiente de Ma et al., puede producir un resultado reproducible, pero no necesariamente idéntico al secuencial, y la referencia de corrección debe definirse en consecuencia. Segundo, Lo amplía las capas de celdas entre zonas hasta que las circunsferas de los tetraedros de frontera quedan acotadas [104]; una partición análoga para las regiones de esta tesis requeriría un criterio equivalente, porque su extensión depende de la circunsfera de cada semilla y no se conoce de antemano.

Esto favorece los esquemas por rondas sobre listas de trabajo regulares, como los de Chen y Tan [61].

2.9. Síntesis y posicionamiento de la tesis

Esta sección integra los hallazgos de la revisión, los ordena según las preguntas de investigación de la tesis y delimita el espacio que ocupa la propuesta.

2.9.1. Hallazgos según las preguntas de investigación

Construcción de poliedros mediante cavidades. La literatura muestra que unir tetraedros en celdas mayores es viable y útil: la recombinación hexaédrica [77, 84], la aglomeración para métodos poliédricos [30, 85, 86] y la familia Polylla [1, 3] lo hacen con criterios distintos. Lo que distingue a estos métodos es el alcance y la naturaleza de la decisión de unión: patrones de topología fija ordenados por calidad, particiones globales guiadas por un indicador, criterios locales ligados a la estabilidad temporal, como el paso de tiempo del agregado, o criterios locales geométricos, como la cara mayor en Polylla 3D. Ninguno usa como regla de unión la posición de los circuncentros respecto de la circunsfera de un tetraedro semilla; en los trabajos revisados, las circunsferas aparecen solo de forma auxiliar, en la construcción de mallas duales [49] o en el filtrado de elementos según su radio circunscrito [86]. Respecto de la elección de semillas, el antecedente más cercano es la prioridad por medida de los elementos en el refinamiento de Chen y Tan [61], y la selección codiciosa por calidad en la recombinación [77].

Estructura de datos. Los generadores tetraédricos eficientes usan estructuras mínimas basadas en el tetraedro y en la adyacencia por cara, con punteros en TetGen y con arreglos indexados en HXT [2, 58], y GPU Polylla aplica una representación indexada análoga en dos dimensiones [93]; la elección concreta debe hacerse a partir de las consultas que realiza el algoritmo [96]. No se encontró una estructura diseñada para representar a la vez una tetraedralización y su partición en regiones poliédricas, ni una que unifique algoritmos de una misma familia.

Viabilidad y calidad. Muchos análisis de los métodos poliédricos exigen que las celdas sean estrelladas, aunque algunos admiten uniones de subdominios estrellados, y la existencia y el tamaño del *kernel* son la medida más directamente vinculada con esa exigencia [4, 9]. La evidencia numérica indica que las hipótesis son más restrictivas de lo necesario, pero buena parte de ella es bidimensional o de orden bajo [9, 27]. Polylla 3D obtuvo una reducción cercana al 70 % del número de celdas, pero con poliedros muy lejos de ser convexos y sin evaluar su calidad ni su comportamiento numérico [1]. Existe un algoritmo de cálculo del *kernel* adecuado para celdas pequeñas [41].

Paralelización. La construcción concurrente de cavidades es un problema estudiado, con soluciones basadas en bloqueos [102, 110], particiones [58, 104], conjuntos independientes [45, 111] y prioridades atómicas en GPU [61]. Las fases irregulares, como la

reparación o la recuperación de la frontera, suelen limitar la aceleración global [58, 93], y la reproducibilidad del resultado depende de cómo se resuelven los conflictos [58, 111].

2.9.2. Posicionamiento respecto de los enfoques existentes

La Tabla 5 compara la propuesta de esta tesis con los enfoques más cercanos según las dimensiones que la revisión mostró relevantes.

Enfoque	**Alcance de la decisión**	**Criterio de unión o construcción**	**Garantía sobre la forma**	**Validación reportada**
Dual de tetraedralización [49]	Global	Dualidad con circuncentros, corregidos hacia el centroide cuando quedan fuera	Prueba conservadora de estrellamiento y optimización	Ejemplos, sin simulaciones
VoroCrust [51]	Global	Semillas de Voronoi	Celdas convexas	Geométrica, sin simulaciones
Recombinación en hexaedros [77]	Local, codiciosa	Patrones de hexaedros ordenados por calidad	Topología fija	Análisis modal y acústico
Aglomeración para VEM [30]	Global	Partición del grafo dual ponderada por calidad	Ninguna; calidad como peso	VEM
Aglomeración con redes neuronales [85]	Global	Bisección aprendida del grafo	Ninguna	Galerkin discontinuo
PVEM [86]	Local, codiciosa	Paso de tiempo del agregado	Ninguna	Flujos con superficie libre
Polylla 3D por caras [1]	Local	Cara terminal según un criterio de unión	Ninguna; repara poliedros no simples	Estadísticas de unión; VEM pendiente
Propuesta de esta tesis	Local, por semillas ordenadas	Circuncentro del vecino dentro de la circunsfera de la semilla	Ninguna a priori; *kernel* medido	Calidad geométrica

Tabla 5: Posicionamiento de la propuesta de esta tesis respecto de los enfoques de generación poliédrica más cercanos.

La propuesta comparte con Polylla la estrategia indirecta, local y sin inserción de puntos, y con la recombinación hexaédrica y PVEM la selección codiciosa a partir de elementos priorizados. Se diferencia de todos ellos en que su criterio de unión se deriva de la estructura de Delaunay de la malla de entrada. Se diferencia también de los métodos de aglomeración en que no resuelve una partición global, lo que la hace más simple y potencialmente paralelizable; en su forma actual, sin embargo, su criterio de unión no incorpora un objetivo explícito de calidad.

2.9.3. Interpretación geométrica del criterio de circunsferas

La revisión de la Sección 2.4 permite dar al criterio una interpretación geométrica, que es una lectura propia y no un resultado tomado de la literatura. Para dos tetraedros de Delaunay adyacentes, el segmento que une sus circuncentros es la arista de Voronoi dual a la cara que comparten [50]. Por lo tanto, que el circuncentro de un vecino directo quede dentro de la circunsfera de la semilla equivale a que esa arista de Voronoi sea más corta que el radio de la circunsfera de la semilla. En el límite en que cinco o más vértices son cosféricos, los circuncentros coinciden, la arista de Voronoi tiene longitud nula

y los tetraedros involucrados son una subdivisión arbitraria de una misma celda convexa de Delaunay [43]. El criterio puede leerse, entonces, como una relajación de la unión de tetraedros cosféricos: agrupa tetraedros cuyas circunsferas están «cerca» de coincidir, en una escala dada por el radio de la semilla (Figura 6). Las aristas de Voronoi cortas son, además, un defecto conocido de las mallas de Voronoi [51], de modo que el criterio tiende a unir precisamente las configuraciones que producen elementos duales degenerados. Para vecinos no directos, la comparación sigue haciéndose con la circunsfera de la semilla, de modo que los circuncentros de todos los tetraedros de una región quedan dentro de una misma bola, y no solo encadenados por aristas de Voronoi cortas.

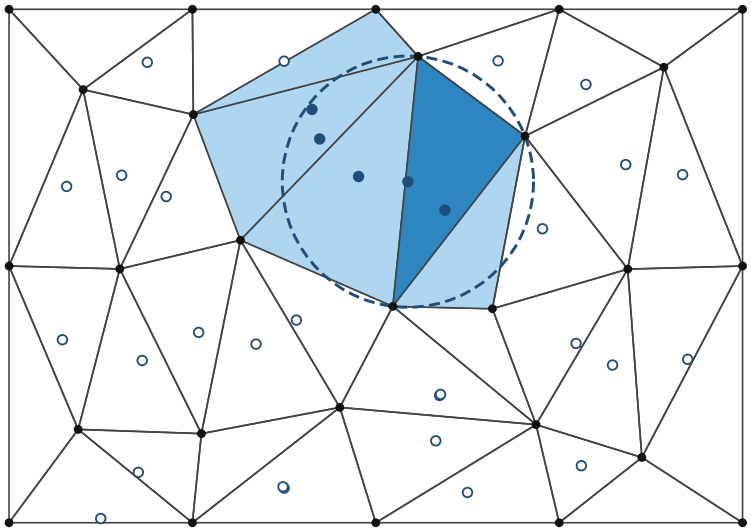


Figura 6: Análogo bidimensional de una región inducida por la circunsfera de un elemento semilla (elaboración propia). El triángulo semilla (azul oscuro) y su circunferencia circunscrita (segmentada) determinan qué vecinos se agregan: aquellos cuyo circuncentro (punto relleno) cae dentro de ella. Los circuncentros que quedan fuera se muestran vacíos. A diferencia de la cavidad del panel (b) de la Figura 2, la región no se define respecto de un punto que se inserta.

Esta interpretación también explica por qué el criterio no garantiza por sí mismo el estrellamiento. La cavidad de Bowyer–Watson es estrellada respecto del punto insertado porque está formada por los tetraedros cuyas circunsferas contienen ese punto [2]; la región de esta tesis, en cambio, está formada por tetraedros cuyos *circuncentros* están dentro de una circunsfera dada, condición que no involucra un punto común de visibilidad. La existencia del *kernel* debe, por lo tanto, medirse.

2.9.4. Brechas y riesgos

La revisión delimita también los riesgos que el trabajo debe abordar.

- **Estrellamiento no garantizado.** Ningún método de aglomeración revisado garantiza celdas estrelladas; la calidad se mide a posteriori o se incorpora como peso [30, 42, 85]. La propuesta comparte esta limitación.
- **Circuncentros fuera del dominio.** Los tetraedros de frontera de una malla de Delaunay pueden tener su circuncentro fuera del dominio [49], y varios métodos descartan esos elementos [53], lo que afecta a cualquier criterio basado en circuncentros cerca de la frontera.
- **Inestabilidad numérica.** Los circuncentros son puntos construidos: si se redondean a punto flotante, los predicados exactos posteriores dejan de garantizar decisiones correctas respecto de los circuncentros exactos [47], y son inestables en los *slivers*, que son frecuentes en mallas de Delaunay [36].
- **Restricciones del dominio.** La malla de entrada de un generador con restricciones solo es localmente de Delaunay [44], y una región no debería atravesar las caras del PLC si se desea preservar las fronteras internas.
- **Dependencia del orden.** La competencia de varias semillas por los mismos tetraedros hace que el resultado dependa del orden de procesamiento, del mismo modo que la reproducibilidad del generador paralelo de HXT depende del número de hilos [58]; una versión paralela debe definir qué resultado considera correcto [111].
- **Caras no planas y caras pequeñas.** Las caras de una celda formada por tetraedros son uniones de triángulos que pueden no ser coplanares, y muchas hipótesis suponen caras planas y un número acotado de ellas [9, 14], aunque algunos análisis admiten caras coplanares consecutivas [15] o muchas caras pequeñas, representando las interfaces mediante triángulos coplanares [21].

2.9.5. Conclusión

El estado del arte muestra que la generación indirecta de mallas poliédricas por unión de tetraedros es una línea activa, motivada por métodos numéricos que admiten celdas generales, aunque sus análisis suelen exigir que sean estrelladas. Los métodos existentes deciden las uniones mediante patrones, particiones globales o criterios locales que no controlan la forma de las celdas. La geometría de Delaunay ofrece un criterio local con significado propio, basado en la proximidad de las circunsferas, que no ha sido explorado como regla de unión en el corpus revisado. Esta tesis propone y evalúa ese criterio, mide la calidad de las celdas resultantes mediante su *kernel*, diseña una estructura de datos común a la familia Polylla y analiza su paralelización a la luz de las técnicas de construcción concurrente de cavidades. La ausencia de un método equivalente es una conclusión limitada al corpus revisado y no una prueba de novedad absoluta.

Capítulo 3

Trabajo anterior

La propuesta de esta tesis se sitúa dentro de la familia Polylla, cuyo principio es construir celdas mediante la unión de elementos de una malla simplicial preexistente. En dos dimensiones, las regiones se definen a partir de aristas terminales; en la extensión basada en caras, se agrupan tetraedros según una noción de cara terminal [1, 3]. El panorama de la Sección 2.6 permite ubicar esta estrategia entre los métodos indirectos. Aquí se desarrolla su mecanismo: cómo se delimitan las regiones, cómo se extrae su frontera y qué problemas resuelve la reparación. El propósito es distinguir las propiedades que estos procedimientos establecen de las que permanecen como preguntas de investigación para el algoritmo de cavidades.

3.1. Polylla en dos dimensiones

3.1.1. Caminos de propagación y regiones terminales

Sea $\mathcal{T}$ una triangulación y sea $\ell(t)$ la arista seleccionada como la más larga del triángulo t. El camino de propagación de la arista más larga (*longest-edge propagation path*, Lepp) de un triángulo t_0 es una sucesión de triángulos vecinos: cada paso cruza $\ell(t_i)$ para alcanzar el siguiente triángulo. En el caso interior, el camino termina al alcanzar dos triángulos que comparten su arista más larga. Esa arista es una *arista terminal* y ambos triángulos son terminales [3]. Por tanto, la propagación no selecciona un vecino cualquiera: la elección de salida depende del triángulo actualmente visitado.

La *región de arista terminal* asociada a una arista e reúne todos los triángulos cuyos Lepp terminan en e. Con una notación de conjuntos, la definición del artículo se puede expresar como

$$R_e = \cup_{t\in\mathcal{T}:\ \text{terminal}(\text{Lepp}(t))=e}\ t.$$

Varios caminos pueden converger a la misma arista; su unión define una sola región. Las regiones cubren el dominio sin superponerse y están delimitadas por aristas frontera. Cuando su contorno incluye aristas del borde del dominio, el artículo distingue una región terminal de borde [3].

Para una arista interior compartida por t_a y t_b, el etiquetado distingue tres casos: es terminal si ambos triángulos la seleccionan; es frontera si ninguno la selecciona; y es interna si la selecciona exactamente uno. Las aristas del borde del dominio se tratan como frontera. Cuando existen longitudes iguales, el método escoge un orden entre ellas para disponer de una única arista seleccionada por triángulo [3]. Conviene separar esta elección local de la construcción de la región: la primera proporciona las etiquetas; la segunda recupera los elementos conectados por ellas. El artículo advierte un caso degenerado: con triángulos equiláteros y ciertas selecciones de aristas iguales puede aparecer una región sin arista terminal. Propone corregirla seleccionando una misma arista como mayor en sus dos triángulos incidentes [3]. Por ello, el desempate local no debe entenderse como una garantía suficiente para evitar toda degeneración.

3.1.2. Etiquetado y construcción de polígonos

Polylla organiza el proceso en tres fases: etiquetado, recorrido y reparación. El etiquetado comienza por almacenar la arista más larga de cada triángulo. Luego recorre las aristas para marcar las fronteras y reconocer las terminales. Para cada terminal interior se guarda como semilla el triángulo incidente de menor índice; para una terminal de borde se guarda su único triángulo. Esta semilla permite iniciar la construcción del polígono correspondiente sin necesidad de construir explícitamente todos los Lepp [3].

Durante el recorrido se navega entre triángulos de la región y se almacenan las aristas frontera como una polilínea cerrada, con sus vértices en orden antihorario [3]. La Figura 4 del capítulo anterior ilustra la convergencia de los caminos; aquí interesa la consecuencia operativa: el etiquetado permite extraer el contorno a partir de una semilla. La salida de esta fase todavía puede contener polígonos no simples, por lo que no constituye por sí sola el resultado definitivo [3].

3.1.3. Aristas barrera y reparación

Una región puede contener aristas etiquetadas como frontera en su interior. El artículo las denomina *aristas barrera*: aunque pertenecen al sistema de fronteras, no forman parte del contorno exterior de la región. Su presencia origina polígonos no simples. Una *punta de barrera* es un extremo de una arista barrera que no es compartido por otra arista barrera o frontera [3]. En consecuencia, «no simple» describe aquí un problema específico de la representación y de la partición, y no debe emplearse como sinónimo de «no convexo».

La reparación divide estos polígonos aprovechando aristas ya existentes. Para cada punta se ordenan las aristas internas incidentes dentro de la región y se elige la central; cuando hay dos candidatas centrales, se escoge una de ellas. La arista seleccionada se convierte

en frontera y sus dos triángulos incidentes se incorporan a una lista de nuevas semillas. Después se repite el recorrido para construir los polígonos resultantes. Un arreglo de marcas evita que varias semillas reconstruyan el mismo polígono [3].

El objetivo de esta operación es conservar los vértices iniciales. El artículo descarta simplemente borrar las barreras porque ello exigiría eliminar sus extremos, que considera relevantes para la entrada. Además, la selección de aristas centrales busca producir piezas de tamaño similar [3]. La reparación es, por ello, una subdivisión de la región inicial; su finalidad no es imponer convexidad a todas las piezas.

Pendiente. Figura: representar una región bidimensional con una arista barrera, su punta y la arista interna elegida para dividirla. Preparar un esquema propio basado en las figuras 4 y 11 de P001, con etiquetas en español y referencia al artículo; comprobar que se distingan frontera exterior y barrera interior.

3.2. GPolylla y la representación compacta

La adaptación a GPU conserva las tres fases y utiliza una representación indexada de la triangulación. Cuatro arreglos almacenan coordenadas, vértices de cada triángulo, vecinos y un triángulo incidente por vértice. Este último es opcional y acelera la reparación. Los polígonos de salida se almacenan mediante un arreglo que contiene sus longitudes y secuencias de vértices, acompañado por otro con la posición de inicio de cada polígono [93]. Estos detalles complementan la discusión general de paralelismo en la Sección 2.8.

El etiquetado y el recorrido aprovechan la independencia de los elementos y de las regiones; la reparación requiere repetir subdivisiones hasta absorber las puntas de barrera en las fronteras. El artículo emplea operaciones atómicas para gestionar el almacenamiento concurrente de los polígonos [93]. La existencia de regiones independientes facilita distribuir el trabajo, pero la reparación exige un tratamiento distinto al de las fases iniciales.

En sus pruebas, los autores comparan versiones secuencial, paralela y mixta con triangulaciones de uno a ocho millones de puntos. La versión mixta ejecuta etiquetado y recorrido en GPU, y reparación secuencial; cada prueba se repite cinco veces. El etiquetado es la fase de mayor tiempo y el recorrido la de menor tiempo. La reparación paralela no presenta una ventaja significativa frente a la secuencial para esos casos. Aunque los resultados muestran aceleración, los autores indican que necesitan más experimentos antes de calcular el *speedup* [93]. Por ello, este antecedente respalda la viabilidad del paralelismo bidimensional, pero no proporciona un factor definitivo de mejora transferible a tres dimensiones.

Una segunda línea de trabajo aborda la representación compacta. La versión de Polylla basada en la estructura sucinta *pemb* reduce la representación de la conectividad a operaciones sobre tres vectores de bits estáticos. El artículo utiliza siete de las diecisiete consultas disponibles y deja como trabajo futuro explorar operaciones adicionales, como

inserción de vértices y cambios de aristas; también plantea el uso futuro en GPU [94]. Este antecedente se distingue de GPolylla: estudia otra representación y no debe interpretarse como evidencia de que esa representación compacta ya esté ejecutándose en GPU.

3.3. Extensión tridimensional

3.3.1. Alcance de Face y Edge

La distinción entre una extensión basada en aristas y otra basada en caras requiere cautela documental. P001 anuncia como trabajo en desarrollo una extensión tridimensional mediante regiones de arista terminal [3]. Sin embargo, el texto local de P003, correspondiente a la versión v2, presenta un algoritmo de regiones de cara terminal [95]. Por ello, se desarrolla a continuación la variante Face con el respaldo publicado de P005, sin extrapolar a Edge las definiciones bidimensionales ni los pasos de Face.

Pendiente. Polylla 3D Edge: obtener el texto primario de la variante basada en regiones de arista terminal, comprobando si corresponde a la versión anterior de P003 o al trabajo de Magdalena Álvarez citado en la propuesta. Incorporar una copia local y verificar la definición de camino entre tetraedros, las reglas de pertenencia a una región, las fases de construcción y reparación y sus resultados. El P003 local v2 no permite reconstruir esos pasos con rigor.

3.3.2. Cara terminal y camino de propagación

En Face, un *criterio de unión* ordena las caras de cada tetraedro y selecciona una sola como mayor; los empates se resuelven escogiendo una de ellas. La palabra «mayor» depende del criterio elegido. Una cara interior es *terminal* cuando es la seleccionada por ambos tetraedros incidentes. Si es la mayor de un tetraedro y pertenece al borde, se denomina cara terminal de borde [1].

El *camino de propagación de la cara mayor* (*largest-face propagation path*, Lfpp) parte de un tetraedro y pasa sucesivamente al vecino a través de la cara mayor del tetraedro actual. La región de cara terminal asociada a f se define como la unión de los tetraedros cuyos caminos terminan en esa misma cara:

$$R_f = \cup_{t \in \mathcal{T}:\ \text{terminal}(\text{Lfpp}(t)) = f}\ t.$$

Esta expresión adapta a tres dimensiones la notación de la Sección 3.1.1. P005 establece que las regiones de cara terminal no se superponen [1]. La selección de una sola cara por tetraedro es, por tanto, parte de la definición del método y no un detalle de presentación.

El etiquetado operativo considera frontera una cara que no es seleccionada por ninguno de los dos tetraedros incidentes, así como toda cara del borde del dominio. Las caras restantes permiten recorrer los tetraedros del agregado. Al igual que en dos dimensiones, una cara marcada como frontera puede quedar dentro de la región: se denomina *cara*

barrera. Una *punta de barrera* es ahora una arista incidente a una sola cara frontera, mientras las otras caras incidentes son internas [1]. El cambio de dimensión transforma así las puntas desde vértices en aristas.

3.3.3. Etiquetado, recorrido y reparación de Face

La primera fase calcula las caras seleccionadas por el criterio, identifica las terminales y escoge un tetraedro semilla por región. También construye un vector de bits de caras frontera. La salida auxiliar consiste en ese vector y un arreglo de semillas, que se emplean en las fases siguientes [1].

La segunda fase realiza una búsqueda en profundidad desde cada semilla. El recorrido avanza a través de las caras que no son frontera y registra las caras frontera como parte de la representación del poliedro. P005 representa la salida como una lista de poliedros, cada uno descrito por una lista de caras. Si una cara aparece repetida en la lista de un poliedro, el método reconoce una barrera y envía el elemento a reparación [1].

La tercera fase busca las aristas que son puntas de barrera. Para una arista candidata, se cuenta cuántas caras frontera son incidentes a ella; una sola incidencia identifica una punta en la configuración considerada. Se ordenan entonces las caras internas alrededor de esa arista, partiendo de la cara barrera, y se convierte la cara interna central en una nueva frontera. Los dos tetraedros incidentes a esa cara se guardan como semillas para volver a ejecutar el recorrido. Un arreglo de uso evita reconstruir varias veces un mismo poliedro [1].

Este procedimiento separa dos decisiones: la formación inicial de regiones por propagación y su subdivisión para resolver barreras. En particular, el criterio que seleccionó las caras no se sustituye, durante la reparación descrita, por una prueba del kernel. La reparación de P005 se formula en términos de incidencias, nuevas fronteras y recorridos, mientras la utilidad de los elementos para simulaciones permanece como trabajo futuro [1].

Pendiente. Figura: construir un esquema de Face que muestre una cara terminal, los tetraedros de su región y una cara barrera; añadir una segunda vista con una punta y la nueva cara frontera elegida durante la reparación. Basar las incidencias en las definiciones y el procedimiento de P005, secciones 2 y 3.

3.4. Criterios de unión y resultados publicados

3.4.1. Criterios investigados

P005 estudia dos métricas para seleccionar la cara mayor: el área de la cara y el radio de su circunferencia inscrita. En ambos casos se selecciona el máximo de las cuatro caras [1]. El radio inscrito se calcula sobre una cara triangular; no corresponde al radio de una esfera del tetraedro. Esta distinción importa al introducir la propuesta de cavidades, cuya definición en la introducción utiliza la circunsfera del tetraedro semilla.

Ambos criterios seleccionan una interfaz local de unión. Como interpretación del procedimiento, escoger la mayor cara según una métrica no equivale a optimizar directamente la forma de toda la región resultante. Los propios autores concluyen que las estrategias de unión deberían incorporar criterios de calidad del poliedro generado [1].

3.4.2. Resultados bidimensionales y alcance de su transferencia

En P001 se estudian conjuntos de puntos aleatorios dentro de un cuadrado, variando entre cien y un millón de puntos. Para entradas de más de diez mil puntos, el artículo informa promedios de 6,5 triángulos por polígono y 8,5 vértices por polígono [3]. Sus simulaciones preliminares comparan mallas Polylla y de Voronoi y encuentran precisión y costo computacional similares para la solución con VEM [3]. Estos resultados corresponden a las pruebas bidimensionales del artículo; no resuelven la evaluación de los poliedros de Face.

En conjunto, estos antecedentes motivan conservar la separación entre construcción, reparación y evaluación. Que la salida tenga menos elementos es una propiedad distinta de su comportamiento numérico, y que GPolylla acelere algunas fases no establece la escalabilidad del algoritmo tridimensional de esta tesis. Ambas comparaciones deberán realizarse bajo un protocolo propio.

3.4.3. Experimentos de Face

P005 evalúa cuatro tipos de malla tetraédrica en un cubo, denominados Random, Poisson, Grid y Quality. Cada tipo considera tamaños de 500, 1.000, 5.000, 10.000 y 50.000 vértices. El texto resume una unión de aproximadamente tres tetraedros por poliedro, una reducción del número de elementos cercana al 70 % y una proporción próxima a un quinto de poliedros con caras barrera [1]. Son resultados preliminares sobre esas entradas, no garantías del algoritmo para cualquier dominio.

La Tabla 6 recoge los valores totales de la tabla publicada para evitar reemplazar la comparación por un único promedio redondeado. La fila de barreras se conserva como estadística del artículo: no representa una medición del kernel ni permite deducir cuántas celdas son estrelladas.

Indicador	**Radio inscrito**	**Área máxima**
Reducción de elementos	67,4 %	69,5 %
Tetraedros por poliedro	3,0	3,0
Poliedros con barreras	21,0 %	20,1 %

Tabla 6: Valores totales reportados en la tabla 1 de [1] para los dos criterios de Face.

El artículo concluye que los resultados son casi independientes del criterio escogido y propone mejorar la definición de región con estrategias que consideren la calidad del

agregado. También señala que las celdas están lejos de ser convexas y que aún debe evaluarse su utilidad para simulaciones con VEM [1]. Por ello, el balance experimental ofrece una referencia de reducción y composición de las celdas, pero deja abierta su adecuación numérica.

3.5. Limitaciones y motivación del algoritmo de cavidades

La distinción de la Sección 2.3 entre convexidad y estrellamiento es fundamental para interpretar estos resultados. P003 v2 destaca la relevancia de que el kernel no sea vacío en las celdas destinadas al VEM [95].

> **Pendiente.** Bibliografía de P003: la entrada `salinasfernandez20233d` describe la v1 (2023, dos autores), pero el contenido citado corresponde a la v2 (arXiv:2310.03665v2, 2025, tres autores). Actualizar `literatura/papers.json` para identificar la versión consultada y regenerar `references.bib`.

Sin embargo, los resultados de P005 aquí revisados cuantifican reducción, tamaño de los agregados y barreras; no presentan una medición del porcentaje de poliedros con kernel vacío. Por tanto, que las celdas sean no convexas no permite concluir que carezcan de kernel, y la frecuencia de caras barrera tampoco sustituye esa medición.

> **Pendiente.** Diagnóstico de kernel en Polylla 3D: localizar el estudio primario al que alude la introducción cuando menciona ausencia de kernel. Verificar en él la variante evaluada, las entradas, el procedimiento de cálculo y los resultados exactos. Si no hay resultados publicados verificables, mantenerlo como pregunta abierta y diseñar, para ejecución posterior por el autor, una evaluación de Face y Edge que mida fracción de kernels no vacíos y razón entre volumen del kernel y volumen de la celda, distinguiendo las mallas antes y después de reparar barreras.

El límite que se desprende de estos antecedentes es preciso: la unión por una cara seleccionada y la reparación de barreras no constituyen una validación de la calidad geométrica o numérica del agregado completo. Esta lectura coincide con la necesidad de incorporar calidad y realizar simulaciones que explicita P005 [1]. No se dispone aquí de evidencia para afirmar que Face sea inutilizable, ni para atribuir a las cavidades una mejora ya demostrada.

En esta tesis se plantea cambiar la definición de la región candidata, utilizando la circunsfera de un tetraedro semilla como referencia para las cavidades. El capítulo siguiente deberá precisar la pertenencia a esas regiones, la selección de semillas y la resolución de conflictos entre candidatos. La motivación no es únicamente reducir primitivas: es investigar si otra regla de agrupación permite obtener celdas cuya calidad pueda verificarse explícitamente. La existencia de kernel, la reducción de elementos, el tiempo y la memoria permanecen como dimensiones de evaluación, de acuerdo con las preguntas de investigación

de la introducción. Se trata de una propuesta que debe contrastarse, no de un resultado que pueda inferirse de los antecedentes de Polylla.

Capítulo 4

Algoritmo de cavidades

4.1. Generalización y alcance

Este capítulo distingue el algoritmo secuencial implementado en `3D-GPolylla`, sus herramientas de inspección y la propuesta paralela. La hipótesis de calidad y eficiencia permanece por evaluar.

Como organización conceptual de este capítulo, se caracteriza la familia Polylla tridimensional mediante dos ejes: la forma de unión de las primitivas y el criterio que decide cuáles unir. La revisión de Sección 2.6 sitúa los antecedentes de esta familia; la variante de cavidades utiliza vecinos por cara para recorrer la malla, pero decide su incorporación comparando circuncentros con la circunsfera de una semilla. Por consiguiente, la conectividad usada para recorrer y la condición geométrica de aceptación son componentes distintos.

Esta separación no equivale a disponer de una biblioteca que combine libremente cualquier forma y cualquier criterio. La interfaz `Algorithm` recibe una `Mesh` y devuelve una `PolyMesh`, mientras que `CavityAlgorithm` concreta ese contrato. En el código activo, tanto el orden por radio como la condición de inclusión están escritos directamente en `cavity.cpp`; no se reciben como estrategias intercambiables. La adaptación de Polylla Face y Edge permanece en la lista de trabajo del `README.md`. Así, la generalización es una dirección de organización del software, con una interfaz común inicial, y no una cobertura implementada de toda la familia.

El borrador de rediseño en `docs/superpowers/specs/` propone representar mallas, construir poliedros y medir resultados como capacidades permanentes. Se considera una especificación, no una implementación. La propuesta original priorizaba regiones de cara terminal y consideraba después cavidades; este capítulo sigue el alcance vigente de la introducción.

Pendiente. Justificación de diseño: precisar con el autor qué parte de la separación «forma de unión × criterio» constituye una contribución de la tesis. Completar el contrato común con las precondiciones y salidas exigidas a Face, Edge y Cavity antes de afirmar que el ecosistema quedó unificado.

4.2. Definiciones operacionales

Sea $M = (V, F, T)$ la malla de entrada representada por el programa: V contiene las coordenadas de los vértices, F las caras triangulares y T los tetraedros. Cada tetraedro almacena cuatro índices de vértice y cuatro índices de cara. Cada cara conserva tres índices de vértice y hasta dos tetraedros incidentes; el valor `-1` representa una incidencia ausente. Estas son propiedades de la representación, que presupone una entrada compatible con esas capacidades.

Un **tetraedro candidato a semilla** es cualquier tetraedro de entrada. Una **semilla efectiva** es un candidato que todavía no tiene propietario cuando llega su turno en la secuencia de procesamiento. Esta distinción es necesaria porque `seeds()` conserva la lista ordenada de todos los candidatos, aunque algunos hayan sido absorbidos antes de iniciar una región propia.

Para un tetraedro no degenerado t, se denotan por c_t y r_t el centro y el radio calculados por `circumsphere`. La circunsfera es la esfera que pasa por sus cuatro vértices; el radio se obtiene como la distancia entre el centro calculado y el primer vértice. No se trata de la esfera inscrita definida en la introducción. La función forma matrices de orden cuatro, calcula determinantes y divide por dos veces el determinante de la matriz de coordenadas homogéneas. No contiene una comprobación previa de determinante nulo.

Para una semilla s, la condición operacional de aceptación de un vecino t es

$$\|c_t - c_s\| < r_s + \varepsilon, \quad \varepsilon = 10^{-8}.$$

El valor de ε corresponde a `TOLERANCE` en `utils.h`. Aunque el radio se almacena como `double` y los determinantes se calculan con `Matrix4d`, `Vertex` hereda de `Eigen::Vector3f`: las coordenadas del centro se almacenan en precisión simple. La desigualdad reproduce la función `Cavity::isInside`, incluida su tolerancia absoluta.

La **cavidad construida** C_s es el conjunto de tetraedros alcanzados desde s por el recorrido de vecinos por cara, aceptados con esa condición y todavía sin propietario al ser visitados. La semilla se incorpora directamente; el test geométrico se aplica a sus vecinos. Debe distinguirse la esfera de referencia de la región discreta que finalmente se construye. Tampoco se exige que los vértices del vecino estén dentro de la esfera, que su circunsfera esté contenida en ella ni que el volumen del tetraedro completo esté contenido en la bola.

La cavidad de Bowyer–Watson responde a otra consulta: reúne tetraedros cuyas circunsferas contienen un punto de inserción, para sustituir esa región durante la actualización de la tetraedralización [2]. En el procedimiento aquí estudiado se fija la esfera de una semilla y se consultan los circuncentros de los vecinos. No se inserta un punto ni se vuelve a tetraedralizar la región. Los fundamentos de la primera construcción se desarrollan en Sección 2.4; la coincidencia del nombre no transfiere sus garantías a esta segunda operación.

El **poliedro resultante** se representa mediante la unión de los tetraedros de C_s, una lista de caras de frontera y una lista de vértices únicos. Esta última se forma con todos los vértices de los tetraedros incorporados, no únicamente con los vértices de las caras externas. La frontera continúa triangulada: la construcción no fusiona triángulos coplanares en caras poligonales mayores.

Pendiente. Figura: preparar, a partir de una malla pequeña elegida por el autor, una vista con semilla, circunsfera, circuncentros aceptados y rechazados, y caras de frontera. Mostrar un tetraedro cuyo circuncentro sea aceptado sin que se exija contener todos sus vértices; contrastar con la cavidad de Bowyer–Watson de Figura 2. No representar esta figura como un resultado experimental.

4.3. Selección de semillas y construcción

4.3.1. Preparación y prioridad

La entrada del algoritmo es `CavityAlgorithm::operator()`. La primera fase, `labelCavities`, calcula una circunsfera por tetraedro, incorpora su índice a la lista de candidatos y ordena esa lista por radio creciente. Inicializa además `owners` con `-1`. No hay un filtro previo por calidad, volumen o posición espacial: todos los tetraedros participan como candidatos.

El orden induce una prioridad secuencial. Cuando un tetraedro ya pertenece a una región anterior, `buildCavities` omite su candidatura. Por ello no basta con definir la condición geométrica para especificar el resultado: también se debe conservar el orden de las semillas. El comparador de `std::sort` solo evalúa la desigualdad entre radios y no incluye un desempate por índice. El código no especifica, por tanto, una prioridad total entre candidatos de igual radio.

Pendiente. Justificación de diseño: documentar por qué se eligió comenzar por el menor radio. Contrastar, cuando el autor autorice los experimentos, orden creciente, decreciente y un desempate explícito por índice, registrando número de poliedros, tetraedros aislados y presencia de kernel. La implementación actual demuestra la elección, pero no su superioridad.

4.3.2. Recorrido y propiedad exclusiva

La segunda fase copia los arreglos de vértices, caras y tetraedros al resultado. Para cada semilla efectiva crea tres contenedores: un conjunto ordenado de vértices únicos, una lista

de caras y una lista de tetraedros. El recorrido `DepthFirstSearch::operator()` asigna inmediatamente el tetraedro visitado a la semilla y lo añade a la lista. Luego incorpora sus vértices al conjunto y examina sus cuatro caras.

Si la cara no tiene un tetraedro al otro lado, se incorpora a la frontera. Si el vecino ya tiene propietario, se añade la cara solo cuando ese propietario es otra semilla. Cuando el vecino está libre, se compara su circuncentro con la esfera de la semilla inicial: si satisface la condición, se continúa recursivamente; en caso contrario, se registra la cara de frontera. La esfera usada no cambia al avanzar por el recorrido.

```
construir(M):
  para cada tetraedro t:
    esfera[t] = circumsphere(t, M)
    candidatos.agregar(t)
  ordenar candidatos por esfera[t].radio creciente
  dueño[t] = -1 para todo t
  R.vértices, R.caras, R.tetraedros = copias de M
  para s en candidatos:
    si dueño[s] != -1: continuar
    U = conjunto vacío; B = []; C = []
    visitar(s, s, U, B, C)
    para t en C: R.tetraedros[t].poliedro = |R.celdas|
    R.celdas.agregar(Poliedro(U, B, C))
  devolver R
```

Listado 1: Preparación y construcción secuencial. Corresponde a `labelCavities`, `buildCavities` y `CavityAlgorithm::operator()` en `src/cavity.cpp`.

```
visitar(t, s, U, B, C):
  dueño[t] = s; C.agregar(t)
  insertar en U los cuatro vértices de t
  para f en caras(t):
    u = tetraedro al otro lado de f
    si u == -1:
      B.agregar(f); continuar
    si dueño[u] != -1:
      si dueño[u] != s: B.agregar(f)
      continuar
    si esfera[s].contiene(esfera[u].centro):
      visitar(u, s, U, B, C)
    en otro caso:
      B.agregar(f)
```

Listado 2: Recorrido recursivo de `DepthFirstSearch::operator()` en `src/cavity.cpp`. La pertenencia se evalúa siempre respecto de la semilla.

Los identificadores de propietario tienen dos significados distintos. En `owners`, el valor es el índice del tetraedro semilla. En `Tetrahedron::polyhedron`, asignado después de completar el recorrido, es el índice de la celda en `result.cells`. No deben intercambiarse al consultar una región o exportar resultados. Los accesores de `CavityAlgorithm` permiten consultar las esferas, los candidatos y los propietarios después de la ejecución.

4.3.3. Propiedades deducibles y límites

Bajo una conectividad válida y un cálculo geométrico definido, cada tetraedro adquiere propietario a lo sumo una vez: el recorrido solo entra en vecinos libres y los marca antes de explorar sus caras. También obtiene propietario al menos una vez, porque todos los tetraedros aparecen como candidatos y cualquiera que permanezca libre inicia su propia región. De ambas observaciones se deduce una partición de los índices de tetraedros. Cada región es conexa por caras, pues toda incorporación posterior a la semilla ocurre desde un vecino ya incorporado. Estas son deducciones del control de flujo, no mediciones.

La exclusividad y la conectividad no constituyen por sí solas una demostración de simplicidad topológica, convexidad o existencia de kernel. El predicado no consulta esas propiedades y la construcción no rechaza una región por carecer de ellas. Las métricas se calculan después, mediante otro módulo. La lectura correcta de la hipótesis es, por tanto, que esas propiedades deben evaluarse, no que el crecimiento las garantice.

Para N tetraedros, F caras y V vértices almacenados, el cálculo de circunsferas efectúa una cantidad fija de operaciones matriciales por tetraedro. La ordenación requiere $O(N \log N)$ comparaciones. En el recorrido, cada tetraedro se visita una vez y examina cuatro caras; sus cuatro inserciones en `std::set` tienen costo logarítmico en el tamaño del conjunto. Así, para esta construcción y las copias de entrada se deduce una cota superior $O(V + F + N \log N)$, sin incluir lectura, escritura ni estadísticas. Se supone costo constante para las operaciones geométricas de tamaño fijo y los accesos por índice.

Los contenedores auxiliares, la salida y las copias requieren espacio $O(V + F + N)$ bajo esas mismas condiciones. La profundidad de la recursión puede crecer con el número de tetraedros de una región; la cota asintótica no evita un agotamiento de pila en recorridos grandes. Estas cotas describen la estructura del programa, no su escalabilidad observada.

Pendiente. Robustez y validación: definir precondiciones de entrada y tratamiento de tetraedros degenerados, centros no finitos, radios iguales y cambios de escala. Añadir casos de prueba para cada situación y justificar una tolerancia acorde con la precisión de las coordenadas. Sustituir la recursión por una pila explícita si se necesita controlar su memoria; verificar que conserve el orden de visita y la partición.

4.4. Reparación y cobertura de pruebas

Existe una función `fixCavities`, pero su llamada está comentada en el operador principal. En consecuencia, la salida descrita en Sección 4.3 no incluye una fase activa de reparación ni de refinamiento. Una celda formada por un solo tetraedro permanece como tal. Tampoco se añaden vértices ni se subdividen los tetraedros durante las fases activas.

El código inactivo ensaya incorporar los tetraedros aislados al vecino que minimice la distancia entre circuncentros dividida por el radio del vecino. Actualiza una lista de

tetraedros y el identificador de poliedro, pero no reconstruye en ese paso las listas de vértices y caras. Después genera una lista nueva de poliedros iterando sobre tetraedros y copiando sus celdas propietarias, sin una deduplicación explícita por poliedro. La propia función se presenta como provisional y enumera validación, identificación de frontera y controles de calidad como tareas por completar. No corresponde describirla como reparación terminada ni suponer que basta con habilitar su llamada.

`test/cavity_test.cpp` contiene una prueba de ejecución que comprueba que los contenedores del resultado no estén vacíos y que se conserve la cantidad de vértices, además de una comparación con una malla poliédrica de referencia. Hay pruebas adicionales para lectura TetGen y escritura VisF. La inspección de esas pruebas aporta ejemplos del contrato esperado; no equivale a ejecutarlas ni a validar exhaustivamente las propiedades geométricas anteriores. En particular, la prueba de lectura aún referencia `mesh.cells`, mientras que `Mesh` expone `tetras`; esa prueba requiere actualización antes de emplearla como validación de la interfaz vigente.

Pendiente. Reparación: decidir si conservar tetraedros aislados es admisible para el uso previsto. Si se requiere absorberlos, definir el criterio de aceptación, reconstruir frontera y vértices del agregado y mantener una correspondencia única entre tetraedro y poliedro. Probar conservación de volumen con orientación controlada, exclusividad de pertenencia y ausencia de caras internas en la frontera; evaluar el kernel antes y después de cada modificación.

4.5. Representación de la malla y objetivo paralelo

La representación pública emplea índices enteros y arreglos dinámicos. `Mesh` agrupa vectores de vértices, caras y tetraedros; `PolyMesh` extiende esa clase con un vector de `Polyhedron`. Este último contiene vectores de índices de vértices, caras y tetraedros constituyentes. La malla de salida conserva, por tanto, la subdivisión tetraédrica además de la agrupación poliédrica. La reducción del número de celdas expuestas no implica eliminar del almacenamiento los tetraedros originales.

La vecindad se obtiene indirectamente: un tetraedro referencia una cara, y esta contiene las dos incidencias que permiten escoger el tetraedro del otro lado. No existe en `Tetrahedron` un arreglo adicional de cuatro vecinos. En la lectura, `buildFaces` genera las caras de cada tetraedro y elimina duplicados con un conjunto de dispersión; luego las ordena por sus índices de vértice ordenados. `buildConnectivity` usa un mapa de caras a posiciones locales para completar ambas direcciones de incidencia. El archivo `neighbours.cpp` conserva código comentado, por lo que no representa una segunda construcción activa.

La capacidad fija de dos incidencias por cara presupone que no se intentará registrar una tercera. El bucle que completa esa información no comprueba explícitamente el límite antes de escribir. Además, la numeración de entrada requiere atención: los lectores insertan un elemento ficticio si el primer índice es uno, tanto para vértices como para tetraedros. Los

recorridos posteriores comienzan en cero. Esto identifica una condición que debe revisarse antes de aceptar de manera general entradas con numeración desde uno.

Para el objetivo específico 2, los índices y las incidencias fijas ofrecen una base para separar topología inmutable y asignaciones. Sin embargo, los vectores por poliedro, el conjunto ordenado, la recursión y las escrituras secuenciales en `owners` requieren adaptación. No constituyen una implementación masivamente paralela. Las representaciones alternativas se revisan en Sección 2.7.

Pendiente. Estructura de datos: validar numeración, referencias y multiplicidad de incidencias durante la lectura. Para el objetivo paralelo, especificar una representación aplanada de las listas de cada poliedro con desplazamientos y tamaños; comparar memoria total, copias y accesos con la representación actual, incluyendo la subdivisión tetraédrica retenida. Registrar el costo de construir esa representación, no solo el de recorrerla.

4.6. Arquitectura y flujo de datos

El proyecto define una biblioteca C++20, `GPolyllaLib`, accesible también mediante el alias `GPolylla::gpolylla`. CMake permite habilitar por separado el ejecutable, las pruebas y el renderer. Las interfaces `Reader`, `Algorithm` y `Writer` separan lectura, transformación y escritura; las realizaciones aquí utilizadas son `TetgenReader`, `CavityAlgorithm` y `VisFWriter`. Se dispone así de un núcleo reutilizable por las dos aplicaciones, aunque la interfaz pública mantiene geometría, conectividad y comportamiento en un mismo encabezado.

`TetgenReader` recibe rutas a archivos `.node` y `.ele`, lee coordenadas y cuádruplas de vértices y construye las caras y su conectividad. No se debe confundir este lector con una invocación del generador TetGen: el flujo procesa una malla ya disponible. `VisFWriter` escribe el encabezado ASCII de malla de poliedros, las coordenadas, las caras dirigidas y la pertenencia de caras a celdas. El bloque de vecindad entre polígonos se escribe como cero. No hay en estas interfaces una lectura VisF ni una escritura TetGen implementadas.

La orientación de las caras de salida se determina respecto del vértice opuesto del tetraedro incidente. El escritor recorre los tetraedros de cada poliedro, selecciona sus caras de frontera y revierte el orden de sus vértices cuando lo indica el predicado de orientación. La exportación mantiene caras triangulares y registra las incidencias dirigidas por celda; no construye una representación compacta de caras poligonales compartidas entre poliedros.

El ejecutable admite las rutas de entrada y salida y las opciones `--make-stats` y `--detail-stats`. Mide por separado lectura, ejecución del algoritmo y escritura mediante un reloj de alta resolución, con duraciones en milisegundos. Las estadísticas se calculan después de esas fases. Su suma, exportada como tiempo total, no incluye el cálculo de estadísticas ni la exportación detallada. La opción de detalle se procesa dentro de la rama de generación de estadísticas y produce archivos OFF por poliedro y `details.csv`.

El logger declara el nivel `INFO` y escribe en la salida estándar con ese prefijo; no configura archivos de trazas ni mide memoria.

Pendiente. Reproducibilidad del software: fijar revisiones de las dependencias QuickHull y polyhedron-kernel, actualmente referidas a ramas en CMake, y eliminar la inclusión de ImGui desde una ruta de compilación en `stat.cpp`. Documentar una configuración reproducible antes de asociar mediciones a esta versión. La compilación y ejecución del programa quedan a cargo del autor.

4.7. Métricas implementadas y su interpretación

El módulo `stat` construye un registro `PolyStat` por poliedro. Almacena una envolvente convexa, un kernel opcional y tres razones escalares. Estas medidas permiten instrumentar la pregunta sobre calidad, pero no establecen por sí solas la admisibilidad de una malla para un método numérico. Las condiciones de uso de las celdas se discuten en Sección 2.3 y Sección 2.2.

4.7.1. Área, volumen y envolvente convexa

El área de una cara triangular se calcula como la mitad de la norma del producto cruzado de dos de sus lados. `Polyhedron::area` suma esas áreas sobre las caras de frontera. Para el volumen, `Tetrahedron::volume` divide por seis el producto mixto de tres aristas desde un mismo vértice; `Polyhedron::volume` suma esos valores sobre los tetraedros constituyentes. No aplica valor absoluto por tetraedro ni al total. La orientación de la entrada condiciona, por tanto, el signo y las posibles cancelaciones de esta magnitud.

`Hull` entrega a QuickHull las coordenadas referenciadas por `poly.vertices` y construye caras triangulares a partir de los índices devueltos. Sus métodos de área y volumen operan sobre esa superficie. En particular, `generalVolume` utiliza el primer vértice como referencia, suma productos mixtos asociados a las caras, toma el valor absoluto de la suma y divide por seis. Es una ruta de cálculo diferente de la suma tetraédrica usada para el poliedro original.

La envolvente calculada constituye información para estudiar convexidad, pero `PolyStat` no contiene un indicador booleano de poliedro convexo ni calcula una razón entre volumen del poliedro y volumen de su envolvente. No se deben atribuir esas definiciones al campo `volumeRatio`.

4.7.2. Kernel y razones de calidad

`Kernel` remapea los índices de vértice a una numeración local, orienta las caras mediante `getDirectedFaces` y delega el cálculo a `PolyhedronKernel`. Las caras devueltas con más de tres vértices se triangulan en abanico. `empty()` comprueba si el vector de vértices obtenido está vacío; `computeStats` representa ese caso con `std::nullopt`. Así, la presencia de kernel registrada por el programa es operacionalmente la presencia de vértices en la salida de esa dependencia.

Para un poliedro P, su envolvente $H(P)$ y el kernel calculado $K(P)$, las tres razones implementadas son:

$$q_{e(P)} = \ell_{\min}/\ell_{\max}, \quad q_{v(P)} = V(P)/V(K(P)), \quad q_{a(P)} = A(P)/A(H(P)).$$

Los extremos $\ell_{\min}$ y $\ell_{\max}$ se obtienen recorriendo las tres aristas de cada triángulo de frontera. Por tanto, incluyen las aristas de la triangulación de la superficie y no solo las de una eventual representación con caras poligonales fusionadas. El campo `edgeRatio` almacena q_e; `surfaceRatio` almacena q_a. Para `volumeRatio`, el programa asigna cero si no hay kernel y usa q_v en caso contrario. No invierte esa fracción: el numerador es el volumen del poliedro.

Esta convención mezcla ausencia de kernel con un valor numérico especial en el mismo campo. Además, las divisiones no comprueban explícitamente volumen de kernel, área de envolvente o longitud máxima nulos. El cálculo de volumen superficial accede al primer vértice, por lo que requiere una colección no vacía. Estas condiciones delimitan cómo deben interpretarse estadísticas de entradas degeneradas o resultados incompletos.

Pendiente. Métricas: justificar las tres razones respecto de los requisitos del método numérico que se utilizará. Separar «sin kernel» de cero numérico, definir el manejo de denominadores nulos y verificar orientación y volumen con casos analíticos. Si se incorpora una medida de convexidad volumétrica, definirla y probarla como una métrica nueva, sin renombrar silenciosamente `volumeRatio`.

4.7.3. Agregación y archivos de salida

El renderer cuenta poliedros de un solo tetraedro y calcula mínimo, máximo y promedio del número de tetraedros por celda y de las tres razones. También calcula la fracción de celdas cuyo registro contiene kernel, y cuenta los vértices usados por las caras de frontera. Son cálculos presentes en el código; este capítulo no proporciona valores obtenidos sobre mallas concretas.

El ejecutable usa otra definición para `points coverage`: marca los vértices incluidos en `poly.vertices` y divide su cantidad por el tamaño del vector de vértices de entrada. Como la construcción incorpora los vértices de todos los tetraedros asignados, esta medida no cuenta exclusivamente los vértices de la superficie poliédrica. Tampoco es la razón de reducción de tetraedros a poliedros. Los conteos de ambas clases de celdas se exportan por separado.

La función `createStats` presenta inconsistencias visibles en la serialización: faltan separadores después de algunos promedios y se vuelven a escribir los extremos de superficie donde el encabezado anuncia extremos de arista. En cambio, la salida detallada enumera

por celda las tres razones, presencia de kernel y áreas y volúmenes de poliedro, envolvente y kernel. La existencia de esa ruta no valida automáticamente todos sus valores.

Pendiente. Exportación: corregir y probar la correspondencia entre columnas y valores de `createStats`, incluidos separadores y extremos de arista. Definir por separado reducción de celdas y uso de vértices de frontera. Antes del capítulo experimental, contrastar cada fila con el registro detallado y registrar por separado el tiempo de estadísticas y exportaciones auxiliares.

4.8. Visualización y trabajo de integración

El renderer combina OpenGL, GLFW e ImGui en una aplicación con visor y paneles de información y herramientas. Su acción `Start`, para la opción Cavity, ejecuta `CavityAlgorithm`, recupera los propietarios, actualiza los colores y calcula estadísticas. Las opciones Face y Edge están visibles, pero muestran `Not implemented`. La disponibilidad de controles en pantalla no equivale, por tanto, a disponer de las tres variantes.

Las acciones para centrar la vista, mostrar vecinos o circunsfera, aislar una celda y añadir un plano contienen solo comentarios. El panel de memoria muestra `N/A`. Esos controles no son funcionalidades terminadas, y la visualización no prueba que la construcción se ejecute en GPU.

El `README.md` mantiene como tareas abiertas añadir mallas de prueba, visualizar la envolvente convexa, las cavidades y los tetraedros aislados, centrar la cámara en un poliedro, corregir los controles de zoom y clic, y adaptar Face y Edge. Marca como completados el cálculo de volumen, el cálculo de envolvente convexa y la incorporación de cálculos a la visualización. Esa lista expresa el estado de seguimiento del repositorio; los detalles y limitaciones de las funciones se han descrito aquí desde el código.

Pendiente. Integración y visualización: incorporar las tareas abiertas del README a una verificación funcional. Conectar cada acción del visor con una operación comprobable; añadir vistas de cavidad, envolvente y celdas aisladas; revisar cámara y selección. Incorporar nuevas mallas de prueba y completar adaptadores Face y Edge antes de ofrecer comparaciones entre variantes desde la interfaz.

4.9. Propuesta de versión paralela

Esta sección es un diseño propuesto para el objetivo específico 5 y la cuarta pregunta de investigación. No describe una implementación paralela disponible. El esquema conceptual se apoya en la separación entre cálculos locales, crecimiento con conflictos y construcción de salida revisada en Sección 2.8. No se trasladan al algoritmo de esta tesis garantías de las cavidades de inserción de Delaunay, porque aquí la operación es una asignación de tetraedros existentes.

Se propone comenzar por los cálculos de circunsfera: cada tarea leería los cuatro vértices de un tetraedro y escribiría su centro y radio en una posición propia. La ordenación produciría prioridades globales, con un desempate por índice especificado explícitamente. Esta adición sería una decisión nueva: antes de comparar salidas se debería fijar el mismo desempate en la referencia secuencial.

Para sustituir la recursión se propone una lista de trabajo de pares `(semilla, tetraedro)`. Cada ronda examinaría las caras vecinas y generaría propuestas de incorporación con el predicado ya definido. Las propuestas pueden coincidir en un mismo tetraedro; ejecutar en paralelo la escritura ordinaria de `owners` no resuelve esa competencia. Se propone separar la producción de candidaturas de su aceptación, arbitrar las coincidentes con prioridades y publicar después el estado de propiedad aceptado.

La prioridad por sí sola no demuestra equivalencia con el recorrido secuencial. Un tetraedro reclamado puede ser el acceso que otra región necesita para alcanzar vecinos posteriores. Por ello, un diseño que permita reemplazar propietarios tendría que retirar o recalcular las expansiones dependientes; uno que solo acepte propietarios definitivos tendría que demostrar que su selección no contradice las prioridades pendientes. Se propone estudiar primero un esquema por lotes de regiones sin tetraedros candidatos compartidos, posponiendo las regiones en conflicto. La equivalencia, la terminación y el costo de descubrir esos lotes quedan por establecer.

Para generar las listas de salida se propone contar por región los tetraedros y caras aceptados, calcular desplazamientos mediante sumas prefijas y llenar arreglos aplanados. Los vértices podrían deduplicarse mediante pares `(región, vértice)` ordenados, en lugar de mantener un `std::set` por tarea. Esta representación debe distinguir caras internas, interfaces entre regiones y frontera exterior. La propuesta reutiliza los índices de entrada, pero requiere implementar la compactación y definir quién escribe cada incidencia.

Se propone validar primero propiedad exclusiva y reproducibilidad en memoria compartida, antes de abordar transferencias y granularidad en GPU. La evaluación debería distinguir tiempo del núcleo y tiempo de extremo a extremo, incluyendo lectura y transferencias. Es una secuencia de desarrollo propuesta.

Pendiente. Diseño paralelo: escoger con el autor el contrato de corrección: igualdad de partición con la referencia secuencial o partición válida bajo una nueva prioridad por rondas. Formalizar progreso y resolución de conflictos; implementar casos con regiones superpuestas y comprobar cobertura, exclusividad, conectividad y reproducibilidad al variar hilos. Solo después medir tiempo por fase, trabajo repetido, rondas, memoria máxima y transferencias, con tamaño de problema fijo y creciente. No se han ejecutado estas mediciones.

Capítulo 5

Diseño experimental

5.1. Alcance y preguntas de evaluación

Este capítulo establece un diseño experimental para contrastar la hipótesis de la introducción. Los guiones de las tablas indican ausencia de medición, nunca un valor nulo. El objeto de evaluación será la construcción secuencial descrita en Sección 4.3; la reparación y el diseño paralelo requieren desarrollos adicionales antes de incorporarse como variantes. Las cuatro preguntas requieren, respectivamente, validar la construcción, comparar representaciones, evaluar viabilidad y comprobar el diseño paralelo. La Tabla 7 establece sus observables.

Pregunta	**Observables propuestos**	**Comparación necesaria**
1	Partición, conectividad y kernel	Órdenes de semillas; con y sin reparación validada
2	Memoria y tiempos por fase	Vectores actuales frente a listas aplanadas futuras
3	Reducción, calidad, fallos y recursos	Cavidades, Face y Edge; igual entrada
4	Corrección, tiempo y conflictos	Secuencial frente a paralela futura

Tabla 7: Relación entre preguntas y verificaciones propuestas.

Se distinguirán validez topológica, calidad y costo. Reducir celdas no bastará para sostener la hipótesis si aumentan los fallos o se pierde calidad. La aptitud para VEM requerirá validación numérica, según Sección 2.2.

Antes de medir se propone fijar criterios de aceptación: cada tetraedro válido pertenecerá a una sola celda, cada celda será conexa por caras y su frontera se contrastará con las incidencias del conjunto de tetraedros. Se comprobará la conservación del dominio mediante volúmenes con orientación controlada. Los casos que incumplan estas verificaciones se registrarán como fallos; no se incorporarán silenciosamente a promedios de calidad.

Pendiente. Validación inicial: implementar un verificador de cobertura, exclusividad, conectividad y frontera; contrastarlo con mallas pequeñas de solución inspeccionable. Definir con el autor tolerancias y criterios de aceptación antes de procesar el conjunto completo. Registrar entradas rechazadas y causa de rechazo.

5.2. Métricas y reglas de interpretación

Para una entrada con N_T tetraedros válidos y una salida con N_P poliedros, se propone medir la reducción $R = 1 - \frac{N_P}{N_T}$ y la razón $\frac{N_T}{N_P}$, siempre que ambos conteos sean positivos. Se registrarán mínimo, mediana y máximo de tetraedros por celda y la fracción de celdas con un único tetraedro. Los conteos se obtendrán de elementos válidos, sin confundirlos con posiciones reservadas de los vectores. El lector actual puede insertar un elemento ficticio para numeración desde uno; esta condición deberá resolverse antes de medir. Para el kernel se propone registrar $f_K = \frac{N_K}{N_P}$, donde N_K cuenta celdas con kernel calculado no vacío, acompañado por la cantidad de cálculos fallidos o indeterminados. La implementación representa la presencia de kernel mediante un registro opcional que depende de los vértices devueltos por la biblioteca externa. Por ello, f_K describirá inicialmente ese procedimiento, cuya validación con casos analíticos es un requisito previo. Las razones disponibles son las de Sección 4.7: arista mínima sobre máxima, volumen del poliedro sobre volumen del kernel y área del poliedro sobre área de su envolvente convexa. La razón volumétrica no mide volumen del poliedro sobre volumen de la envolvente. Se propone informar sus distribuciones por malla, separando celdas sin kernel, denominadores nulos y valores no finitos. No se promediará el cero que el código usa como sustituto de kernel ausente junto con razones volumétricas definidas. El ejecutable cronometra lectura, algoritmo y escritura en milisegundos; su suma no incluye el cálculo de estadísticas. Se propone conservar esos tres tiempos y añadir tiempo completo del proceso y tiempo de estadísticas por separado. La memoria se evaluará mediante el máximo residente del proceso y un inventario de contenedores, distinguiendo entrada, salida, auxiliares y copias. Para comparar representaciones, se conservarán idénticos algoritmo, entrada y opciones. Se informará mediana e intervalo intercuartílico temporal por entrada. La calidad se resumirá por malla para no confundir variabilidad de ejecución y geométrica.

Pendiente. Instrumentación: corregir `createStats` y verificar encabezados, separadores y valores; añadir conteos por celda y memoria máxima. Validar orientación, kernel y razones con casos analíticos. Probar un lector del CSV detallado que descarte la columna vacía final y separe estados inválidos.

5.3. Conjunto de entradas y comparadores

La Tabla 8 transcribe exclusivamente el primer campo de los encabezados `.node` y `.ele` locales. Son cantidades declaradas de entrada, no mediciones de salida ni evidencia de validez de los archivos. El nombre de un archivo no se utiliza para inferir su distribución espacial o su procedencia.

Prefijo en `data/`	Vértices	Tetraedros
basic	8	5
minimal	8	5
3D_100	100	535
socket	836	2285
1000points	1000	4374
1000points07	1000	6274
mage	3881	19211
angel	24979	90087

Tabla 8: Inventario declarado en los pares locales de archivos TetGen.

Los casos pequeños servirán para inspección. Los restantes no constituyen una serie de escalabilidad, pues cambian tamaño y geometría. Se propone generar familias con dominio fijo, densidad creciente y semillas registradas; sus tamaños se fijarán mediante un piloto. Como extensión se proponen distribuciones aleatorias, Poisson y regulares en un cubo, junto con entradas con criterios de calidad controlados. P005 emplea esas familias en su evaluación de Face [1]; ello justifica considerarlas como diseño comparativo, sin afirmar que los archivos locales pertenezcan a esas categorías. Se añadirían dominios no convexos y casos de radios iguales, reescalamiento y tetraedros próximos a degeneración para estudiar robustez. Cada entrada nueva deberá incluir origen, licencia, generador, opciones, semilla, encabezados y huella del archivo. Los comparadores serán Face con área máxima y radio inscrito máximo de la cara, según Sección 3.4.1, y Edge una vez resuelto el pendiente documental de Sección 3.3.1. Se evaluará la salida antes y después de reparar únicamente cuando ambas rutas estén verificadas. El programa actual invoca directamente `CavityAlgorithm`; no dispone de una opción de línea de comandos que seleccione Face o Edge. Los resultados publicados del capítulo 3 serán contexto, no filas comparables a mediciones locales realizadas con otro equipo o conjunto de datos.

Pendiente. Entradas y baselines: verificar índices y orientación de los ocho pares, registrar procedencia y huellas; preparar las familias controladas. Obtener la fuente primaria de Edge e integrar adaptadores Face y Edge con idéntico contrato de salida y evaluación. Documentar sus comandos reales solo cuando existan; no hay un comando de selección de variante en la CLI actual.

5.4. Protocolo reproducible

Se propone fijar la revisión del código, las revisiones de dependencias, el compilador, sus opciones de optimización y la configuración de construcción. El registro de hardware contendrá modelo de CPU, núcleos físicos y lógicos, RAM, sistema operativo y almacenamiento. Si se implementa una variante GPU se añadirán dispositivo, memoria, controlador y entorno de compilación. Se propone realizar una ejecución de calentamiento y diez repeticiones medidas por combinación de entrada y variante, cada una en un proceso

independiente. Se alternará el orden de variantes y se mantendrán equipo, estado energético y carga de fondo controlados. Se conservarán todas las observaciones; cualquier exclusión deberá justificarse por una causa registrada. Los fallos y límites de recursos se reportarán aunque impidan completar las repeticiones.

La CLI recibe `-n`, `-e` y `-o`; `--detail-stats` requiere además `--make-stats` para producir detalle. Tras localizar el binario construido y crear el directorio de salida, una ejecución futura desde `3D-GPolylla/` tendrá la siguiente forma. La ruta del binario es la esperada para una construcción CMake con directorio `build`; deberá comprobarse en el equipo final.

```
mkdir -p resultados/basic/r01
build/src/exe/GPolyllaExe \
  -n data/basic.node -e data/basic.ele \
  -o resultados/basic/r01/cavity.visf \
  --make-stats --detail-stats
```

El programa deriva `cavity.csv`, `cavity_detail/details.csv` y archivos OFF por celda del nombre de salida. El tiempo externo incluirá esas exportaciones. Se propone anteponer `/usr/bin/time -l` en macOS o `/usr/bin/time -v` en Linux, verificando las unidades de memoria del equipo. Las ejecuciones sin estadísticas se identificarán como otra configuración.

Pendiente. Campaña reproducible: el autor deberá fijar el equipo, resolver los requisitos de compilación de Sección 4.6 y validar las exportaciones. Después, ejecutar el comando indicado para cada prefijo y repetición, usando carpetas distintas, conservar salida estándar y errores, recursos y manifiesto de versiones. No iniciar la campaña antes de validar entradas y métricas.

5.5. Tablas previstas y criterio de contraste

Las siguientes tablas fijan el formato de presentación. Cada fila se desagregará por malla, variante, criterio y estado de reparación. Todas las celdas de resultados permanecen vacías; no se extrapolan cifras de otros trabajos.

Variante / malla	N_P	R	f_K	Aislados
Cavidades / malla	—	—	—	—
Face área / malla	—	—	—	—
Face radio / malla	—	—	—	—
Edge / malla	—	—	—	—

Tabla 9: Plantilla de reducción y calidad; sin resultados.

Pendiente. Resultado de calidad (Tabla 9): definir, implementar y validar estados de kernel por celda —presente, ausente confirmado, indeterminado y error— y exportarlos en `details.csv`. Tras validar cada variante y su CSV, obtener conteos de pertenencia y contabilizar los cuatro estados exportados. Calcular reducción desde los conteos válidos y adjuntar distribuciones de las tres razones, con mínimos, medianas, máximos y casos excluidos explícitos.

Variante / malla	Lectura	Algoritmo	Escritura	RAM máx.
Cavidades / malla	—	—	—	—
Face / por criterio	—	—	—	—
Edge / malla	—	—	—	—

Tabla 10: Plantilla de recursos: tiempos en ms y memoria en MiB.

Pendiente. Resultado de recursos (Tabla 10): ejecutar la campaña de Sección 5.4, agregar mediana e intervalo intercuartílico por fase y memoria máxima; añadir tiempo completo y de estadísticas en una tabla complementaria. Para límites de tamaño, aumentar densidad dentro de cada familia y registrar el primer fallo, causa y último caso completado.

Configuración futura	Tiempo núcleo	Tiempo completo	Aceleración
Secuencial / misma entrada	—	—	—
Paralela / hilos fijados	—	—	—

Tabla 11: Plantilla condicionada a implementar y validar paralelismo.

Pendiente. Resultado paralelo (Tabla 11): completar el contrato y la implementación de Sección 4.9, comprobar sus invariantes y ejecutar con tamaño fijo y creciente. Calcular aceleración como tiempo secuencial dividido por paralelo bajo el mismo alcance temporal; registrar hilos, rondas, conflictos y transferencias. No calcular aceleración sin una variante válida.

Capítulo 6

Conclusión

6.1. Síntesis y alcance de la contribución

El trabajo documenta una construcción secuencial por cavidades: ordena candidatos por radio de circunsfera y agrupa vecinos por cara según la posición de sus circuncentros respecto de la semilla. Se distingue de las regiones terminales (Sección 3.3) y de las cavidades de inserción (Sección 2.4).

La reparación está inactiva y la integración de Face y Edge permanece pendiente. El diseño paralelo de Sección 4.9 requiere implementación y validación. El protocolo de Sección 5.1 propone verificar partición, calidad y recursos. La hipótesis permanece por contrastar mediante la evaluación propuesta.

6.2. Respuestas a las preguntas de investigación

Pregunta 1: construcción, semillas y refinamiento. El procedimiento está especificado. Bajo conectividad válida y geometría definida, el control de propiedad permite deducir cobertura y exclusividad de índices; el recorrido por vecinos, conectividad por caras (Sección 4.3). Estas propiedades no prueban simplicidad de frontera ni kernel no vacío. Se usa radio creciente sin desempate por índice; no se ha probado su superioridad. El refinamiento permanece abierto, pues la reparación provisional no está activa.

Pendiente. Respuesta experimental a la pregunta 1: validar los invariantes en entradas inspeccionables; comparar órdenes de semillas con desempate fijado y medir celdas aisladas, reducción y kernel. Si se requiere reparación, completar la reconstrucción de fronteras antes de compararla con la salida original.

Pregunta 2: representación y unificación. Los vectores indexados y las listas por poliedro expresan la topología y conservan los tetraedros constituyentes; no demuestran eficiencia frente a otras estructuras. La interfaz común existe, pero faltan adaptadores Face

y Edge. Reducir celdas no implica reducir proporcionalmente el almacenamiento, pues se conserva la subdivisión tetraédrica.

Pendiente. Respuesta cuantitativa a la pregunta 2: completar adaptadores y fijar su contrato; instrumentar memoria de entrada, salida y auxiliares y comparar con una representación aplanada, incluyendo construcción y copias. Completar la tabla de recursos de Sección 5.5 antes de concluir eficiencia.

Pregunta 3: viabilidad, kernel, reducción y límites. No existen mediciones propias que permitan asignar porcentajes o tamaños máximos procesables. Los cálculos y exportaciones requieren las validaciones de Sección 4.7. El inventario de Sección 5.3 identifica entradas disponibles, no casos procesados con éxito. La viabilidad para VEM requiere evaluar el método numérico, sin inferirla de la disponibilidad de estadísticas.

Pendiente. Respuesta a la pregunta 3: completar calidad y recursos por entrada y baseline, registrar fallos y ampliar familias de tamaño controlado. Para sostener aplicabilidad numérica, elegir un problema con solución de referencia y comparar error, convergencia y costo del sistema bajo condiciones comunes.

Pregunta 4: paralelización. La separación de cálculos locales sugiere tareas independientes para circunsferas y una salida aplanada construida por conteos y desplazamientos. Sin embargo, el crecimiento puede competir por un mismo tetraedro, por lo que no basta con ejecutar simultáneamente el recorrido actual. La propuesta por rondas identifica ese conflicto y plantea resolver prioridades, pero no establece todavía equivalencia con el orden secuencial, terminación o aceleración. La respuesta disponible es un diseño por estudiar, no una versión paralela comprobada.

Pendiente. Respuesta a la pregunta 4: acordar equivalencia exacta o invariantes de una partición alternativa; formalizar progreso y arbitraje, implementar casos con competencia y validar reproducibilidad al variar hilos. Después completar Tabla 11 con costos de sincronización y transferencias.

6.3. Trabajo futuro y balance de la hipótesis

Se propone validar primero entradas, orientación, tolerancias y métricas; luego corregir exportaciones, integrar comparadores y ejecutar el protocolo. El paralelismo requiere un contrato de corrección. La contribución actual es el método secuencial documentado; sus ventajas empíricas siguen pendientes. Si no se confirma la hipótesis, se informarán los límites observados.

Bibliografía

[1] S. Salinas-Fernández, M. Álvarez, y N. Hitschfeld-Kahler, «Exploring polyhedral mesh generation from Delaunay tetrahedral meshes», en *SIAM International Meshing Roundtable 2025 - Research Notes*, 2025. [En línea]. Disponible en: https://meshinglab.dcc.uchile.cl/publication/salinas-2025-exploring/

[2] H. Si, «TetGen, a Delaunay-Based Quality Tetrahedral Mesh Generator», *ACM Transactions on Mathematical Software*, 2015, doi: 10.1145/2629697.

[3] S. Salinas-Fernández, N. Hitschfeld, A. Ortiz-Bernardin, y H. Si, «POLYLLA: polygonal meshing algorithm based on terminal-edge regions», *Engineering With Computers*, 2022, doi: 10.1007/s00366-022-01643-4.

[4] L. B. da Veiga, F. Brezzi, A. Cangiani, G. Manzini, L. D. Marini, y A. Russo, «BASIC PRINCIPLES OF VIRTUAL ELEMENT METHODS», *Mathematical Models and Methods in Applied Sciences*, 2012, doi: 10.1142/s0218202512500492.

[5] M. Alvarez, «Generación de mallas de poliedros: optimización y evaluación», Tesis de Magíster, 2024.

[6] L. B. da Veiga, C. Lovadina, y D. Mora, «A Virtual Element Method for elastic and inelastic problems on polytope meshes», *Computer Methods in Applied Mechanics and Engineering*, 2015, doi: 10.1016/j.cma.2015.07.013.

[7] M. Frittelli, A. Madzvamuse, y I. Sgura, «Virtual element method for elliptic bulk-surface PDEs in three space dimensions», *Numerical Methods for Partial Differential Equations*, 2023, doi: 10.1002/num.23040.

[8] L. B. da Veiga, F. Dassi, y A. Russo, «A C1 Virtual Element Method on polyhedral meshes», *Computers & Mathematics with Applications*, 2019, doi: 10.1016/j.camwa.2019.06.019.

[9] T. Sorgente, S. Biasotti, G. Manzini, y M. Spagnuolo, «Polyhedral mesh quality indicator for the Virtual Element Method», *Computers & Mathematics with Applications*, 2022, doi: 10.1016/j.camwa.2022.03.042.

[10] L. B. da Veiga, F. Brezzi, L. D. Marini, y A. Russo, «The Hitchhiker's Guide to the Virtual Element Method», *Mathematical Models and Methods in Applied Sciences*, 2014, doi: 10.1142/s021820251440003x.

[11] A. L. Gain, C. Talischi, y G. H. Paulino, «On the Virtual Element Method for three-dimensional linear elasticity problems on arbitrary polyhedral meshes»,

Computer Methods in Applied Mechanics and Engineering, 2014, doi: 10.1016/j.cma.2014.05.005.

[12] L. B. da Veiga, F. Dassi, y A. Russo, «High-order Virtual Element Method on polyhedral meshes», *Computers & Mathematics with Applications*, 2017, doi: 10.1016/j.camwa.2017.03.021.

[13] L. B. da Veiga, F. Dassi, D. A. D. Pietro, y J. Droniou, «Arbitrary-order pressure--robust DDR and VEM methods for the Stokes problem on polyhedral meshes», *Computer Methods in Applied Mechanics and Engineering*, 2022, doi: 10.1016/j.cma.2022.115061.

[14] O. Čertík, F. Gardini, G. Manzini, y G. Vacca, «The virtual element method for eigenvalue problems with potential terms on polytopic meshes», *Applications of Mathematics*, 2018, doi: 10.21136/am.2018.0093-18.

[15] A. Cangiani, E. H. Georgoulis, T. Pryer, y O. J. Sutton, «A posteriori error estimates for the virtual element method», *Numerische Mathematik*, 2017, doi: 10.1007/s00211-017-0891-9.

[16] D. A. D. Pietro, J. Droniou, y G. Manzini, «Discontinuous Skeletal Gradient Discretisation methods on polytopal meshes», *Journal of Computational Physics*, 2017, doi: 10.1016/j.jcp.2017.11.018.

[17] D. Boffi y D. A. D. Pietro, «Unified formulation and analysis of mixed and primal discontinuous\n skeletal methods on polytopal meshes». [En línea]. Disponible en: https://arxiv.org/pdf/1609.04601

[18] D. A. D. Pietro y J. Droniou, «An arbitrary-order discrete de Rham complex on polyhedral meshes: Exactness, Poincar'e inequalities, and consistency». [En línea]. Disponible en: https://arxiv.org/pdf/2101.04940

[19] L. Mu, J. Wang, y X. Ye, «Weak Galerkin Finite Element Methods on Polytopal Meshes». [En línea]. Disponible en: https://arxiv.org/pdf/1204.3655

[20] Z. Dong, «Discontinuous Galerkin Methods for the Biharmonic Problem on Polygonal and Polyhedral Meshes». [En línea]. Disponible en: https://arxiv.org/pdf/1807.07817

[21] P. F. Antonietti, P. Houston, X. Hu, M. Sarti, y M. Verani, «Multigrid algorithms for hp-version Interior Penalty Discontinuous Galerkin methods on polygonal and polyhedral meshes». [En línea]. Disponible en: https://arxiv.org/pdf/1412.0913

[22] P. F. Antonietti, S. Berrone, M. Busetto, y M. Verani, «Agglomeration-Based Geometric Multigrid Schemes for the Virtual Element Method», *SIAM Journal on Numerical Analysis*, 2023, doi: 10.1137/21m1466864.

[23] Y. Pan y P. Persson, «Agglomeration-based geometric multigrid solvers for compact discontinuous Galerkin discretizations on unstructured meshes», *Journal of Computational Physics*, 2021, doi: 10.1016/j.jcp.2021.110775.

[24] M. Cicuttin, D. A. D. Pietro, y A. Ern, «Implementation of Discontinuous Skeletal methods on arbitrary-dimensional, polytopal meshes using generic programming», *Journal of Computational and Applied Mathematics*, 2017, doi: 10.1016/j.cam.2017.09.017.

[25] S. C. Brenner y L.-yeng Sung, «Virtual element methods on meshes with small edges or faces», *Mathematical Models and Methods in Applied Sciences*, 2018, doi: 10.1142/s0218202518500355.

[26] J. Huang y Y. Yu, «Some Estimates for Virtual Element Methods in Three Dimensions», *Computational Methods in Applied Mathematics*, 2022, doi: 10.1515/cmam-2022-0062.

[27] S. Cao y L. Chen, «Anisotropic Error Estimates of the Linear Virtual Element Method on Polygonal Meshes», *SIAM Journal on Numerical Analysis*, 2018, doi: 10.1137/17m1154369.

[28] S. Cao y L. Chen, «Anisotropic Error Estimates of the Linear Nonconforming Virtual Element Methods», *SIAM Journal on Numerical Analysis*, 2019, doi: 10.1137/18m1196455.

[29] F. Dassi y L. Mascotto, «Exploring high-order three dimensional virtual elements: Bases and stabilizations», *Computers & Mathematics with Applications*, 2018, doi: 10.1016/j.camwa.2018.02.005.

[30] T. Sorgente, F. Vicini, S. Berrone, S. Biasotti, G. Manzini, y M. Spagnuolo, «Mesh optimization for the virtual element method: How small can an agglomerated mesh become?», *Journal of Computational Physics*, 2024, doi: 10.1016/j.jcp.2024.113552.

[31] P. Knupp, «Algebraic Mesh Quality Metrics», *SIAM Journal on Scientific Computing*, 2001, doi: 10.1137/s1064827500371499.

[32] P. Knupp, «Algebraic mesh quality metrics for unstructured initial meshes», *Finite Elements in Analysis and Design*, 2002, doi: 10.1016/s0168-874x(02)00070-7.

[33] P. Knupp, «Achieving finite element mesh quality via optimization of the Jacobian matrix norm and associated quantities. Part II?A framework for volume mesh optimization and the condition number of the Jacobian matrix», *International Journal for Numerical Methods in Engineering*, 2000, doi: 10.1002/(sici)1097-0207(20000720)48:8<1165::aid-nme940>3.0.co;2-y.

[34] P. Alliez, D. Cohen-Steiner, M. Yvinec, y M. Desbrun, «Variational tetrahedral meshing», *ACM Transactions on Graphics*, 2005, doi: 10.1145/1073204.1073238.

[35] L. Freitag y C. Ollivier-Gooch, «Tetrahedral mesh improvement using swapping and smoothing», *International Journal for Numerical Methods in Engineering*, 1997, doi: 10.1002/(sici)1097-0207(19971115)40:21<3979::aid-nme251>3.0.co;2-9.

[36] S.-W. Cheng, T. K. Dey, H. Edelsbrunner, M. A. Facello, y S. Teng, «Sliver exudation», *Journal of the ACM*, 2000, doi: 10.1145/355483.355487.

[37] T. Schneider, Y. Hu, J. Dumas, X. Gao, D. Panozzo, y D. Zorin, «Decoupling simulation accuracy from mesh quality», *ACM Transactions on Graphics*, 2018, doi: 10.1145/3272127.3275067.

[38] T. Schneider, Y. Hu, X. Gao, J. Dumas, D. Zorin, y D. Panozzo, «A Large-Scale Comparison of Tetrahedral and Hexahedral Elements for Solving Elliptic PDEs with the Finite Element Method», *ACM Transactions on Graphics*, 2022, doi: 10.1145/3508372.

[39] T. Sorgente, S. Biasotti, G. Manzini, y M. Spagnuolo, «A Survey of Indicators for Mesh Quality Assessment», *Computer Graphics Forum*, 2023, doi: 10.1111/cgf.14779.

[40] W. Huang y Y. Wang, «Anisotropic mesh quality measures and adaptation for polygonal meshes», *Journal of Computational Physics*, 2020, doi: 10.1016/j.jcp.2020.109368.

[41] T. Sorgente, S. Biasotti, y M. Spagnuolo, «Polyhedron Kernel Computation Using a Geometric Approach». [En línea]. Disponible en: http://arxiv.org/abs/2202.06625

[42] T. Sorgente, F. Vicini, S. Berrone, S. Biasotti, G. Manzini, y M. Spagnuolo, «Mesh quality agglomeration algorithm for the virtual element method applied to discrete fracture networks», *CALCOLO*, 2023, doi: 10.1007/s10092-023-00517-5.

[43] J. R. Shewchuk, «General-Dimensional Constrained Delaunay and Constrained Regular Triangulations, I: Combinatorial Properties», *Discrete & Computational Geometry*, 2008, doi: 10.1007/s00454-008-9060-3.

[44] H. Si, «Constrained Delaunay tetrahedral mesh generation and refinement», *Finite Elements in Analysis and Design*, 2009, doi: 10.1016/j.finel.2009.06.017.

[45] D. A. Spielman, S. Teng, y A. Üngör, «Parallel Delaunay Refinement: Algorithms and Analyses». [En línea]. Disponible en: http://arxiv.org/abs/cs/0207063

[46] Y. Yang, J. Yong, y J. Sun, «An algorithm for tetrahedral mesh generation based on conforming constrained Delaunay tetrahedralization», *Computers & Graphics*, 2005, doi: 10.1016/j.cag.2005.05.011.

[47] L. Diazzi, D. Panozzo, A. Vaxman, y M. Attene, «Constrained Delaunay Tetrahedrization: A Robust and Practical Approach», *ACM Transactions on Graphics*, 2023, doi: 10.1145/3618352.

[48] Y. S. Elshakhs, K. M. Deliparaschos, T. Charalambous, G. Oliva, y A. Zolotas, «A Comprehensive Survey on Delaunay Triangulation: Applications, Algorithms, and Implementations Over CPUs, GPUs, and FPGAs», *IEEE Access*, 2024, doi: 10.1109/access.2024.3354709.

[49] R. Garimella, J. Kim, y M. Berndt, «Polyhedral Mesh Generation and Optimization for Non-manifold Domains», 2013. doi: 10.1007/978-3-319-02335-9_18.

[50] D. Engwirda, «JIGSAW-GEO (1.0): locally orthogonal staggered unstructured grid generation for general circulation modelling on the sphere», *Geoscientific model development*, 2017, doi: 10.5194/gmd-10-2117-2017.

[51] A. Abdelkader *et al.*, «VoroCrust», *ACM Transactions on Graphics*, 2020, doi: 10.1145/3337680.

[52] E. VanderZee, A. N. Hirani, D. Guoy, y E. A. Ramos, «Well-Centered Triangulation», *SIAM Journal on Scientific Computing*, 2010, doi: 10.1137/090748214.

[53] J. Guo, D. Yan, L. Chen, X. Zhang, O. Deußen, y P. Wonka, «Tetrahedral meshing via maximal Poisson-disk sampling», *Computer Aided Geometric Design*, 2016, doi: 10.1016/j.cagd.2016.02.004.

[54] L. Feng, P. Alliez, L. Busé, H. Delingette, y M. Desbrun, «Curved optimal delaunay triangulation», *ACM Transactions on Graphics*, 2018, doi: 10.1145/3197517.3201358.

[55] S.-W. Cheng y S.-H. Poon, «Three-Dimensional Delaunay Mesh Generation», *Discrete & Computational Geometry*, 2006, doi: 10.1007/s00454-006-1252-5.

[56] A. Bowyer, «Computing Dirichlet tessellations», *The Computer Journal*, 1981, doi: 10.1093/comjnl/24.2.162.

[57] D. F. Watson, «Computing the n-dimensional Delaunay tessellation with application to Voronoi polytopes», *The Computer Journal*, 1981, doi: 10.1093/comjnl/24.2.167.

[58] C. Marot, J. Pellerin, y J. Remacle, «One machine, one minute, three billion tetrahedra», *International Journal for Numerical Methods in Engineering*, 2018, doi: 10.1002/nme.5987.

[59] L. Guibas, D. E. Knuth, y M. Sharir, «Randomized incremental construction of Delaunay and Voronoi diagrams», *Algorithmica*, 1992, doi: 10.1007/bf01758770.

[60] D. J. Mavriplis, «An Advancing Front Delaunay Triangulation Algorithm Designed for Robustness», *Journal of Computational Physics*, 1995, doi: 10.1006/jcph.1995.1047.

[61] Z. Chen y T.-S. Tan, «Computing Three-Dimensional Constrained Delaunay Refinement Using the GPU», en *2019 28th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2019. doi: 10.1109/pact.2019.00039.

[62] F. Labelle y J. R. Shewchuk, «Isosurface stuffing», *ACM Transactions on Graphics*, 2007, doi: 10.1145/1276377.1276448.

[63] J. R. Shewchuk, «Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates», *Discrete & Computational Geometry*, 1997, doi: 10.1007/pl00009321.

[64] C. Geuzaine y J. Remacle, «Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities», *International Journal for Numerical Methods in Engineering*, 2009, doi: 10.1002/nme.2579.

[65] J. Boissonnat, K. Shi, J. Tournois, y M. Yvinec, «Anisotropic Delaunay Meshes of Surfaces», *ACM Transactions on Graphics*, 2015, doi: 10.1145/2721895.

[66] J. Schöberl, «NETGEN An advancing front 2D/3D-mesh generator based on abstract rules», *Computing and Visualization in Science*, 1997, doi: 10.1007/s007910050004.

[67] P. Alliez, C. Jamin, L. Rineau, S. phane Tayeb, J. Tournois, y M. Yvinec, «3D Mesh Generation», en *CGAL User and Reference Manual*, 5.6., CGAL Editorial Board,

2023. [En línea]. Disponible en: https://doc.cgal.org/5.6/Manual/packages.html#PkgMesh3

[68] Q. Du y D. Wang, «Tetrahedral mesh generation and optimization based on centroidal Voronoi tessellations», *International Journal for Numerical Methods in Engineering*, 2003, doi: 10.1002/nme.616.

[69] Y. Hu, Q. Zhou, X. Gao, A. Jacobson, D. Zorin, y D. Panozzo, «Tetrahedral meshing in the wild», *ACM Transactions on Graphics*, 2018, doi: 10.1145/3197517.3201353.

[70] Y. Hu, T. Schneider, B. Wang, D. Zorin, y D. Panozzo, «Fast tetrahedral meshing in the wild», *ACM Transactions on Graphics*, 2020, doi: 10.1145/3386569.3392385.

[71] C. Marot y J. Remacle, «Quality tetrahedral mesh generation with HXT». [En línea]. Disponible en: http://arxiv.org/abs/2008.08508

[72] F. Dassi, L. Kamenski, P. Farrell, y H. Si, «Tetrahedral mesh improvement using moving mesh smoothing, lazy searching flips, and RBF surface reconstruction», *Computer-Aided Design*, 2017, doi: 10.1016/j.cad.2017.11.010.

[73] J. Tournois, C. Wormser, P. Alliez, y M. Desbrun, «Interleaving Delaunay refinement and optimization for practical isotropic tetrahedron mesh generation», *ACM Transactions on Graphics*, 2009, doi: 10.1145/1531326.1531381.

[74] J. P. D'Amato y M. Vénere, «A CPU–GPU framework for optimizing the quality of large meshes», *Journal of Parallel and Distributed Computing*, 2013, doi: 10.1016/j.jpdc.2013.03.007.

[75] D. Ströter, J. S. Mueller-Roemer, D. Weber, y D. W. Fellner, «Fast harmonic tetrahedral mesh optimization», *The Visual Computer*, 2022, doi: 10.1007/s00371-022-02547-6.

[76] N. Xi, Y. Sun, L. Xiao, y G. Mei, «Designing Parallel Adaptive Laplacian Smoothing for Improving Tetrahedral Mesh Quality on the GPU», *Applied Sciences*, 2021, doi: 10.3390/app11125543.

[77] T. C. Baudouin, J. Remacle, É. Marchandise, F. Henrotte, y C. Geuzaine, «A frontal approach to hex-dominant mesh generation», *Advanced Modeling and Simulation in Engineering Sciences*, 2014, doi: 10.1186/2213-7467-1-8.

[78] D. Yan, W. Wang, B. Lévy, y Y. Liu, «Efficient computation of clipped Voronoi diagram for mesh generation», *Computer-Aided Design*, 2011, doi: 10.1016/j.cad.2011.09.004.

[79] A. Abdelkader *et al.*, «Sampling Conditions for Conforming Voronoi Meshing by the VoroCrust Algorithm», *Leibniz International Proceedings in Informatics (SoCG 2018)*, 2018, doi: 10.4230/lipics.socg.2018.1.

[80] Y. Liu, A. A. Saputra, J. Wang, F. Tin-Loi, y C. Song, «Automatic polyhedral mesh generation and scaled boundary finite element analysis of STL models», *Computer Methods in Applied Mechanics and Engineering*, 2016, doi: 10.1016/j.cma.2016.09.038.

[81] S. Ya, S. Eisenträger, C. Song, y J. Li, «An open-source ABAQUS implementation of the scaled boundary finite element method to study interfacial problems using polyhedral meshes», *Computer Methods in Applied Mechanics and Engineering*, 2021, doi: 10.1016/j.cma.2021.113766.

[82] S. Y. Lee, «Polyhedral Mesh Generation and A Treatise on Concave Geometrical Edges», en *Procedia Engineering*, 2015. doi: 10.1016/j.proeng.2015.10.131.

[83] L. Diazzi y M. Attene, «Convex polyhedral meshing for robust solid modeling», *ACM Transactions on Graphics*, 2021, doi: 10.1145/3478513.3480564.

[84] S. J. Owen y S. Saigal, «H-Morph: an indirect approach to advancing front hex meshing», *International Journal for Numerical Methods in Engineering*, 2000, doi: 10.1002/1097-0207(20000910/20)49:1/2<289::aid-nme934>3.0.co;2-l.

[85] P. F. Antonietti, M. Corti, y G. Martinelli, «Polytopal mesh agglomeration via geometrical deep learning for three-dimensional heterogeneous domains». [En línea]. Disponible en: http://arxiv.org/abs/2406.10587

[86] C. Fu, M. Cremonesi, U. Perego, B. Hudobivnik, y P. Wriggers, «Particle Virtual Element Method (PVEM): an agglomeration technique for mesh optimization in explicit Lagrangian free-surface fluid modelling», *Computer Methods in Applied Mechanics and Engineering*, 2024, doi: 10.1016/j.cma.2024.117461.

[87] D. Yan, B. Lévy, Y. Liu, F. Sun, y W. Wang, «Isotropic Remeshing with Fast and Exact Computation of Restricted Voronoi Diagram», *Computer Graphics Forum*, 2009, doi: 10.1111/j.1467-8659.2009.01521.x.

[88] J. Zhang, A. Ankit, H. Gravenkamp, S. Eisenträger, y C. Song, «A massively parallel explicit solver for elasto-dynamic problems exploiting octree meshes», *Computer Methods in Applied Mechanics and Engineering*, 2021, doi: 10.1016/j.cma.2021.113811.

[89] D. R. Bukenberger, M. Tarini, y H. P. A. Lensch, «At-Most-Hexa Meshes», *Computer Graphics Forum*, 2021, doi: 10.1111/cgf.14393.

[90] P. F. Antonietti, N. Farenga, E. Manuzzi, G. Martinelli, y L. Saverio, «Agglomeration of polygonal grids using graph neural networks with applications to multigrid solvers», *Computers & Mathematics with Applications*, 2023, doi: 10.1016/j.camwa.2023.11.015.

[91] N. Sukumar y M. Tupek, «Virtual elements on agglomerated finite elements to increase the critical time step in elastodynamic simulations», *International Journal for Numerical Methods in Engineering*, 2022, doi: 10.1002/nme.7052.

[92] M.-C. Rivara, «New longest-edge algorithms for the refinement and/or improvement of unstructured triangulations», *International journal for numerical methods in Engineering*, 1997.

[93] S. Salinas, J. Ojeda, N. Hitschfeld, y A. Ortiz-Bernardin, «GPU Parallel algorithm for the generation of polygonal meshes based on terminal-edge regions». [En línea]. Disponible en: http://arxiv.org/abs/2204.05438

[94] S. Salinas-Fernández, J. Fuentes-Sepúlveda, y N. Hitschfeld, «Generation of Polygonal Meshes in Compact Space», en *Lecture notes in computational science and engineering*, 2024. doi: 10.1007/978-3-031-40594-5_1.

[95] S. Salinas-Fernández y N. Hitschfeld, «3D POLYLLA: Polyhedral meshing algorithm based on terminal-edge regions and terminal-face regions». [En línea]. Disponible en: http://arxiv.org/abs/2310.03665

[96] R. Garimella, «Mesh data structure selection for mesh generation and FEA applications», *International Journal for Numerical Methods in Engineering*, 2002, doi: 10.1002/nme.509.

[97] A. H. Mahmoud, S. D. Porumbescu, y J. D. Owens, «RXMesh», *ACM Transactions on Graphics*, 2021, doi: 10.1145/3450626.3459748.

[98] N. Ray, I. Grindeanu, X. Zhao, V. Mahadevan, y X. Jiao, «Array-based, parallel hierarchical mesh refinement algorithms for unstructured meshes», *Computer-Aided Design*, 2016, doi: 10.1016/j.cad.2016.07.011.

[99] C. A. Navarro, N. Hitschfeld, y L. Mateu, «A Survey on Parallel Computing and Its Applications in Data-Parallel Problems Using GPU Architectures», *Communications in Computational Physics*, 2013, doi: 10.4208/cicp.110113.010813a.

[100] G. E. Blelloch, G. L. Miller, J. C. Hardwick, y D. Talmor, «Design and Implementation of a Practical Parallel Delaunay Algorithm», *Algorithmica*, 1999, doi: 10.1007/pl00008262.

[101] N. Chrisochoides y D. Nave, «Parallel Delaunay mesh generation kernel», *International Journal for Numerical Methods in Engineering*, 2003, doi: 10.1002/nme.765.

[102] D. Nave, N. Chrisochoides, y L. P. Chew, «Guaranteed-quality parallel Delaunay refinement for restricted polyhedral domains», *Computational Geometry*, 2004, doi: 10.1016/j.comgeo.2004.03.009.

[103] L. Linardakis y N. Chrisochoides, «Delaunay Decoupling Method for Parallel Guaranteed Quality Planar Mesh Refinement», *SIAM Journal on Scientific Computing*, 2006, doi: 10.1137/030602812.

[104] S. Lo, «Parallel Delaunay triangulation in three dimensions», *Computer Methods in Applied Mechanics and Engineering*, 2012, doi: 10.1016/j.cma.2012.05.009.

[105] Y. Ito *et al.*, «Parallel unstructured mesh generation by an advancing front method», *Mathematics and Computers in Simulation*, 2007, doi: 10.1016/j.matcom.2006.12.008.

[106] J. Chen, D. Zhao, Z. Huang, Y. Zheng, y D. Wang, «Improvements in the reliability and element quality of parallel tetrahedral mesh generation», *International Journal for Numerical Methods in Engineering*, 2012, doi: 10.1002/nme.4345.

[107] F. Yu, Y. Zeng, Z. Guan, y S. Lo, «A robust Delaunay-AFT based parallel method for the generation of large-scale fully constrained meshes», *Computers & Structures*, 2019, doi: 10.1016/j.compstruc.2019.106170.

[108] X. Chen, L. Chen, L. Chen, y M. Shi, «A highly solid model boundary preserving method for large-scale parallel 3D Delaunay meshing on parallel computers», *Computer-Aided Design*, 2014, doi: 10.1016/j.cad.2014.08.013.

[109] A. Loseille, F. Alauzet, y V. Menier, «Unique cavity-based operator and hierarchical domain partitioning for fast parallel generation of anisotropic meshes», *Computer-Aided Design*, 2016, doi: 10.1016/j.cad.2016.09.008.

[110] J. Kohout, I. Kolingerová, y J. Žára, «Parallel Delaunay triangulation in E2 and E3 for computers with shared memory», *Parallel Computing*, 2005, doi: 10.1016/j.parco.2005.02.010.

[111] C. Ma, J. Huang, H. Luo, y C. Yang, «CPAFT: A consistent parallel advancing front technique for unstructured triangular/tetrahedral mesh generation», *Computer Physics Communications*, 2025, doi: 10.1016/j.cpc.2025.109535.

[112] N. Coll y M. Guerrieri, «Parallel constrained Delaunay triangulation on the GPU», *International Journal of Geographical Information Systems*, 2017, doi: 10.1080/13658816.2017.1300804.

[113] Q. Zhou, Q. Wang, y Z. Yu, «SAFT: Shotgun advancing front technique for massively parallel mesh generation on graphics processing unit», *International Journal for Numerical Methods in Engineering*, 2022, doi: 10.1002/nme.7038.

[114] L. Oliker, R. Biswas, y H. N. Gabow, «Parallel tetrahedral mesh adaptation with dynamic load balancing», *Parallel Computing*, 2000, doi: 10.1016/s0167-8191(00)00047-8.

[115] M. Qi, K. Yan, y Y. Zheng, «GPredicates: GPU Implementation of Robust and Adaptive Floating-Point Predicates for Computational Geometry», *IEEE Access*, 2019, doi: 10.1109/access.2019.2911641.

[116] W. Gao y R. Chen, «Parallel 3D Delaunay Triangulation on the GPU», *Computer-Aided Design*, 2025, doi: 10.1016/j.cad.2025.103933.

[117] D. Mlakar, M. Winter, P. Stadlbauer, H. Seidel, M. Steinberger, y R. Zayer, «Subdivision-Specialized Linear Algebra Kernels for Static and Dynamic Mesh Connectivity on the GPU», *Computer Graphics Forum*, 2020, doi: 10.1111/cgf.13934.

[118] N. Morrical, A. Sahistan, U. Güdükbay, I. Wald, y V. Pascucci, «Quick Clusters: A GPU-Parallel Partitioning for Efficient Path Tracing of Unstructured Volumetric Grids», *IEEE Transactions on Visualization and Computer Graphics*, 2022, doi: 10.1109/tvcg.2022.3209418.